

Squirrel

Generated by Doxygen 1.9.8

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 BufState Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 buf	5
3.1.2.2 ptr	6
3.1.2.3 size	6
4 File Documentation	7
4.1 squirrel3_squirrel_sqapiByOpenAI.cpp File Reference	7
4.1.1 Macro Definition Documentation	14
4.1.1.1 _GETSAFE_OBJ	14
4.1.1.2 sq_aux_paramscheck	14
4.1.2 Function Documentation	15
4.1.2.1 _getmemberbyhandle()	15
4.1.2.2 buf_lexfeed()	16
4.1.2.3 sq_addrref()	17
4.1.2.4 sq_arrayappend()	18
4.1.2.5 sq_arrayinsert()	18
4.1.2.6 sq_arraypop()	19
4.1.2.7 sq_arrayremove()	20
4.1.2.8 sq_arrayresize()	21
4.1.2.9 sq_arrayreverse()	22
4.1.2.10 sq_aux_gettypedarg()	22
4.1.2.11 sq_aux_invalidtype()	23
4.1.2.12 sq_bindenv()	24
4.1.2.13 sq_call()	25
4.1.2.14 sq_clear()	26
4.1.2.15 sq_clone()	27
4.1.2.16 sq_close()	28
4.1.2.17 sq_cmp()	28
4.1.2.18 sq_collectgarbage()	29
4.1.2.19 sq_compile()	29
4.1.2.20 sq_compilebuffer()	30
4.1.2.21 sq_createinstance()	31
4.1.2.22 sq_deleteslot()	32
4.1.2.23 sq_enableddebuginfo()	33

4.1.2.24 sq_free()	33
4.1.2.25 sq_get()	34
4.1.2.26 sq_getattributes()	34
4.1.2.27 sq_getbase()	35
4.1.2.28 sq_getbool()	36
4.1.2.29 sq_getbyhandle()	36
4.1.2.30 sq_getcallee()	37
4.1.2.31 sq_getclass()	38
4.1.2.32 sq_getclosureinfo()	38
4.1.2.33 sq_getclosurename()	39
4.1.2.34 sq_getclouseroot()	40
4.1.2.35 sq_getdefaultdelegate()	41
4.1.2.36 sq_getdelegate()	42
4.1.2.37 sq_geterrorfunc()	43
4.1.2.38 sq_getfloat()	43
4.1.2.39 sq_getforeignptr()	44
4.1.2.40 sq_getfreevariable()	44
4.1.2.41 sq_gethash()	45
4.1.2.42 sq_getinstanceup()	46
4.1.2.43 sq_getinteger()	47
4.1.2.44 sq_getlasterror()	47
4.1.2.45 sq_getlocal()	48
4.1.2.46 sq_getmemberhandle()	49
4.1.2.47 sq_getobjtypetag()	49
4.1.2.48 sq_getprintfunc()	50
4.1.2.49 sq_getrefcount()	51
4.1.2.50 sq_getreleasehook()	51
4.1.2.51 sq_getscratchpad()	52
4.1.2.52 sq_getsharedforeignptr()	52
4.1.2.53 sq_getsharedreleasehook()	53
4.1.2.54 sq_getsize()	53
4.1.2.55 sq_getstackobj()	54
4.1.2.56 sq_getstring()	54
4.1.2.57 sq_getstringandsize()	55
4.1.2.58 sq_getthread()	56
4.1.2.59 sq_gettop()	56
4.1.2.60 sq_gettype()	57
4.1.2.61 sq_gettypetag()	58
4.1.2.62 sq_getuserdata()	58
4.1.2.63 sq_getuserpointer()	60
4.1.2.64 sq_getversion()	61
4.1.2.65 sq_getvmrefcount()	61

4.1.2.66 sq_getvmreleasehook()	61
4.1.2.67 sq_getvmstate()	62
4.1.2.68 sq_getweakrefval()	62
4.1.2.69 sq_instanceof()	63
4.1.2.70 sq_malloc()	64
4.1.2.71 sq_move()	64
4.1.2.72 sq_newarray()	65
4.1.2.73 sq_newclass()	65
4.1.2.74 sq_newclosure()	66
4.1.2.75 sq_newmember()	67
4.1.2.76 sq_newslot()	68
4.1.2.77 sq_newtable()	68
4.1.2.78 sq_newtableex()	69
4.1.2.79 sq_newthread()	69
4.1.2.80 sq_newuserdata()	70
4.1.2.81 sq_next()	70
4.1.2.82 sq_notifyallexceptions()	71
4.1.2.83 sq_objtobool()	72
4.1.2.84 sq_objtofloat()	72
4.1.2.85 sq_objtointeger()	72
4.1.2.86 sq_objtostring()	73
4.1.2.87 sq_objtouserpointer()	73
4.1.2.88 sq_open()	74
4.1.2.89 sq_pop()	74
4.1.2.90 sq_poptop()	76
4.1.2.91 sq_push()	76
4.1.2.92 sq_pushbool()	77
4.1.2.93 sq_pushconsttable()	77
4.1.2.94 sq_pushfloat()	77
4.1.2.95 sq_pushinteger()	78
4.1.2.96 sq_pushnull()	78
4.1.2.97 sq_pushobject()	79
4.1.2.98 sq_pushregistrytable()	79
4.1.2.99 sq_pushroottable()	80
4.1.2.100 sq_pushstring()	80
4.1.2.101 sq_pushthread()	80
4.1.2.102 sq_pushuserpointer()	81
4.1.2.103 sq_rawdeleteslot()	81
4.1.2.104 sq_rawget()	82
4.1.2.105 sq_rawnewmember()	83
4.1.2.106 sq_rawset()	84
4.1.2.107 sq_readclosure()	85

4.1.2.108 <code>sq_realloc()</code>	86
4.1.2.109 <code>sq_release()</code>	87
4.1.2.110 <code>sq_remove()</code>	87
4.1.2.111 <code>sq_reservestack()</code>	88
4.1.2.112 <code>sq_reseterror()</code>	88
4.1.2.113 <code>sq_resetobject()</code>	90
4.1.2.114 <code>sq_resume()</code>	90
4.1.2.115 <code>sq_resurrectunreachable()</code>	91
4.1.2.116 <code>sq_set()</code>	92
4.1.2.117 <code>sq_setattributes()</code>	92
4.1.2.118 <code>sq_setbyhandle()</code>	93
4.1.2.119 <code>sq_setclassudsize()</code>	94
4.1.2.120 <code>sq_setclosureroot()</code>	95
4.1.2.121 <code>sq_setcompilererrorhandler()</code>	96
4.1.2.122 <code>sq_setconsttable()</code>	96
4.1.2.123 <code>sq_setdebughook()</code>	97
4.1.2.124 <code>sq_setdelegate()</code>	98
4.1.2.125 <code>sq_seterrorhandler()</code>	99
4.1.2.126 <code>sq_setforeignptr()</code>	99
4.1.2.127 <code>sq_setfreevariable()</code>	100
4.1.2.128 <code>sq_setinstanceup()</code>	101
4.1.2.129 <code>sq_setnativeclosurename()</code>	102
4.1.2.130 <code>sq_setnativedebughook()</code>	102
4.1.2.131 <code>sq_setparamscheck()</code>	103
4.1.2.132 <code>sq_setprintfunc()</code>	104
4.1.2.133 <code>sq_setreleasehook()</code>	104
4.1.2.134 <code>sq_setroottable()</code>	105
4.1.2.135 <code>sq_setsharedforeignptr()</code>	106
4.1.2.136 <code>sq_setsharedreleasehook()</code>	106
4.1.2.137 <code>sq_settop()</code>	107
4.1.2.138 <code>sq_settypetag()</code>	107
4.1.2.139 <code>sq_setvmreleasehook()</code>	109
4.1.2.140 <code>sq_suspendvm()</code>	110
4.1.2.141 <code>sq_tailcall()</code>	110
4.1.2.142 <code>sq_throwerror()</code>	111
4.1.2.143 <code>sq_throwobject()</code>	113
4.1.2.144 <code>sq_tobool()</code>	113
4.1.2.145 <code>sq_tostring()</code>	113
4.1.2.146 <code>sq_typeof()</code>	114
4.1.2.147 <code>sq_wakeupvm()</code>	115
4.1.2.148 <code>sq_weakref()</code>	116
4.1.2.149 <code>sq_writeclosure()</code>	116

4.2 squirrel3_squirrel_sqapiByOpenAI.cpp	117
--	-----

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

BufState	バッファの状態を保持する構造体。	5
--------------------------	----------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

[squirrel3_squirrel_sqapiByOpenAI.cpp](#) 7

Chapter 3

Data Structure Documentation

3.1 BufState Struct Reference

バッファの状態を保持する構造体。

Data Fields

- `const SQChar * buf`
- `SQInteger ptr`
- `SQInteger size`

3.1.1 Detailed Description

バッファの状態を保持する構造体。

スクリプトバッファを解析する際に使用する読み込み位置などの情報を保持する。 `buf` は読み込む文字列、`ptr` は現在位置、`size` はバッファサイズを示す。

Definition at line [3282](#) of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

3.1.2 Field Documentation

3.1.2.1 buf

```
const SQChar* BufState::buf
```

Definition at line [3283](#) of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

3.1.2.2 ptr

`SQInteger BufState::ptr`

Definition at line 3284 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

3.1.2.3 size

`SQInteger BufState::size`

Definition at line 3285 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

Referenced by [buf_lexfeed\(\)](#), and [sq_compilebuffer\(\)](#).

The documentation for this struct was generated from the following file:

- [squirrel3_squirrel_sqapiByOpenAI.cpp](#)

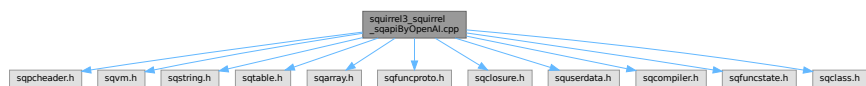
Chapter 4

File Documentation

4.1 squirrel3_squirrel_sqapiByOpenAl.cpp File Reference

```
#include "sqpcheader.h"
#include "sqvm.h"
#include "sqstring.h"
#include "sqtable.h"
#include "sqarray.h"
#include "sqfuncproto.h"
#include "sqclosure.h"
#include "squserdata.h"
#include "sqcompiler.h"
#include "sqfuncstate.h"
#include "sqclass.h"
```

Include dependency graph for squirrel3_squirrel_sqapiByOpenAl.cpp:



Data Structures

- struct [BufState](#)
バッファの状態を保持する構造体。

Macros

- #define [_GETSAFE_OBJ](#)(v, idx, type, o) { if(![sq_aux_gettypedarg](#)(v,idx,type,&o)) return SQ_ERROR; }
指定型のオブジェクトをスタックから安全に取得する。
- #define [sq_aux_paramscheck](#)(v, count)
スタックに十分な数の引数があるかを検査する。

Functions

- static bool [sq_aux_gettypedarg](#) (HSQUIRRELVm v, SQInteger idx, SQObjectType type, SQObjectPtr **o)
引数の型をチェックし、指定の型と一致していれば対象オブジェクトを取得する。
- SQInteger [sq_aux_invalidtype](#) (HSQUIRRELVm v, SQObjectType type)
想定外のオブジェクト型が指定された場合のエラーを返す。
- HSQUIRRELVm [sq_open](#) (SQInteger initialstacksize)
新しい *Squirrel* 仮想マシン (VM) インスタンスを作成し初期化する。
- HSQUIRRELVm [sq_newthread](#) (HSQUIRRELVm friendvm, SQInteger initialstacksize)
新しいスレッド VM (子VM) を生成する。
- SQInteger [sq_getvmstate](#) (HSQUIRRELVm v)
仮想マシン (VM) の現在の状態を取得する。
- void [sq_seterrorhandler](#) (HSQUIRRELVm v)
仮想マシンにエラーハンドラを設定する。
- void [sq_setnatividebughook](#) (HSQUIRRELVm v, SQDEBUGHOOK hook)
ネイティブのデバッグフック関数を設定する。
- void [sq_setdebughook](#) (HSQUIRRELVm v)
スクリプトレベルのデバッグフッククロージャを設定する。
- void [sq_close](#) (HSQUIRRELVm v)
仮想マシンを終了し、関連するリソースを解放する。
- SQInteger [sq_getversion](#) ()
Squirrel 仮想マシンのバージョン番号を取得する。
- SQRESULT [sq_compile](#) (HSQUIRRELVm v, SQLEXREADFUNC read, SQUserPointer p, const SQChar *sourcename, SQBool raiseerror)
スクリプトソースをコンパイルして、実行可能なクロージャをスタックにプッシュする。
- void [sq_enableddebuginfo](#) (HSQUIRRELVm v, SQBool enable)
コンパイル時にデバッグ情報を埋め込むかどうかを設定する。
- void [sq_notifyallexceptions](#) (HSQUIRRELVm v, SQBool enable)
すべての例外に対して通知を行うかどうかを設定する。
- void [sq_addref](#) (HSQUIRRELVm v, HSQOBJECT *po)
指定されたオブジェクトの参照カウントを 1 増加させる。
- SQUnsignedInteger [sq_getrefcount](#) (HSQUIRRELVm v, HSQOBJECT *po)
指定されたオブジェクトの参照カウントを取得する。
- SQBool [sq_release](#) (HSQUIRRELVm v, HSQOBJECT *po)
指定されたオブジェクトの参照カウントを減少させ、必要に応じて解放する。
- SQUnsignedInteger [sq_getvmrefcount](#) (HSQUIRRELVm v, const HSQOBJECT *po)
指定されたオブジェクトの参照カウント値を直接取得する (VM 外の補助関数)。
- const SQChar * [sq_objtostring](#) (const HSQOBJECT *o)
オブジェクトを文字列型として取得する。
- SQInteger [sq_objtointeger](#) (const HSQOBJECT *o)
オブジェクトを整数 (*SQInteger*) として取得する。
- SQFloat [sq_objtfloat](#) (const HSQOBJECT *o)
オブジェクトを浮動小数点数 (*SQFloat*) として取得する。
- SQBool [sq_objtobool](#) (const HSQOBJECT *o)
オブジェクトをブール値 (*SQBool*) として取得する。
- SQUserPointer [sq_objtouserpointer](#) (const HSQOBJECT *o)
オブジェクトをユーザーポインタとして取得する。
- void [sq_pushnull](#) (HSQUIRRELVm v)
スタックに *null* 値をプッシュする。
- void [sq_pushstring](#) (HSQUIRRELVm v, const SQChar *s, SQInteger len)
スタックに文字列をプッシュする。

- void [sq_pushinteger](#) (HSQUIRRELM v, SQInteger n)
スタックに整数値をプッシュする。
- void [sq_pushbool](#) (HSQUIRRELM v, SQBool b)
スタックにブール値をプッシュする。
- void [sq_pushfloat](#) (HSQUIRRELM v, SQFloat n)
スタックに浮動小数点数をプッシュする。
- void [sq_pushuserpointer](#) (HSQUIRRELM v, SQUserPointer p)
スタックにユーザーポインタをプッシュする。
- void [sq_pushthread](#) (HSQUIRRELM v, HSQUIRRELM thread)
スレッド (VM インスタンス) をスタックにプッシュする。
- SQUserPointer [sq_newuserdata](#) (HSQUIRRELM v, SQUnsignedInteger size)
新しいユーザーデータ領域を確保してスタックにプッシュする。
- void [sq_newtable](#) (HSQUIRRELM v)
新しい空のテーブルを生成してスタックにプッシュする。
- void [sq_newtableex](#) (HSQUIRRELM v, SQInteger initialcapacity)
指定した初期容量で新しいテーブルを生成し、スタックにプッシュする。
- void [sq_newarray](#) (HSQUIRRELM v, SQInteger size)
指定されたサイズの配列を生成し、スタックにプッシュする。
- SQRESULT [sq_newclass](#) (HSQUIRRELM v, SQBool hasbase)
新しいクラスを生成し、スタックにプッシュする。
- SQBool [sq_instanceof](#) (HSQUIRRELM v)
オブジェクトが指定されたクラスのインスタンスかどうかを判定する。
- SQRESULT [sq_arrayappend](#) (HSQUIRRELM v, SQInteger idx)
指定した配列の末尾に要素を追加する。
- SQRESULT [sq_arraypop](#) (HSQUIRRELM v, SQInteger idx, SQBool pushval)
指定した配列の末尾の要素を削除する。
- SQRESULT [sq_arrayresize](#) (HSQUIRRELM v, SQInteger idx, SQInteger newsize)
指定した配列のサイズを変更する。
- SQRESULT [sq_arrayreverse](#) (HSQUIRRELM v, SQInteger idx)
配列内の要素の順序を反転する。
- SQRESULT [sq_arrayremove](#) (HSQUIRRELM v, SQInteger idx, SQInteger itemidx)
配列から指定されたインデックスの要素を削除する。
- SQRESULT [sq_arrayinsert](#) (HSQUIRRELM v, SQInteger idx, SQInteger destpos)
配列の指定位置に要素を挿入する。
- void [sq_newclosure](#) (HSQUIRRELM v, SQFUNCTION func, SQUnsignedInteger nfreevars)
新しいネイティブクロージャを作成してスタックにプッシュする。
- SQRESULT [sq_getclosureinfo](#) (HSQUIRRELM v, SQInteger idx, SQInteger *nparams, SQInteger *nfreevars)
クロージャの引数数および自由変数数を取得する。
- SQRESULT [sq_setnativeclosurename](#) (HSQUIRRELM v, SQInteger idx, const SQChar *name)
ネイティブクロージャに名前を設定する。
- SQRESULT [sq_setparamscheck](#) (HSQUIRRELM v, SQInteger nparamscheck, const SQChar *typemask)
ネイティブクロージャの引数チェックルールを設定する。
- SQRESULT [sq_bindenv](#) (HSQUIRRELM v, SQInteger idx)
クロージャに環境オブジェクト (*env*) をバインドする。
- SQRESULT [sq_getclosurename](#) (HSQUIRRELM v, SQInteger idx)
クロージャに設定されている名前を取得する。
- SQRESULT [sq_setclosureroot](#) (HSQUIRRELM v, SQInteger idx)
クロージャにルートテーブルを設定する。
- SQRESULT [sq_getclosureroot](#) (HSQUIRRELM v, SQInteger idx)
クロージャに設定されているルートテーブルを取得する。

- SQRESULT [sq_clear](#) (HSQUIRRELM v, SQInteger idx)
テーブルまたは配列の内容をすべて削除する。
- void [sq_pushroottable](#) (HSQUIRRELM v)
現在の仮想マシンに設定されているルートテーブルをスタックにプッシュする。
- void [sq_pushregistrytable](#) (HSQUIRRELM v)
レジストリテーブルをスタックにプッシュする。
- void [sq_pushconsttable](#) (HSQUIRRELM v)
定数 (*const*) テーブルをスタックにプッシュする。
- SQRESULT [sq_setroottable](#) (HSQUIRRELM v)
仮想マシンに新しいルートテーブルを設定する。
- SQRESULT [sq_setconsttable](#) (HSQUIRRELM v)
仮想マシンに新しい定数テーブルを設定する。
- void [sq_setforeignptr](#) (HSQUIRRELM v, SQUserPointer p)
仮想マシンに外部ポインタを関連付ける。
- SQUserPointer [sq_getforeignptr](#) (HSQUIRRELM v)
仮想マシンに関連付けられている外部ポインタを取得する。
- void [sq_setsharedforeignptr](#) (HSQUIRRELM v, SQUserPointer p)
共有状態に外部ポインタを設定する。
- SQUserPointer [sq_getsharedforeignptr](#) (HSQUIRRELM v)
共有状態に設定された外部ポインタを取得する。
- void [sq_setvmreleasehook](#) (HSQUIRRELM v, SQRELEASEHOOK hook)
仮想マシンの解放時に実行されるリリースフックを設定する。
- SQRELEASEHOOK [sq_getvmreleasehook](#) (HSQUIRRELM v)
設定されている仮想マシンのリリースフック関数を取得する。
- void [sq_setsharedreleasehook](#) (HSQUIRRELM v, SQRELEASEHOOK hook)
共有状態のリリースフック関数を設定する。
- SQRELEASEHOOK [sq_getsharedreleasehook](#) (HSQUIRRELM v)
共有状態に設定されているリリースフック関数を取得する。
- void [sq_push](#) (HSQUIRRELM v, SQInteger idx)
スタックに値をプッシュする。
- SQObjectType [sq_gettype](#) (HSQUIRRELM v, SQInteger idx)
指定インデックスのスタック要素の型を取得する。
- SQRESULT [sq_typeof](#) (HSQUIRRELM v, SQInteger idx)
指定されたスタックインデックスのオブジェクトの型名を取得する。
- SQRESULT [sq_tostring](#) (HSQUIRRELM v, SQInteger idx)
スタックの指定インデックスにあるオブジェクトを文字列に変換してプッシュする。
- void [sq_tobool](#) (HSQUIRRELM v, SQInteger idx, SQBool *b)
指定インデックスのオブジェクトを真偽値として取得する。
- SQRESULT [sq_getinteger](#) (HSQUIRRELM v, SQInteger idx, SQInteger *i)
スタック上の数値またはブール値を整数として取得する。
- SQRESULT [sq_getfloat](#) (HSQUIRRELM v, SQInteger idx, SQFloat *f)
スタック上の数値を浮動小数点数として取得する。
- SQRESULT [sq_getbool](#) (HSQUIRRELM v, SQInteger idx, SQBool *b)
スタック上のブール値を取得する。
- SQRESULT [sq_getstringandsize](#) (HSQUIRRELM v, SQInteger idx, const SQChar **, SQInteger *size)
指定インデックスの文字列とそのサイズを取得する。
- SQRESULT [sq_getstring](#) (HSQUIRRELM v, SQInteger idx, const SQChar **c)
指定インデックスの文字列を取得する。
- SQRESULT [sq_getthread](#) (HSQUIRRELM v, SQInteger idx, HSQUIRRELM *thread)
指定インデックスのスレッドオブジェクトを取得する。
- SQRESULT [sq_clone](#) (HSQUIRRELM v, SQInteger idx)

- 指定インデックスのオブジェクトをクローンする。
- SQInteger [sq_getsize](#) (HSQUIRRELM v, SQInteger idx)
- 指定インデックスのオブジェクトのサイズを取得する。
- SQHash [sq_gethash](#) (HSQUIRRELM v, SQInteger idx)
- 指定インデックスのオブジェクトのハッシュ値を取得する。
- SQRESULT [sq_getuserdata](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer *p, SQUserPointer *typetag)
- 指定インデックスのユーザーデータと型タグを取得する。
- SQRESULT [sq_settypetag](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer typetag)
- 指定インデックスのオブジェクトに型タグを設定する。
- SQRESULT [sq_getobjtypetag](#) (const HSQOBJECT *o, SQUserPointer *typetag)
- オブジェクトの型タグを取得する。
- SQRESULT [sq_gettypetag](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer *typetag)
- スタックインデックスのオブジェクトに設定された型タグを取得する。
- SQRESULT [sq_getuserpointer](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer *p)
- スタックインデックスのユーザーポインタを取得する。
- SQRESULT [sq_setinstanceup](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer p)
- インスタンスにユーザーポインタを設定する。
- SQRESULT [sq_setclassudsize](#) (HSQUIRRELM v, SQInteger idx, SQInteger udsiz)
- クラスのユーザーデータサイズを設定する。
- SQRESULT [sq_getinstanceup](#) (HSQUIRRELM v, SQInteger idx, SQUserPointer *p, SQUserPointer typetag, SQBool throwerror)
- クラスインスタンスのユーザーポインタを取得する。
- SQInteger [sq_gettop](#) (HSQUIRRELM v)
- スタック上の現在のトップインデックスを取得する。
- void [sq_settop](#) (HSQUIRRELM v, SQInteger newtop)
- スタックのトップを新しい位置に設定する。
- void [sq_pop](#) (HSQUIRRELM v, SQInteger nelemstopop)
- スタックのトップから指定数の要素をポップする。
- void [sq_poptop](#) (HSQUIRRELM v)
- スタックのトップ要素を 1 つポップする。
- void [sq_remove](#) (HSQUIRRELM v, SQInteger idx)
- スタックの指定インデックスの要素を削除する。
- SQInteger [sq_cmp](#) (HSQUIRRELM v)
- スタックトップ 2 つのオブジェクトを比較する。
- SQRESULT [sq_newslot](#) (HSQUIRRELM v, SQInteger idx, SQBool bstatic)
- テーブルまたはクラスに新しいスロット（キーと値のペア）を追加する。
- SQRESULT [sq_deleteslot](#) (HSQUIRRELM v, SQInteger idx, SQBool pushval)
- テーブルのスロット（キーと値のペア）を削除する。
- SQRESULT [sq_set](#) (HSQUIRRELM v, SQInteger idx)
- テーブルまたはクラスのスロットを設定する。
- SQRESULT [sq_rawset](#) (HSQUIRRELM v, SQInteger idx)
- テーブル・クラス・インスタンスなどのスロットを生で設定する。
- SQRESULT [sq_newmember](#) (HSQUIRRELM v, SQInteger idx, SQBool bstatic)
- クラスに新しいメンバーを追加する。
- SQRESULT [sq_rawnewmember](#) (HSQUIRRELM v, SQInteger idx, SQBool bstatic)
- クラスに新しいメンバーを生で追加する。
- SQRESULT [sq_setdelegate](#) (HSQUIRRELM v, SQInteger idx)
- テーブルまたはユーザーデータにデリゲートを設定する。
- SQRESULT [sq_rawdeleteslot](#) (HSQUIRRELM v, SQInteger idx, SQBool pushval)
- テーブルのスロットを生で削除する。
- SQRESULT [sq_getdelegate](#) (HSQUIRRELM v, SQInteger idx)

- テーブルまたはユーザーデータのデリゲートを取得する。
 - SQRESULT [sq_get](#) (HSQUIRRELV v, SQInteger idx)
- テーブル・クラス・インスタンスなどから値を取得する。
 - SQRESULT [sq_rawget](#) (HSQUIRRELV v, SQInteger idx)
- テーブル・クラス・インスタンスなどから値を生で取得する。
 - SQRESULT [sq_getstackobj](#) (HSQUIRRELV v, SQInteger idx, HSQOBJECT *po)
- スタックインデックスのオブジェクトを取得する。
 - const SQChar * [sq_getlocal](#) (HSQUIRRELV v, SQUnsignedInteger level, SQUnsignedInteger idx)
- 指定レベルのローカル変数の名前を取得する。
 - void [sq_pushobject](#) (HSQUIRRELV v, HSQOBJECT obj)
- オブジェクトをスタックにプッシュする。
 - void [sq_resetobject](#) (HSQOBJECT *po)
- オブジェクトを初期化状態にリセットする。
 - SQRESULT [sq_throwerror](#) (HSQUIRRELV v, const SQChar *err)
- エラーメッセージを設定してエラーを発生させる。
 - SQRESULT [sq_throwobject](#) (HSQUIRRELV v)
- スタックトップのオブジェクトをエラーとして投げる。
 - void [sq_reserror](#) (HSQUIRRELV v)
- ラストエラーをリセットする。
 - void [sq_getlasterror](#) (HSQUIRRELV v)
- ラストエラーをスタックにプッシュする。
 - SQRESULT [sq_reservestack](#) (HSQUIRRELV v, SQInteger nsize)
- スタックサイズを予約する。
 - SQRESULT [sq_resume](#) (HSQUIRRELV v, SQBool retval, SQBool raiseerror)
- サスペンド状態のジェネレータを再開する。
 - SQRESULT [sq_call](#) (HSQUIRRELV v, SQInteger params, SQBool retval, SQBool raiseerror)
- 関数またはクロージャを呼び出す。
 - SQRESULT [sq_tailcall](#) (HSQUIRRELV v, SQInteger nparams)
- 関数またはクロージャを末尾呼び出しする。
 - SQRESULT [sq_suspendvm](#) (HSQUIRRELV v)
- 仮想マシンをサスペンド状態にする。
 - SQRESULT [sq_wakeupvm](#) (HSQUIRRELV v, SQBool wakeupret, SQBool retval, SQBool raiseerror, SQBool throwerror)
- サスペンド状態の仮想マシンを再開する。
 - void [sq_setreleasehook](#) (HSQUIRRELV v, SQInteger idx, SQRELEASEHOOK hook)
- オブジェクトにリリースフックを設定する。
 - SQRELEASEHOOK [sq_getreleasehook](#) (HSQUIRRELV v, SQInteger idx)
- オブジェクトのリリースフックを取得する。
 - void [sq_setcompilererrorhandler](#) (HSQUIRRELV v, SQCOMPILERERROR f)
- コンパイラエラーハンドラを設定する。
 - SQRESULT [sq_writeclosure](#) (HSQUIRRELV v, SQWRITEFUNC w, SQUserPointer up)
- クロージャをストリームに書き出す。
 - SQRESULT [sq_readclosure](#) (HSQUIRRELV v, SQREADFUNC r, SQUserPointer up)
- バイトコードストリームからクロージャを読み込む。
 - SQChar * [sq_getscratchpad](#) (HSQUIRRELV v, SQInteger minsize)
- スクラッチパッドを取得する。
 - SQRESULT [sq_resurrectunreachable](#) (HSQUIRRELV v)
- 到達不能オブジェクトを復活させる。
 - SQInteger [sq_collectgarbage](#) (HSQUIRRELV v)
- ガーベジコレクタを実行し、不要なメモリを解放する。
 - SQRESULT [sq_getcallee](#) (HSQUIRRELV v)

- 現在実行中の呼び出しクロージャを取得する。
- const SQChar * [sq_getfreevariable](#) (HSQUIRRELVm v, SQInteger idx, SQUnsignedInteger nval)
クロージャの自由変数の名前を取得する。
- SQRESULT [sq_setfreevariable](#) (HSQUIRRELVm v, SQInteger idx, SQUnsignedInteger nval)
クロージャの自由変数を設定する。
- SQRESULT [sq_setattributes](#) (HSQUIRRELVm v, SQInteger idx)
クラスのメンバー属性を設定する。
- SQRESULT [sq_getattributes](#) (HSQUIRRELVm v, SQInteger idx)
クラスのメンバー属性を取得する。
- SQRESULT [sq_getmemberhandle](#) (HSQUIRRELVm v, SQInteger idx, HSQMEMBERHANDLE *handle)
クラスメンバーのハンドルを取得する。
- SQRESULT [_getmemberbyhandle](#) (HSQUIRRELVm v, SQObjectPtr &self, const HSQMEMBERHANDLE *handle, SQObjectPtr *&val)
メンバーのハンドル情報から実体を取得する。
- SQRESULT [sq_getbyhandle](#) (HSQUIRRELVm v, SQInteger idx, const HSQMEMBERHANDLE *handle)
メンバーのハンドル情報から値を取得する。
- SQRESULT [sq_setbyhandle](#) (HSQUIRRELVm v, SQInteger idx, const HSQMEMBERHANDLE *handle)
メンバーのハンドル情報を使って値を設定する。
- SQRESULT [sq_getbase](#) (HSQUIRRELVm v, SQInteger idx)
クラスの基底クラスを取得する。
- SQRESULT [sq_getclass](#) (HSQUIRRELVm v, SQInteger idx)
インスタンスのクラスを取得する。
- SQRESULT [sq_createinstance](#) (HSQUIRRELVm v, SQInteger idx)
クラスから新しいインスタンスを生成する。
- void [sq_weakref](#) (HSQUIRRELVm v, SQInteger idx)
弱参照を生成する。
- SQRESULT [sq_getweakrefval](#) (HSQUIRRELVm v, SQInteger idx)
弱参照から値を取得する。
- SQRESULT [sq_getdefaultdelegate](#) (HSQUIRRELVm v, SQObjectType t)
デフォルトデリゲートを取得する。
- SQRESULT [sq_next](#) (HSQUIRRELVm v, SQInteger idx)
イテレータを進め、次の要素を取得する。
- SQInteger [buf_lexfeed](#) (SQUserPointer file)
バッファから 1 文字読み込む。
- SQRESULT [sq_compilebuffer](#) (HSQUIRRELVm v, const SQChar *s, SQInteger size, const SQChar *sourcename, SQBool raiseerror)
バッファからスクリプトをコンパイルする。
- void [sq_move](#) (HSQUIRRELVm dest, HSQUIRRELVm src, SQInteger idx)
スタックの値を他の VM に移動する。
- void [sq_setprintfunc](#) (HSQUIRRELVm v, SQPRINTFUNCTION printfunc, SQPRINTFUNCTION errfunc)
出力関数とエラー出力関数を設定する。
- SQPRINTFUNCTION [sq_getprintfunc](#) (HSQUIRRELVm v)
出力関数を取得する。
- SQPRINTFUNCTION [sq_geterrorfunc](#) (HSQUIRRELVm v)
エラー出力関数を取得する。
- void * [sq_malloc](#) (SQUnsignedInteger size)
メモリを確保する。
- void * [sq_realloc](#) (void *p, SQUnsignedInteger oldsize, SQUnsignedInteger newsize)
メモリを再確保する。
- void [sq_free](#) (void *p, SQUnsignedInteger size)
メモリを解放する。

4.1.1 Macro Definition Documentation

4.1.1.1 _GETSAFE_OBJ

```
#define _GETSAFE_OBJ(
    v,
    idx,
    type,
    o ) { if(!sq_aux_gettypedarg(v,idx,type,&o)) return SQ_ERROR; }
```

指定型のオブジェクトをスタックから安全に取得する。

[sq_aux_gettypedarg\(\)](#) を用いて、スタック上の値が指定された型であることを確認し、該当しない場合は `SQ_ERROR` を返すショートカットマクロ。

Parameters

<i>v</i>	スクリプト仮想マシン (VM) インスタンス。
<i>idx</i>	スタックインデックス。
<i>type</i>	期待する <code>SQObjectType</code> 型。
<i>o</i>	結果として取得される <code>SQObjectPtr*</code> 変数名 (変数そのものではない)。

Note

- はアドレス渡しの変数名であり、呼び出し元で宣言されている必要があります。

Return values

<code>SQ_ERROR</code>	型が一致しない場合。
-----------------------	------------

Definition at line 59 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

4.1.1.2 sq_aux_paramscheck

```
#define sq_aux_paramscheck(
    v,
    count )
```

Value:

```
{ \
    if(sq_gettop(v) < count){ v->Raise_Error(SC("not enough params in the stack")); return SQ_ERROR; }\
}
```

スタックに十分な数の引数があるかを検査する。

引数の数が `count` に満たない場合、エラーメッセージを設定して `SQ_ERROR` を返す。マクロの展開先での早期リターンに使用される。

Parameters

<i>v</i>	スクリプト仮想マシン (VM) インスタンス。
<i>count</i>	最低限必要な引数の数。

Return values

<code>SQ_ERROR</code>	引数が不足している場合。
-----------------------	--------------

Definition at line 73 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
00074 { \
00075     if(sq_gettop(v) < count){ v->Raise_Error(_SC("not enough params in the stack")); return SQ_ERROR;
    }\
00076 }
```

4.1.2 Function Documentation

4.1.2.1 _getmemberbyhandle()

```
SQRESULT _getmemberbyhandle (
    HSQUIRRELVm v,
    SQObjectPtr & self,
    const HSQMEMBERHANDLE * handle,
    SQObjectPtr *& val )
```

メンバーのハンドル情報から実体を取得する。

指定されたクラスまたはインスタンスのメンバーを `handle` で指定し、実際の値ポインタを `val` に格納する。型がクラスまたはインスタンスでない場合はエラーを返す。

Parameters

	<code>v</code>	対象の仮想マシン。
	<code>self</code>	対象オブジェクト。
	<code>handle</code>	メンバーのハンドル情報。
out	<code>val</code>	メンバー値へのポインタ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	無効な型の場合。

Definition at line 3030 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
03031 {
03032     switch(sq_type(self)) {
03033     case OT_INSTANCE: {
03034         SQInstance *i = _instance(self);
03035         if(handle->_static) {
03036             SQClass *c = i->_class;
03037             val = &c->methods[handle->_index].val;
03038         }
03039         else {
03040             val = &i->values[handle->_index];
03041         }
03042     }
03043     }
03044     break;
03045     case OT_CLASS: {
03046         SQClass *c = _class(self);
03047         if(handle->_static) {
03048             val = &c->methods[handle->_index].val;
03049         }
03050         else {
03051             val = &c->defaultvalues[handle->_index].val;
```



```

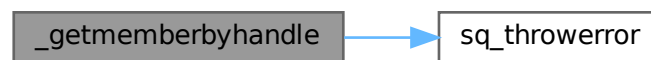
03052         }
03053     }
03054     break;
03055     default:
03056         return sq_throwerror(v,_SC("wrong type(expected class or instance)"));
03057     }
03058     return SQ_OK;
03059 }

```

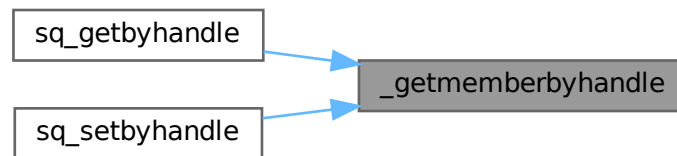
References [sq_throwerror\(\)](#).

Referenced by [sq_getbyhandle\(\)](#), and [sq_setbyhandle\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.2 buf_lexfeed()

```

SQInteger buf_lexfeed (
    SQUserPointer file )

```

バッファから 1 文字読み込む。

[BufState](#) 構造体で管理されているバッファから現在位置の文字を読み込み、ポインタを進める。バッファ終端に到達した場合は 0 を返す。

Parameters

<i>file</i>	読み込み対象の BufState へのポインタ。
-------------	--

Returns

SQInteger 読み込んだ文字、終端の場合は 0。

Definition at line 3298 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
03299 {
03300     BufState *buf=(BufState*) file;
03301     if (buf->size<(buf->ptr+1))
03302         return 0;
03303     return buf->buf[buf->ptr++];
03304 }
```

References [BufState::buf](#), [BufState::ptr](#), and [BufState::size](#).

Referenced by [sq_compilebuffer\(\)](#).

Here is the caller graph for this function:



4.1.2.3 sq_addrf()

```
void sq_addrf (
    HSQUIRRELVN v,
    HSQOBJECT * po )
```

指定されたオブジェクトの参照カウントを 1 つ増加させる。

参照カウントを持つオブジェクト（文字列、配列、テーブルなど）に対して、明示的にリファレンスを保持したい場合に使用する。

NO_GARBAGE_COLLECTOR が定義されている場合、ビルドに応じて異なる処理が行われる。

Parameters

<i>v</i>	対象となる仮想マシン。
<i>po</i>	対象の HSQOBJECT。型が参照カウント対象でない場合は無視される。

Definition at line 351 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
00352 {
00353     if (!ISREFCOUNTED(sq_type(*po))) return;
00354     #ifdef NO_GARBAGE_COLLECTOR
00355         __AddRef(po->_type, po->_unVal);
00356     #else
00357         _ss(v)->_refs_table.AddRef(*po);
00358     #endif
00359 }
```

4.1.2.4 sq_arrayappend()

```
SQRESULT sq_arrayappend (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定した配列の末尾に要素を追加する。

スタックのトップにある要素を、`idx` で指定された配列の末尾に追加する。追加後、スタックトップから要素は削除される。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	配列が存在するスタックインデックス。

Return values

<code>SQ_OK</code>	正常に追加された場合。
<code>SQ_ERROR</code>	対象が配列でない場合、またはパラメータ数が不足している場合。

Definition at line 753 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00754 {
00755     sq_aux_paramscheck(v,2);
00756     SQObjectPtr *arr;
00757     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00758     _array(*arr)->Append(v->GetUp(-1));
00759     v->Pop();
00760     return SQ_OK;
00761 }
```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.5 sq_arrayinsert()

```
SQRESULT sq_arrayinsert (
    HSQUIRRELVM v,
    SQInteger idx,
    SQInteger destpos )
```

配列の指定位置に要素を挿入する。

スタックのトップにある値を、`destpos` で指定された配列のインデックス位置に挿入する。挿入後、スタックトップは自動的にポップされる。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象の配列のスタックインデックス。
<code>destpos</code>	挿入する位置（インデックス）。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Return values

<code>SQ_ERROR</code>	配列以外が指定された場合や、 <code>destpos</code> が範囲外の場合。
-----------------------	--

Definition at line 885 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00886 {
00887     sq_aux_paramscheck(v, 1);
00888     SQObjectPtr *arr;
00889     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00890     SQRESULT ret = _array(*arr)->Insert(destpos, v->GetUp(-1)) ? SQ_OK : sq_throwerror(v, _SC("index
    out of range"));
00891     v->Pop();
00892     return ret;
00893 }
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.6 sq_arraypop()

```
SQRESULT sq_arraypop (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool pushval )
```

指定した配列の末尾の要素を削除する。

指定された配列の末尾要素を削除する。`pushval` が `true` の場合、削除する前の要素をスタックにプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象の配列のスタックインデックス。
<code>pushval</code>	削除前の要素をスタックにプッシュするかどうか。

Return values

<code>SQ_OK</code>	正常に削除された場合。
<code>SQ_ERROR</code>	配列が空の場合、または引数が不正な場合。

Definition at line 777 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00778 {
00779     sq_aux_paramscheck(v, 1);
00780     SQObjectPtr *arr;
00781     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00782     if(_array(*arr)->Size() > 0) {
00783         if(pushval != 0){ v->Push(_array(*arr)->Top()); }
00784         _array(*arr)->Pop();
00785         return SQ_OK;
00786     }
00787     return sq_throwerror(v, _SC("empty array"));
00788 }

```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.7 sq_arrayremove()

```

SQLRESULT sq_arrayremove (
    HSQUIRRELVLM v,
    SQInteger idx,
    SQInteger itemidx )

```

配列から指定されたインデックスの要素を削除する。

配列の itemidx で指定された位置の要素を削除し、後続の要素が前に詰められる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象の配列のスタックインデックス。
<i>itemidx</i>	削除する要素のインデックス。

Return values

<i>SQ_OK</i>	成功（削除された場合）。
<i>SQ_ERROR</i>	配列以外の型、またはインデックス範囲外の場合。

Definition at line 863 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00864 {
00865     sq_aux_paramscheck(v, 1);
00866     SQObjectPtr *arr;
00867     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00868     return _array(*arr)->Remove(itemidx) ? SQ_OK : sq_throwerror(v, _SC("index out of range"));
00869 }

```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.8 sq_arrayresize()

```

SQRESULT sq_arrayresize (
    HSQUIRRELVLM v,
    SQInteger idx,
    SQInteger newsize )
  
```

指定した配列のサイズを変更する。

配列のサイズを newsize に変更する。新しいサイズが現在より大きい場合は null で埋められ、小さい場合は末尾から要素が削除される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象の配列のスタックインデックス。
<i>newsize</i>	新しい配列のサイズ (0 以上)。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	newsize が負の値だった場合、または対象が配列でない場合。

Definition at line 805 of file squirrel3_squirrel_sqapiByOpenAl.cpp.

```

00806 {
00807     sq_aux_paramscheck(v,1);
00808     SQObjectPtr *arr;
00809     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00810     if(newsize >= 0) {
00811         _array(*arr)->Resize(newsize);
00812         return SQ_OK;
00813     }
00814     return sq_throwerror(v,_SC("negative size"));
00815 }
  
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.9 sq_arrayreverse()

```

SQRESULT sq_arrayreverse (
    HSQUIRRELVLM v,
    SQInteger idx )
  
```

配列内の要素の順序を反転する。

配列の要素をインプレースで反転する（先頭と末尾を交換、次にその内側...）。配列が空または1要素のみの場合は何も行わない。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象の配列のスタックインデックス。

Return values

<i>SQ_OK</i>	成功（反転したか、何もしなかったか）。
<i>SQ_ERROR</i>	対象が配列でない、または引数が不正な場合。

Definition at line 830 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00831 {
00832     sq_aux_paramscheck(v, 1);
00833     SQObjectPtr *o;
00834     _GETSAFE_OBJ(v, idx, OT_ARRAY, o);
00835     SQArray *arr = _array(*o);
00836     if(arr->Size() > 0) {
00837         SQObjectPtr t;
00838         SQInteger size = arr->Size();
00839         SQInteger n = size » 1; size -= 1;
00840         for(SQInteger i = 0; i < n; i++) {
00841             t = arr->_values[i];
00842             arr->_values[i] = arr->_values[size-i];
00843             arr->_values[size-i] = t;
00844         }
00845         return SQ_OK;
00846     }
00847     return SQ_OK;
00848 }
  
```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.10 sq_aux_gettypedarg()

```

static bool sq_aux_gettypedarg (
    HSQUIRRELVLM v,
  
```

```
SQInteger idx,
SQObjectType type,
SQObjectPtr ** o ) [static]
```

引数の型をチェックし、指定の型と一致していれば対象オブジェクトを取得する。

仮想マシンスタックから指定インデックスのオブジェクトを取得し、与えられた型と一致するかを検証する。型が一致しない場合、エラーメッセージを生成して `false` を返す。

Parameters

	<i>v</i>	スクリプト仮想マシン (VM) インスタンス。
	<i>idx</i>	スタック上の引数インデックス。
	<i>type</i>	期待されるオブジェクト型。
out	<i>o</i>	対応するオブジェクトが格納されるポインタへのポインタ。

Return values

<i>true</i>	型が一致し、*o に対象が格納された場合。
<i>false</i>	型が一致せず、エラーが発生した場合。

Definition at line 33 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00034 {
00035     *o = &stack_get(v,idx);
00036     if(sq_type(**o) != type){
00037         SQObjectPtr oval = v->PrintObjVal(**o);
00038         v->Raise_Error(_SC("wrong argument type, expected '%s' got
'%s'",IdType2Name(type),_stringval(oval)));
00039         return false;
00040     }
00041     return true;
00042 }
```

4.1.2.11 sq_aux_invalidtype()

```
SQInteger sq_aux_invalidtype (
    HSQUIRRELM v,
    SQObjectType type )
```

想定外のオブジェクト型が指定された場合のエラーを返す。

指定された型情報を文字列に変換して、"unexpected type < 型名 >" というエラーメッセージを生成し、[sq_throwerror\(\)](#) を通じて例外として返す。

Parameters

<i>v</i>	スクリプト仮想マシン (VM) インスタンス。
<i>type</i>	想定外だった SQObjectType。

Returns

SQInteger エラーコード (常に SQ_ERROR を返す)。

Definition at line 90 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

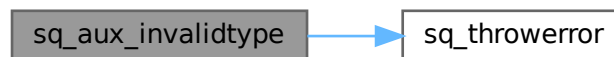
00091 {
00092     SQUnsignedInteger buf_size = 100 * sizeof(SQChar);
00093     scsprintf(_ss(v)->GetScratchPad(buf_size), buf_size, _SC("unexpected type %s"),
        IdType2Name(type));
00094     return sq_throwerror(v, _ss(v)->GetScratchPad(-1));
00095 }

```

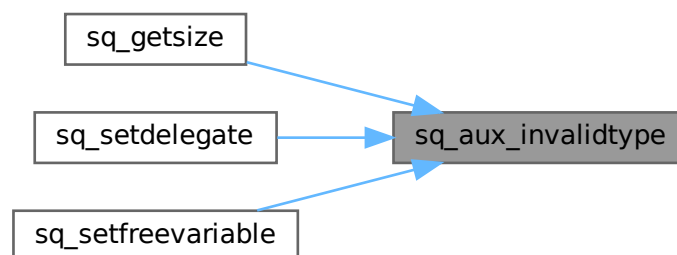
References [sq_throwerror\(\)](#).

Referenced by [sq_getsize\(\)](#), [sq_setdelegate\(\)](#), and [sq_setfreevariable\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.12 sq_bindenv()

```

SQLRESULT sq_bindenv (
    HSQUIRRELVM v,
    SQInteger idx )

```

クロージャに環境オブジェクト (env) をバインドする。

スタックの `idx` にあるクロージャに対して、トップにある環境オブジェクト (テーブル、配列、クラス、インスタンス) をバインドする。

環境は弱参照として内部に保持される。対象がネイティブクロージャまたは通常クロージャでない場合はエラー。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	クロージャのスタックインデックス。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がクロージャでない、または無効な環境が指定された場合。

Definition at line 1038 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01039 {
01040     SQObjectPtr &o = stack_get(v, idx);
01041     if(!sq_isnativeclosure(o) &&
01042         !sq_isclosure(o))
01043         return sq_throwerror(v, _SC("the target is not a closure"));
01044     SQObjectPtr &env = stack_get(v, -1);
01045     if(!sq_istable(env) &&
01046         !sq_isarray(env) &&
01047         !sq_isclass(env) &&
01048         !sq_isinstance(env))
01049         return sq_throwerror(v, _SC("invalid environment"));
01050     SQWeakRef *w = _refcounted(env)->GetWeakRef(sq_type(env));
01051     SQObjectPtr ret;
01052     if(sq_isclosure(o)) {
01053         SQClosure *c = _closure(o)->Clone();
01054         __ObjRelease(c->_env);
01055         c->_env = w;
01056         __ObjAddRef(c->_env);
01057         if(_closure(o)->_base) {
01058             c->_base = _closure(o)->_base;
01059             __ObjAddRef(c->_base);
01060         }
01061         ret = c;
01062     }
01063     else { //then must be a native closure
01064         SQNativeClosure *c = _nativeclosure(o)->Clone();
01065         __ObjRelease(c->_env);
01066         c->_env = w;
01067         __ObjAddRef(c->_env);
01068         ret = c;
01069     }
01070     v->Pop();
01071     v->Push(ret);
01072     return SQ_OK;
01073 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.13 sq_call()

```

SQLRESULT sq_call (
    HSQUIRRELM v,

```

```

    SQInteger params,
    SQBool retval,
    SQBool raiseerror )

```

関数またはクロージャを呼び出す。

スタックトップにある関数（クロージャ）を、指定した引数の数 `params` を用いて呼び出す。呼び出し結果をスタックに残すかは `retval` で制御する。呼び出し時にエラーが発生した場合、`raiseerror` が `true` なら例外を送出する。

Parameters

<code>v</code>	対象の仮想マシン。
<code>params</code>	引数の個数。
<code>retval</code>	戻り値を残す場合 <code>true</code> 。
<code>raiseerror</code>	エラー発生時に例外を送出する場合 <code>true</code> 。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	呼び出し失敗時。

Definition at line 2554 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02555 {
02556     SQObjectPtr res;
02557     if (!v->Call(v->GetUp(-(params+1)), params, v->_top-params, res, raiseerror?true:false)) {
02558         v->Pop(params); //pop args
02559         return SQ_ERROR;
02560     }
02561     if (!v->_suspended)
02562         v->Pop(params); //pop args
02563     if(retval)
02564         v->Push(res); // push result
02565     return SQ_OK;
02566 }

```

4.1.2.14 sq_clear()

```

SQRESULT sq_clear (
    HSQUIRRELVN v,
    SQInteger idx )

```

テーブルまたは配列の内容をすべて削除する。

指定インデックスのオブジェクトがテーブルなら `Clear()` を、配列なら `Resize(0)` によって要素を全消去する。対象がどちらでもない場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	テーブルまたは配列が存在するスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Return values

<code>SQ_ERROR</code>	対象がテーブルでも配列でもない場合。
-----------------------	--------------------

Definition at line 1167 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01168 {
01169     SQObject &o=stack_get(v,idx);
01170     switch(sq_type(o)) {
01171         case OT_TABLE: _table(o)->Clear(); break;
01172         case OT_ARRAY: _array(o)->Resize(0); break;
01173         default:
01174             return sq_throwerror(v, _SC("clear only works on table and array"));
01175         break;
01176     }
01177     return SQ_OK;
01178 }
01179 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.15 sq_clone()

```

SQRESULT sq_clone (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定インデックスのオブジェクトをクローンする。

スタックの `idx` にあるオブジェクトをクローンし、新しいクローンオブジェクトをスタック上にプッシュする。クローンに失敗した場合はエラーコードを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	クローン対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クローンに失敗した場合。

Definition at line 1643 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01644 {
01645     SQObjectPtr &o = stack_get(v, idx);
01646     v->PushNull();
01647     if(!v->Clone(o, stack_get(v, -1))){
01648         v->Pop();
01649         return SQ_ERROR;
01650     }
01651     return SQ_OK;
01652 }

```

4.1.2.16 sq_close()

```

void sq_close (
    HSQUIRRELVm v )

```

仮想マシンを終了し、関連するリソースを解放する。

ルートVM に対して `Finalize()` を呼び出し、共有状態オブジェクト (`SQSharedState`) を解放する。この操作により、仮想マシンとそのリソースは完全に破棄される。

Parameters

<code>v</code>	対象となる仮想マシンインスタンス。
----------------	-------------------

Definition at line 253 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```

00254 {
00255     SQSharedState *ss = _ss(v);
00256     _thread(ss->_root_vm)->Finalize();
00257     sq_delete(ss, SQSharedState);
00258 }

```

4.1.2.17 sq_cmp()

```

SQInteger sq_cmp (
    HSQUIRRELVm v )

```

スタックトップ 2 つのオブジェクトを比較する。

スタックのトップにある 2 つのオブジェクトを比較し、比較結果を整数として返す。

Parameters

<code>v</code>	対象の仮想マシン。
----------------	-----------

Returns

`SQInteger` 比較結果 (-1, 0, 1)。

Definition at line 1976 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```

01977 {
01978     SQInteger res;
01979     v->ObjCmp(stack_get(v, -1), stack_get(v, -2), res);
01980     return res;
01981 }

```

4.1.2.18 sq_collectgarbage()

```
SQInteger sq_collectgarbage (
    HSQUIRRELVM v )
```

ガーベジコレクタを実行し、不要なメモリを解放する。

仮想マシンのガーベジコレクタを手動で呼び出し、到達不能なオブジェクトを解放する。ガーベジコレクタが無効なビルドの場合は -1 を返す。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQInteger 解放されたオブジェクト数、または -1。

Definition at line 2811 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02812 {
02813     #ifndef NO_GARBAGE_COLLECTOR
02814         return _ss(v)->CollectGarbage(v);
02815     #else
02816         return -1;
02817     #endif
02818 }
```

4.1.2.19 sq_compile()

```
SQRESULT sq_compile (
    HSQUIRRELVM v,
    SQLEXREADFUNC read,
    SQUserPointer p,
    const SQChar * sourcename,
    SQBool raiseerror )
```

スクリプトソースをコンパイルして、実行可能なクロージャをスタックにプッシュする。

ソースコードを読み取るためのリーダー関数 (read) を通じて入力を受け取り、コンパイラを用いてバイトコードに変換し、関数クロージャとして SQClosure を生成・プッシュする。

NO_COMPILER が定義されている場合はビルド構成上コンパイル不可であり、エラーを返す。

Parameters

<i>v</i>	対象となる仮想マシン。
<i>read</i>	ソース入力用の関数ポインタ (1 バイトずつ読み取る関数)。
<i>p</i>	read 関数に渡されるユーザーデータ。
<i>sourcename</i>	エラーメッセージなどに使用されるソース名。
<i>raiseerror</i>	コンパイルエラー発生時に例外を投げるかどうかのフラグ。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Return values

<code>SQ_ERROR</code>	コンパイル失敗。
-----------------------	----------

Definition at line 292 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```

00293 {
00294     SQObjectPtr o;
00295     #ifndef NO_COMPILER
00296         if(Compile(v, read, p, sourcename, o, raiseerror?true:false, _ss(v)->_debuginfo)) {
00297             v->Push(SQClosure::Create(_ss(v), _funcproto(o),
00298                 _table(v->_roottable)->GetWeakRef(OT_TABLE)));
00299             return SQ_OK;
00300         }
00301         return SQ_ERROR;
00302     #else
00303         return sq_throwerror(v, _SC("this is a no compiler build"));
00304     #endif
00305 }
```

References [sq_throwerror\(\)](#).

Referenced by [sq_compilebuffer\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.2.20 sq_compilebuffer()

```

SQRESULT sq_compilebuffer (
    HSQUIRRELVM v,
    const SQChar * s,
    SQInteger size,
    const SQChar * sourcename,
    SQBool raiseerror )
```

バッファからスクリプトをコンパイルする。

与えられた文字列バッファをソースとしてスクリプトをコンパイルする。内部で `buf_lexfeed` を使用して文字を読み込み、スクリプトを解析する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>s</i>	コンパイル対象の文字列。
<i>size</i>	バッファサイズ。
<i>sourcename</i>	ソース名 (エラー出力用など)。
<i>raiseerror</i>	コンパイルエラー時に例外を送出するかどうか。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	失敗。

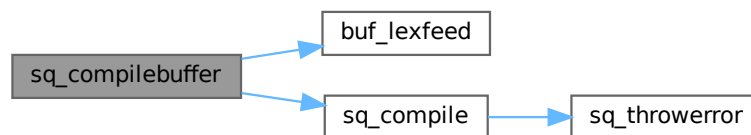
Definition at line 3321 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```

03321 {
03322     BufState buf;
03323     buf.buf = s;
03324     buf.size = size;
03325     buf.ptr = 0;
03326     return sq_compile(v, buf_lexfeed, &buf, sourcename, raiseerror);
03327 }
```

References [BufState::buf](#), [buf_lexfeed\(\)](#), [BufState::ptr](#), [BufState::size](#), and [sq_compile\(\)](#).

Here is the call graph for this function:



4.1.2.21 sq_createinstance()

```

SQLRESULT sq_createinstance (
    HSQUIRRELVm v,
    SQInteger idx )
```

クラスから新しいインスタンスを生成する。

スタックの *idx* にあるクラスオブジェクトから新しいインスタンスを生成し、スタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クラスのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 3163 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03164 {
03165     SQObjectPtr *o = NULL;
03166     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03167     v->Push(_class(*o)->CreateInstance());
03168     return SQ_OK;
03169 }
```

References [_GETSAFE_OBJ](#).

4.1.2.22 `sq_deleteslot()`

```
SQRESULT sq_deleteslot (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool pushval )
```

テーブルのスロット（キーと値のペア）を削除する。

指定インデックス `idx` にあるテーブルからスタックトップにあるキーで指定されるスロットを削除する。削除後に値をスタックにプッシュするかどうかは `pushval` で制御する。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象テーブルのスタックインデックス。
<code>pushval</code>	削除した値をプッシュする場合 <code>true</code> 。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	キーが無効な場合などのエラー。

Definition at line 2023 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02024 {
02025     sq_aux_paramscheck(v, 2);
02026     SQObjectPtr *self;
02027     _GETSAFE_OBJ(v, idx, OT_TABLE,self);
02028     SQObjectPtr &key = v->GetUp(-1);
02029     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
02030     SQObjectPtr res;
02031     if(!v->DeleteSlot(*self, key, res)){
02032         v->Pop();
02033         return SQ_ERROR;
02034     }
02035     if(pushval) v->GetUp(-1) = res;
02036     else v->Pop();
02037     return SQ_OK;
02038 }
```

References [_GETSAFE_OBJ](#), [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.23 sq_enableddebuginfo()

```
void sq_enableddebuginfo (
    HSQUIRRELVm v,
    SQBool enable )
```

コンパイル時にデバッグ情報を埋め込むかどうかを設定する。

デバッグ情報には、関数名・ファイル名・行番号などの情報が含まれ、スタックトレースの出力やデバッガによる解析に必要となる。この設定は仮想マシンの共有状態に対して適用される。

Parameters

<i>v</i>	対象となる仮想マシン。
<i>enable</i>	有効にする場合は true、無効にする場合は false。

Definition at line 317 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00318 {
00319     _ss(v)->_debuginfo = enable?true:false;
00320 }
```

4.1.2.24 sq_free()

```
void sq_free (
    void * p,
    SQUnsignedInteger size )
```

メモリを解放する。

指定されたメモリ領域 *p* を解放する。*size* は解放するメモリのサイズで、内部アロケータに渡される。

Parameters

<i>p</i>	解放するメモリブロックへのポインタ。
<i>size</i>	解放するメモリのサイズ (バイト)。

Definition at line 3432 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03433 {
03434     SQ_FREE(p, size);
03435 }
```

4.1.2.25 sq_get()

```
SQRESULT sq_get (
    HSQUIRRELVm v,
    SQInteger idx )
```

テーブル・クラス・インスタンスなどから値を取得する。

スタックの `idx` にあるオブジェクトから、スタックトップにあるキーを使って値を取得する。キーが存在しない場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	キーが存在しない場合など。

Definition at line 2285 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02286 {
02287     SQObjectPtr &self=stack_get(v,idx);
02288     SQObjectPtr &obj = v->GetUp(-1);
02289     if(v->Get(self,obj,obj,false,DONT_FALL_BACK))
02290         return SQ_OK;
02291     v->Pop();
02292     return SQ_ERROR;
02293 }
```

4.1.2.26 sq_getattributes()

```
SQRESULT sq_getattributes (
    HSQUIRRELVm v,
    SQInteger idx )
```

クラスのメンバー属性を取得する。

スタックの `idx` にあるクラスオブジェクトからメンバー属性を取得する。メンバー名が `null` の場合はクラス全体の属性を取得し、それ以外の場合は該当メンバーの属性を取得してスタックにプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象クラスのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	無効なインデックスの場合。

Definition at line 2967 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02968 {
02969     SQObjectPtr *o = NULL;
02970     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02971     SQObjectPtr &key = stack_get(v,-1);
02972     SQObjectPtr attrs;
02973     if(sq_type(key) == OT_NULL) {
02974         attrs = _class(*o)->_attributes;
02975         v->Pop();
02976         v->Push(attrs);
02977         return SQ_OK;
02978     }
02979     else if(_class(*o)->GetAttributes(key,attrs)) {
02980         v->Pop();
02981         v->Push(attrs);
02982         return SQ_OK;
02983     }
02984     return sq_throwerror(v,_SC("wrong index"));
02985 }
```

References [_GETSAFE_OBJ](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.27 sq_getbase()

```

SQRESULT sq_getbase (
    HSQUIRRELVm v,
    SQInteger idx )
```

クラスの基底クラスを取得する。

スタックの `idx` にあるクラスオブジェクトの基底クラスを取得し、存在すればスタックにプッシュする。基底クラスがない場合は `null` をプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象クラスのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 3122 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03123 {
03124     SQObjectPtr *o = NULL;
03125     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03126     if(_class(*o)->_base)
03127         v->Push(SQObjectPtr(_class(*o)->_base));
```

```

03128     else
03129         v->PushNull();
03130     return SQ_OK;
03131 }

```

References [_GETSAFE_OBJ.](#)

4.1.2.28 sq_getbool()

```

SQRESULT sq_getbool (
    HSQUIRRELVm v,
    SQInteger idx,
    SQBool * b )

```

スタック上のブール値を取得する。

スタックのインデックス `idx` にあるオブジェクトがブール値型である場合に、その値を整数型で `b` に格納する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>b</i>	結果として格納されるブール値。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	ブール値でない場合。

Definition at line 1557 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01558 {
01559     SQObjectPtr &o = stack_get(v, idx);
01560     if(sq_isbool(o)) {
01561         *b = _integer(o);
01562         return SQ_OK;
01563     }
01564     return SQ_ERROR;
01565 }

```

4.1.2.29 sq_getbyhandle()

```

SQRESULT sq_getbyhandle (
    HSQUIRRELVm v,
    SQInteger idx,
    const HSQMEMBERHANDLE * handle )

```

メンバーのハンドル情報から値を取得する。

スタックの `idx` にあるクラスまたはインスタンスオブジェクトから、指定されたメンバーのハンドルを使って値を取得し、スタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックインデックス。
<i>handle</i>	メンバーのハンドル情報。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	取得失敗。

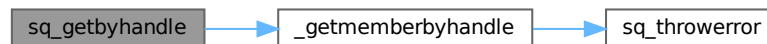
Definition at line 3074 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

03075 {
03076     SQObjectPtr &self = stack_get(v, idx);
03077     SQObjectPtr *val = NULL;
03078     if(SQ_FAILED(_getmemberbyhandle(v, self, handle, val))) {
03079         return SQ_ERROR;
03080     }
03081     v->Push(_realval(*val));
03082     return SQ_OK;
03083 }
```

References [_getmemberbyhandle\(\)](#).

Here is the call graph for this function:



4.1.2.30 sq_getcallee()

```

SQRESULT sq_getcallee (
    HSQUIRRELVM v )
```

現在実行中の呼び出しクロージャを取得する。

呼び出しスタックに存在するクロージャを取得し、スタックにプッシュする。呼び出しスタックにクロージャが存在しない場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
----------------	-----------

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クロージャが存在しない場合。

Definition at line 2831 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

02832 {
02833     if(v->_callsstacksize > 1)
02834     {
02835         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
02836         return SQ_OK;
02837     }
02838     return sq_throwerror(v, _SC("no closure in the calls stack"));
02839 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.31 sq_getclass()

```

SQRESULT sq_getclass (
    HSQUIRRELVM v,
    SQInteger idx )
  
```

インスタンスのクラスを取得する。

スタックの `idx` にあるインスタンスオブジェクトのクラス情報を取得し、スタックにプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象インスタンスのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 3144 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03145 {
03146     SQObjectPtr *o = NULL;
03147     _GETSAFE_OBJ(v, idx, OT_INSTANCE,o);
03148     v->Push(SQObjectPtr(_instance(*o)->_class));
03149     return SQ_OK;
03150 }
  
```

References [_GETSAFE_OBJ](#).

4.1.2.32 sq_getclosureinfo()

```

SQRESULT sq_getclosureinfo (
    HSQUIRRELVM v,
    SQInteger idx,
    SQInteger * nparams,
    SQInteger * nfreevars )
  
```

クロージャの引数数および自由変数数を取得する。

指定されたスタックインデックスのオブジェクトがクロージャ型の場合に、引数数と自由変数数を取得して出力引数に格納する。

- 通常クロージャ (OT_CLOSURE) とネイティブクロージャ (OT_NATIVECLOSURE) に対応。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	対象クロージャのスタックインデックス。
out	<i>nparams</i>	クロージャの引数数が格納される。
out	<i>nfreevars</i>	自由変数の数が格納される。

Return values

<i>SQ_OK</i>	情報の取得に成功。
<i>SQ_ERROR</i>	対象がクロージャ型でない場合。

Definition at line 936 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00937 {
00938     SQObject o = stack_get(v, idx);
00939     if(sq_type(o) == OT_CLOSURE) {
00940         SQClosure *c = _closure(o);
00941         SQFunctionProto *proto = c->_function;
00942         *nparams = proto->_nparameters;
00943         *nfreevars = proto->_noutervalues;
00944         return SQ_OK;
00945     }
00946     else if(sq_type(o) == OT_NATIVECLOSURE)
00947     {
00948         SQNativeClosure *c = _nativeclosure(o);
00949         *nparams = c->_nparamscheck;
00950         *nfreevars = (SQInteger)c->_noutervalues;
00951         return SQ_OK;
00952     }
00953     return sq_throwerror(v, _SC("the object is not a closure"));
00954 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.33 sq_getclosurename()

```

SQRESULT sq_getclosurename (
    HSQUIRRELM v,
    SQInteger idx )
```

クロージャに設定されている名前を取得する。

指定インデックスのオブジェクトがクロージャ (ネイティブまたは通常) である場合に、その名前をスタックにプッシュする。名前が設定されていない場合は null が返される可能性もある。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クロージャのスタックインデックス。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がクロージャ型でない場合。

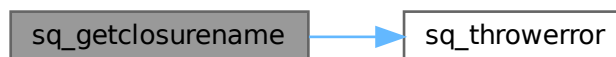
Definition at line 1088 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01089 {
01090     SQObjectPtr &o = stack_get(v, idx);
01091     if(!sq_isnativeclosure(o) &&
01092         !sq_isclosure(o))
01093         return sq_throwerror(v, _SC("the target is not a closure"));
01094     if(sq_isnativeclosure(o))
01095     {
01096         v->Push(_nativeclosure(o)->_name);
01097     }
01098     else { //closure
01099         v->Push(_closure(o)->_function->_name);
01100     }
01101     return SQ_OK;
01102 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.34 sq_getclosureroot()

```

SQRESULT sq_getclosureroot (
    HSQUIRRELVM v,
    SQInteger idx )
```

クロージャに設定されているルートテーブルを取得する。

指定されたクロージャ (OT_CLOSURE) のルートオブジェクトを取得し、スタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クロージャのスタックインデックス。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がクロージャでない場合。

Definition at line 1145 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01146 {
01147     SQObjectPtr &c = stack_get(v, idx);
01148     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
01149     v->Push(_closure(c)->_root->_obj);
01150     return SQ_OK;
01151 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.35 sq_getdefaultdelegate()

```
SQRESULT sq_getdefaultdelegate (
    HSQUIRRELVM v,
    SQObjectType t )
```

デフォルトデリゲートを取得する。

指定された型 *t* に対応するデフォルトデリゲートを取得し、スタックにプッシュする。サポートされていない型の場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>t</i>	デリゲートを取得する対象の型。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対応しない型の場合。

Definition at line 3225 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03226 {
03227     SQSharedState *ss = _ss(v);
03228     switch(t) {
03229         case OT_TABLE: v->Push(ss->_table_default_delegate); break;
03230         case OT_ARRAY: v->Push(ss->_array_default_delegate); break;
```

```

03231     case OT_STRING: v->Push(ss->_string_default_delegate); break;
03232     case OT_INTEGER: case OT_FLOAT: v->Push(ss->_number_default_delegate); break;
03233     case OT_GENERATOR: v->Push(ss->_generator_default_delegate); break;
03234     case OT_CLOSURE: case OT_NATIVECLOSURE: v->Push(ss->_closure_default_delegate); break;
03235     case OT_THREAD: v->Push(ss->_thread_default_delegate); break;
03236     case OT_CLASS: v->Push(ss->_class_default_delegate); break;
03237     case OT_INSTANCE: v->Push(ss->_instance_default_delegate); break;
03238     case OT_WEAKREF: v->Push(ss->_weakref_default_delegate); break;
03239     default: return sq_throwerror(v,_SC("the type doesn't have a default delegate"));
03240 }
03241 return SQ_OK;
03242 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.36 sq_getdelegate()

```

SQLRESULT sq_getdelegate (
    HSQUIRRELVN v,
    SQInteger idx )

```

テーブルまたはユーザーデータのデリゲートを取得する。

スタックの `idx` にあるオブジェクトがテーブルまたはユーザーデータ型の場合、そのデリゲートをスタックにプッシュする。デリゲートが存在しない場合は `null` をプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	無効な型の場合。

Definition at line 2255 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02256 {
02257     SQObjectPtr &self=stack_get(v,idx);
02258     switch(sq_type(self)){
02259     case OT_TABLE:
02260     case OT_USERDATA:
02261         if(!_delegable(self)->_delegate){
02262             v->PushNull();
02263             break;
02264         }
02265         v->Push(SQObjectPtr(_delegable(self)->_delegate));

```

```

02266         break;
02267     default: return sq_throwerror(v,_SC("wrong type")); break;
02268     }
02269     return SQ_OK;
02270
02271 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.37 sq_geterrorfunc()

```

SQPRINTFUNCTION sq_geterrorfunc (
    HSQUIRRELVm v )
```

エラー出力関数を取得する。

設定されているエラー出力用の printf 関数を取得する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQPRINTFUNCTION 設定されているエラー出力関数。

Definition at line 3385 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

03386 {
03387     return _ss(v)->_errorfunc;
03388 }
```

4.1.2.38 sq_getfloat()

```

SQRESULT sq_getfloat (
    HSQUIRRELVm v,
    SQInteger idx,
    SQFloat * f )
```

スタック上の数値を浮動小数点数として取得する。

インデックス *idx* にあるオブジェクトが数値型（整数または浮動小数点数）である場合、浮動小数点数に変換して *f* に格納する。それ以外の型であれば失敗となる。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>f</i>	結果として格納される浮動小数点数のポインタ。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	数値でない場合。

Definition at line 1534 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01535 {
01536     SQObjectPtr &o = stack_get(v, idx);
01537     if(sq_isnumeric(o)) {
01538         *f = tofloat(o);
01539         return SQ_OK;
01540     }
01541     return SQ_ERROR;
01542 }
```

4.1.2.39 sq_getforeignptr()

```

SQUserPointer sq_getforeignptr (
    HSQUIRRELVm v )
```

仮想マシンに関連付けられている外部ポインタを取得する。

[sq_setforeignptr\(\)](#) で設定された外部ポインタの値を返す。 外部アプリケーションが VM に任意のコンテキスト情報を紐づけたい場合に使用される。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQUserPointer 設定された外部ポインタ。未設定の場合は NULL。

Definition at line 1299 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01300 {
01301     return v->_foreignptr;
01302 }
```

4.1.2.40 sq_getfreevariable()

```

const SQChar * sq_getfreevariable (
    HSQUIRRELVm v,
    SQInteger idx,
    SQUnsignedInteger nval )
```

クロージャの自由変数の名前を取得する。

スタックの *idx* にあるクロージャまたはネイティブクロージャの指定された インデックスの自由変数の名前を取得する。存在すればその変数の値をプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックインデックス。
<i>nval</i>	取得する自由変数のインデックス。

Returns

const SQChar* 変数名、存在しない場合は NULL。

Definition at line 2853 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

02854 {
02855     SQObjectPtr &self=stack_get(v,idx);
02856     const SQChar *name = NULL;
02857     switch(sq_type(self))
02858     {
02859     case OT_CLOSURE:{
02860         SQClosure *clo = _closure(self);
02861         SQFunctionProto *fp = clo->_function;
02862         if(((SQUnsignedInteger)fp->noutervalues) > nval) {
02863             v->Push(*(_outer(clo->_outervalues[nval])->_valptr));
02864             SQOuterVar &ov = fp->_outervalues[nval];
02865             name = _stringval(ov._name);
02866         }
02867     }
02868     break;
02869     case OT_NATIVECLOSURE:{
02870         SQNativeClosure *clo = _nativeclosure(self);
02871         if(clo->_noutervalues > nval) {
02872             v->Push(clo->_outervalues[nval]);
02873             name = _SC("@NATIVE");
02874         }
02875     }
02876     break;
02877     default: break; //shutup compiler
02878     }
02879     return name;
02880 }
```

4.1.2.41 sq_gethash()

```

SQHash sq_gethash (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定インデックスのオブジェクトのハッシュ値を取得する。

スタックの *idx* にあるオブジェクトのハッシュ値を計算して返す。ハッシュはオブジェクトの比較やマップのキーとして使用される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	ハッシュを取得するオブジェクトのスタックインデックス。

Returns

SQHash 計算されたハッシュ値。

Definition at line 1692 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01693 {
01694     SQObjectPtr &o = stack_get(v, idx);
01695     return HashObj(o);
01696 }

```

4.1.2.42 sq_getinstanceup()

```

SQLRESULT sq_getinstanceup (
    HSQUIRRELM v,
    SQInteger idx,
    SQUserPointer * p,
    SQUserPointer typetag,
    SQBool throwerror )

```

クラスインスタンスのユーザーポインタを取得する。

スタックの `idx` にあるオブジェクトがクラスインスタンス型である場合、そのユーザーポインタを `p` に取得する。`typetag` を指定すると、インスタンスが指定された型タグを持っているかを確認する。型タグが一致しない場合や、オブジェクトがインスタンスでない場合にはエラーを返す (`throwerror` が `true` の場合)。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>p</i>	取得されたユーザーポインタ。
	<i>typetag</i>	確認する型タグ (省略可)。
	<i>throwerror</i>	型タグが一致しない場合にエラーを投げるかどうか。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	失敗。

Definition at line 1870 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01871 {
01872     SQObjectPtr &o = stack_get(v, idx);
01873     if (sq_type(o) != OT_INSTANCE) return throwerror ? sq_throwerror(v, _SC("the object is not a class
instance")) : SQ_ERROR;
01874     (*p) = _instance(o)->_userpointer;
01875     if (typetag != 0) {
01876         SQClass *cl = _instance(o)->_class;
01877         do {
01878             if (cl->_typetag == typetag)
01879                 return SQ_OK;
01880             cl = cl->_base;
01881         } while (cl != NULL);
01882         return throwerror ? sq_throwerror(v, _SC("invalid type tag")) : SQ_ERROR;
01883     }
01884     return SQ_OK;
01885 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.43 sq_getinteger()

```

SQRESULT sq_getinteger (
    HSQUIRRELVm v,
    SQInteger idx,
    SQInteger * i )
  
```

スタック上の数値またはブール値を整数として取得する。

インデックス `idx` のオブジェクトが数値型（整数または浮動小数点数）またはブール値の場合に、整数として変換し `i` に格納する。どちらでもない場合はエラーを返す。

Parameters

	<code>v</code>	対象の仮想マシン。
	<code>idx</code>	スタックインデックス。
out	<code>i</code>	結果として格納される整数のポインタ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	数値でもブール値でもない場合。

Definition at line 1507 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01508 {
01509     SQObjectPtr &o = stack_get(v, idx);
01510     if(sq_isnumeric(o)) {
01511         *i = tointeger(o);
01512         return SQ_OK;
01513     }
01514     if(sq_isbool(o)) {
01515         *i = SQVM::IsFalse(o)?SQFalse:SQTrue;
01516         return SQ_OK;
01517     }
01518     return SQ_ERROR;
01519 }
  
```

4.1.2.44 sq_getlasterror()

```

void sq_getlasterror (
    HSQUIRRELVm v )
  
```

ラストエラーをスタックにプッシュする。

仮想マシンに現在設定されているラストエラーオブジェクトをスタックにプッシュする。エラー内容を取得して処理したい場合に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 2485 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02486 {
02487     v->Push(v->_lasterror);
02488 }
```

4.1.2.45 sq_getlocal()

```
const SQChar * sq_getlocal (
    HSQUIRRELVM v,
    SQUnsignedInteger level,
    SQUnsignedInteger idx )
```

指定レベルのローカル変数の名前を取得する。

指定されたコールスタックレベル *level* とローカル変数のインデックス *idx* に対応する ローカル変数の名前を返す。該当するローカル変数が存在しない場合は NULL を返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>level</i>	コールスタックレベル。
<i>idx</i>	ローカル変数のインデックス。

Returns

const SQChar* ローカル変数の名前、存在しない場合は NULL。

Definition at line 2374 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02375 {
02376     SQUnsignedInteger cstksize=v->callsstacksize;
02377     SQUnsignedInteger lvl=(cstksize-level)-1;
02378     SQInteger stackbase=v->stackbase;
02379     if (lvl<cstksize) {
02380         for (SQUnsignedInteger i=0; i<level; i++) {
02381             SQVM::CallInfo &ci=v->callsstack[(cstksize-i)-1];
02382             stackbase=ci._prevstkbase;
02383         }
02384         SQVM::CallInfo &ci=v->callsstack[lvl];
02385         if (sq_type(ci._closure) != OT_CLOSURE)
02386             return NULL;
02387         SQClosure *c=_closure(ci._closure);
02388         SQFunctionProto *func=c->_function;
02389         if (func->_noutervalues > (SQInteger)idx) {
02390             v->Push(*_outer(c->_outervalues[idx])->_valptr);
02391             return _stringval(func->_outervalues[idx]._name);
02392         }
02393         idx -= func->_noutervalues;
02394         return func->GetLocal(v, stackbase, idx, (SQInteger)(ci._ip-func->_instructions)-1);
02395     }
02396     return NULL;
02397 }
```


4.1.2.46 sq_getmemberhandle()

```
SQRESULT sq_getmemberhandle (
    HSQUIRRELVm v,
    SQInteger idx,
    HSQMEMBERHANDLE * handle )
```

クラスメンバーのハンドルを取得する。

スタックの `idx` にあるクラスオブジェクトから、指定されたキーに対応する メンバーのハンドル情報を取得し、`handle` に格納する。

Parameters

	<code>v</code>	対象の仮想マシン。
	<code>idx</code>	対象クラスのスタックインデックス。
out	<code>handle</code>	取得したメンバーのハンドル情報。

Return values

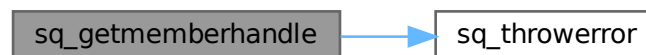
<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	無効なインデックスの場合。

Definition at line 3000 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03001 {
03002     SQObjectPtr *o = NULL;
03003     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03004     SQObjectPtr &key = stack_get(v,-1);
03005     SQTable *m = _class(*o)->_members;
03006     SQObjectPtr val;
03007     if(m->Get(key,val)) {
03008         handle->_static = _isfield(val) ? SQFalse : SQTrue;
03009         handle->_index = _member_idx(val);
03010         v->Pop();
03011         return SQ_OK;
03012     }
03013     return sq_throwerror(v,_SC("wrong index"));
03014 }
```

References [_GETSAFE_OBJ](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.47 sq_getobjtypetag()

```
SQRESULT sq_getobjtypetag (
    const HSQOBJECT * o,
    SQUserPointer * typetag )
```

オブジェクトの型タグを取得する。

与えられた HSQOBJECT がインスタンス、ユーザーデータ、またはクラス型の場合、その型タグを `typetag` に格納する。それ以外の型の場合はエラーを返す。

Parameters

	<i>o</i>	対象のオブジェクトポインタ。
out	<i>typetag</i>	取得した型タグを格納するポインタ。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がサポートされない型の場合。

Definition at line 1756 of file `squirrel3_squirrel_sqapiByOpenAl.cpp`.

```

01757 {
01758     switch(sq_type(*o)) {
01759         case OT_INSTANCE: *typetag = _instance(*o)->_class->_typetag; break;
01760         case OT_USERDATA: *typetag = _userdata(*o)->_typetag; break;
01761         case OT_CLASS:     *typetag = _class(*o)->_typetag; break;
01762         default: return SQ_ERROR;
01763     }
01764     return SQ_OK;
01765 }
```

Referenced by `sq_gettypetag()`.

Here is the caller graph for this function:



4.1.2.48 sq_getprintfunc()

```

SQPRINTFUNCTION sq_getprintfunc (
    HSQUIRRELVm v )
```

出力関数を取得する。

設定されている標準出力用の `printf` 関数を取得する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQPRINTFUNCTION 設定されている出力関数。

Definition at line 3371 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
03372 {
03373     return _ss(v)->_printfunc;
03374 }
```

4.1.2.49 sq_getrefcount()

```
SQUnsignedInteger sq_getrefcount (
    HSQUIRRELVm v,
    HSQOBJECT * po )
```

指定されたオブジェクトの参照カウントを取得する。

参照カウントを持つオブジェクト（例：配列、テーブル、クロージャなど）に対して、現在の参照数（retain count）を返す。対象が参照カウント対象でない場合は 0 を返す。

NO_GARBAGE_COLLECTOR が定義されている場合とそうでない場合で取得方法が異なる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>po</i>	対象の HSQOBJECT ポインタ。

Returns

SQUnsignedInteger 現在の参照カウント。対象外であれば 0。

Definition at line 375 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
00376 {
00377     if (!ISREFCOUNTED(sq_type(*po))) return 0;
00378     #ifdef NO_GARBAGE_COLLECTOR
00379     return po->_unVal.pRefCounted->_uiRef;
00380     #else
00381     return _ss(v)->_refs_table.GetRefCount(*po);
00382     #endif
00383 }
```

4.1.2.50 sq_getreleasehook()

```
SQRELEASEHOOK sq_getreleasehook (
    HSQUIRRELVm v,
    SQInteger idx )
```

オブジェクトのリリースフックを取得する。

スタックの idx にあるオブジェクトがユーザーデータ・インスタンス・クラス型の場合、設定されているリリースフック関数を取得して返す。他の型の場合は NULL を返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Returns

SQRELEASEHOOK 設定されているリリースフック、または NULL。

Definition at line 2683 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02684 {
02685     SQObjectPtr &ud=stack_get(v,idx);
02686     switch(sq_type(ud) ) {
02687     case OT_USERDATA: return _userdata(ud)->_hook; break;
02688     case OT_INSTANCE: return _instance(ud)->_hook; break;
02689     case OT_CLASS: return _class(ud)->_hook; break;
02690     default: return NULL;
02691     }
02692 }
```

4.1.2.51 sq_getscratchpad()

```
SQChar * sq_getscratchpad (
    HSQUIRRELVm v,
    SQInteger minsize )
```

スクラッチパッドを取得する。

仮想マシンの内部スクラッチパッドメモリを取得する。メモリサイズは minsize 以上になるように確保される。スクラッチパッドは一時的な文字列操作などに利用される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>minsize</i>	必要な最小サイズ。

Returns

SQChar* 確保されたスクラッチパッドのポインタ。

Definition at line 2775 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02776 {
02777     return _ss(v)->GetScratchPad(minsize);
02778 }
```

4.1.2.52 sq_getsharedforeignptr()

```
SQUserPointer sq_getsharedforeignptr (
    HSQUIRRELVm v )
```

共有状態に設定された外部ポインタを取得する。

[sq_setsharedforeignptr\(\)](#) で設定されたポインタを取得する。これは全仮想マシンに共通する外部情報を格納する用途に使用される。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQUserPointer 共有外部ポインタ。未設定の場合は NULL。

Definition at line 1330 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
01331 {
01332     return _ss(v)->_foreignptr;
01333 }
```

4.1.2.53 sq_getsharedreleasehook()

```
SQRELEASEHOOK sq_getsharedreleasehook (
    HSQUIRRELVM v )
```

共有状態に設定されているリリースフック関数を取得する。

[sq_setsharedreleasehook\(\)](#) で設定された共有領域の解放フック関数を取得する。フック関数が設定されていない場合は NULL を返す。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQRELEASEHOOK 現在設定されている共有リリースフック関数。

Definition at line 1392 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
01393 {
01394     return _ss(v)->_releasehook;
01395 }
```

4.1.2.54 sq_getsize()

```
SQInteger sq_getsize (
    HSQUIRRELVM v,
    SQInteger idx )
```

指定インデックスのオブジェクトのサイズを取得する。

スタックの *idx* にあるオブジェクトの型に応じて、そのサイズ（文字列長、テーブル要素数、配列要素数、ユーザーデータサイズなど）を返す。対応していない型の場合はエラー値を返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	サイズを取得するオブジェクトのスタックインデックス。

Returns

SQInteger オブジェクトのサイズ。

Definition at line 1665 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

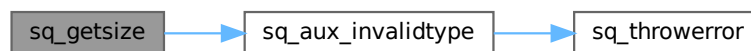
```

01666 {
01667     SQObjectPtr &o = stack_get(v, idx);
01668     SQObjectType type = sq_type(o);
01669     switch(type) {
01670     case OT_STRING:      return _string(o)->_len;
01671     case OT_TABLE:      return _table(o)->CountUsed();
01672     case OT_ARRAY:      return _array(o)->Size();
01673     case OT_USERDATA:   return _userdata(o)->_size;
01674     case OT_INSTANCE:   return _instance(o)->_class->_udsize;
01675     case OT_CLASS:      return _class(o)->_udsize;
01676     default:
01677         return sq_aux_invalidtype(v, type);
01678     }
01679 }

```

References [sq_aux_invalidtype\(\)](#).

Here is the call graph for this function:



4.1.2.55 sq_getstackobj()

```

SQLRESULT sq_getstackobj (
    HSQUIRRELVM v,
    SQInteger idx,
    HSQOBJECT * po )

```

スタックインデックスのオブジェクトを取得する。

指定されたスタックインデックスのオブジェクトを HSQOBJECT として取得し、po に格納する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>po</i>	取得されたオブジェクトを格納するポインタ。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Definition at line 2356 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02357 {
02358     *po=stack_get(v,idx);
02359     return SQ_OK;
02360 }

```

4.1.2.56 sq_getstring()

```

SQLRESULT sq_getstring (

```

```

HSQUIRRELVM v,
SQInteger idx,
const SQChar ** c )

```

指定インデックスの文字列を取得する。

スタックの指定インデックス `idx` にあるオブジェクトが文字列型の場合、そのポインタを `c` に格納する。文字列長は不要な場合にこの関数を使用する。

Parameters

	<code>v</code>	対象の仮想マシン。
	<code>idx</code>	スタックインデックス。
out	<code>c</code>	取得された文字列へのポインタ。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 1602 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01603 {
01604     SQObjectPtr *o = NULL;
01605     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01606     *c = _stringval(*o);
01607     return SQ_OK;
01608 }

```

References [_GETSAFE_OBJ](#).

4.1.2.57 sq_getstringandsize()

```

SQRESULT sq_getstringandsize (
    HSQUIRRELVM v,
    SQInteger idx,
    const SQChar ** c,
    SQInteger * size )

```

指定インデックスの文字列とそのサイズを取得する。

スタックの指定インデックス `idx` にあるオブジェクトが文字列型の場合、そのポインタと長さを取得して `c` および `size` に格納する。

Parameters

	<code>v</code>	対象の仮想マシン。
	<code>idx</code>	スタックインデックス。
out	<code>c</code>	取得された文字列へのポインタ。
out	<code>size</code>	文字列の長さ。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 1580 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01581 {
01582     SQObjectPtr *o = NULL;
01583     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01584     *c = _stringval(*o);
01585     *size = _string(*o)->_len;
01586     return SQ_OK;
01587 }
```

References [_GETSAFE_OBJ](#).

4.1.2.58 sq_getthread()

```
SQRESULT sq_getthread (
    HSQUIRRELVm v,
    SQInteger idx,
    HSQUIRRELVm * thread )
```

指定インデックスのスレッドオブジェクトを取得する。

スタックの指定されたインデックスにあるオブジェクトがスレッド型 (OT_THREAD) であるかを確認し、該当オブジェクトをスレッドポインタとして thread に格納する。スレッドオブジェクトを操作する際に使用される。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>thread</i>	取得されたスレッドオブジェクトへのポインタ。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Definition at line 1623 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01624 {
01625     SQObjectPtr *o = NULL;
01626     _GETSAFE_OBJ(v, idx, OT_THREAD,o);
01627     *thread = _thread(*o);
01628     return SQ_OK;
01629 }
```

References [_GETSAFE_OBJ](#).

4.1.2.59 sq_gettop()

```
SQInteger sq_gettop (
    HSQUIRRELVm v )
```

スタック上の現在のトップインデックスを取得する。

現在のスタックトップの位置を返す。この値はスタックベースからの相対値であり、スタック操作の状態を確認する際に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Returns

SQInteger スタックトップのインデックス。

Definition at line 1897 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01898 {
01899     return (v->_top) - v->_stackbase;
01900 }
```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.60 sq_gettype()

```
SQObjectType sq_gettype (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定インデックスのスタック要素の型を取得する。

指定されたスタックインデックス *idx* に存在するオブジェクトの型を取得し、SQObjectType（例：OT_NULL、OT_INTEGER、OT_STRING など）として返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	調査対象のスタックインデックス。

Returns

SQObjectType スタック上のオブジェクトの型。

Definition at line 1424 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01425 {
01426     return sq_type(stack_get(v, idx));
01427 }
```

4.1.2.61 sq_gettypetag()

```
SQRESULT sq_gettypetag (
    HSQUIRRELVLM v,
    SQInteger idx,
    SQUserPointer * typetag )
```

スタックインデックスのオブジェクトに設定された型タグを取得する。

指定されたスタックインデックスのオブジェクトの型タグを取得し、typetag に格納する。内部的には sq_getobjtypetag を呼び出して処理する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>typetag</i>	取得された型タグを格納するポインタ。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	型タグを取得できなかった場合。

Definition at line 1780 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
01781 {
01782     SQObjectPtr &o = stack_get(v,idx);
01783     if (SQ_FAILED(sq_getobjtypetag(&o, typetag)))
01784         return SQ_ERROR; // this is not an error it should be a bool but would break backward
        compatibility
01785     return SQ_OK;
01786 }
```

References [sq_getobjtypetag\(\)](#).

Here is the call graph for this function:



4.1.2.62 sq_getuserdata()

```
SQRESULT sq_getuserdata (
    HSQUIRRELVLM v,
    SQInteger idx,
    SQUserPointer * p,
    SQUserPointer * typetag )
```

指定インデックスのユーザーデータと型タグを取得する。

スタックの `idx` にあるオブジェクトがユーザーデータ型である場合、そのポインタを `p` に格納し、型タグがある場合は `typetag` に格納する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>p</i>	取得されたユーザーデータポインタ。
out	<i>typetag</i>	型タグを受け取るポインタ（省略可）。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Definition at line 1711 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01712 {
01713     SQObjectPtr *o = NULL;
01714     _GETSAFE_OBJ(v, idx, OT_USERDATA, o);
01715     (*p) = _userdataval(*o);
01716     if(typetag) *typetag = _userdata(*o)->_typetag;
01717     return SQ_OK;
01718 }
```

References [_GETSAFE_OBJ](#).

4.1.2.63 sq_getuserpointer()

```

SQRESULT sq_getuserpointer (
    HSQUIRRELVm v,
    SQInteger idx,
    SQUserPointer * p )
```

スタックインデックスのユーザーポインタを取得する。

指定されたスタックインデックス *idx* にあるオブジェクトがユーザーポインタ型の場合、その値を *p* に格納する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	スタックインデックス。
out	<i>p</i>	取得されたユーザーポインタ。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Definition at line 1800 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01801 {
01802     SQObjectPtr *o = NULL;
01803     _GETSAFE_OBJ(v, idx, OT_USERPOINTER, o);
01804     (*p) = _userpointer(*o);
01805     return SQ_OK;
01806 }
```

References [_GETSAFE_OBJ](#).

4.1.2.64 sq_getversion()

```
SQInteger sq_getversion ( )
```

Squirrel 仮想マシンのバージョン番号を取得する。

定数 `SQUIRREL_VERSION_NUMBER` に定義されているバージョン番号を返す。通常、コンパイル時定義により決まる。

Returns

SQInteger Squirrel のバージョン番号。

Definition at line 269 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00270 {
00271     return SQUIRREL_VERSION_NUMBER;
00272 }
```

4.1.2.65 sq_getvmrefcount()

```
SQUnsignedInteger sq_getvmrefcount (
    HSQUIRRELVm SQ_UNUSED_ARG v,
    const HSQOBJECT * po )
```

指定されたオブジェクトの参照カウント値を直接取得する (VM 外の補助関数)。

この関数は、ガベージコレクションに関係なくオブジェクトの内部 `RefCount` を直接参照して返す。通常の参照管理と無関係に、デバッグや監視の目的で使用される。

Parameters

<i>v</i>	仮想マシン (未使用だが引数として必要)。
<i>po</i>	対象の HSQOBJECT ポインタ。

Returns

SQUnsignedInteger 参照カウント。参照管理対象でない場合は 0。

Definition at line 423 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00424 {
00425     if (!ISREFCOUNTED(sq_type(*po))) return 0;
00426     return po->_unVal.pRefCounted->_uiRef;
00427 }
```

4.1.2.66 sq_getvmreleasehook()

```
SQRELEASEHOOK sq_getvmreleasehook (
    HSQUIRRELVm v )
```

設定されている仮想マシンのリリースフック関数を取得する。

[sq_setvmreleasehook\(\)](#) によって設定された VM の終了時フック関数を返す。フックが未設定の場合は NULL を返す。

Parameters

v	対象の仮想マシン。
----------	-----------

Returns

SQRELEASEHOOK 現在設定されているリリースフック関数。

Definition at line 1361 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01362 {
01363     return v->_releasehook;
01364 }
```

4.1.2.67 sq_getvmstate()

```
SQInteger sq_getvmstate (
    HSQUIRRELVm v )
```

仮想マシン (VM) の現在の状態を取得する。

VM が一時停止中 (suspended)、実行中 (running)、またはアイドル (idle) 状態のいずれであるかを判定し、状態を表す定数を返す。

状態は以下のいずれか：

- SQ_VMSTATE_SUSPENDED：実行が中断されている
- SQ_VMSTATE_RUNNING：現在関数呼び出し中
- SQ_VMSTATE_IDLE：何も実行されていない

Parameters

v	対象となる仮想マシンインスタンス。
----------	-------------------

Returns

SQInteger VM の状態を表す定数。

Definition at line 174 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00175 {
00176     if (v->_suspended)
00177         return SQ_VMSTATE_SUSPENDED;
00178     else {
00179         if (v->_callsstacksize != 0) return SQ_VMSTATE_RUNNING;
00180         else return SQ_VMSTATE_IDLE;
00181     }
00182 }
```

4.1.2.68 sq_getweakrefval()

```
SQRESULT sq_getweakrefval (
    HSQUIRRELVm v,
    SQInteger idx )
```

弱参照から値を取得する。

スタックの `idx` にあるオブジェクトが弱参照型の場合、その参照先の値を取得してスタックにプッシュする。弱参照型でない場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	弱参照オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	失敗。

Definition at line 3203 of file `squirrel3_squirrel_sqapiByOpenAl.cpp`.

```
03204 {
03205     SQObjectPtr &o = stack_get(v, idx);
03206     if(sq_type(o) != OT_WEAKREF) {
03207         return sq_throwerror(v, _SC("the object must be a weakref"));
03208     }
03209     v->Push(_weakref(o)->_obj);
03210     return SQ_OK;
03211 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.69 sq_instanceof()

```
SQBool sq_instanceof (
    HSQUIRRELVM v )
```

オブジェクトが指定されたクラスのインスタンスかどうかを判定する。

スタック上の -1 (トップ) がインスタンス、-2 がクラスである必要がある。インスタンスがそのクラス、またはその派生クラスであるかを確認する。

Parameters

<code>v</code>	対象の仮想マシン。
----------------	-----------

Return values

<code>SQTrue</code>	インスタンスがクラス、またはその派生クラスのものである場合。
<code>SQ_ERROR</code>	引数の型が不正な場合。

Definition at line 731 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00732 {
00733     SQObjectPtr &inst = stack_get(v,-1);
00734     SQObjectPtr &cl = stack_get(v,-2);
00735     if(sq_type(inst) != OT_INSTANCE || sq_type(cl) != OT_CLASS)
00736         return sq_throwerror(v,_SC("invalid param type"));
00737     return _instance(inst)->InstanceOf(_class(cl)) ? SQTrue : SQFalse;
00738 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.70 sq_malloc()

```
void * sq_malloc (
    SQUnsignedInteger size )
```

メモリを確保する。

指定されたサイズのメモリを確保して返す。 スクリプト実行中の内部使用やカスタムアロケータに利用される。

Parameters

<code>size</code>	確保するメモリサイズ (バイト)。
-------------------	-------------------

Returns

`void*` 確保されたメモリ領域へのポインタ。

Definition at line 3400 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03401 {
03402     return SQ_MALLOC(size);
03403 }
```

4.1.2.71 sq_move()

```
void sq_move (
    HSQUIRRELVm dest,
```



```
HSQUIRRELVm src,
SQInteger idx )
```

スタックの値を他の VM に移動する。

src VM のスタックの idx にある値を dest VM にプッシュする。VM 間でオブジェクトを共有したい場合に使用する。

Parameters

<i>dest</i>	コピー先の仮想マシン。
<i>src</i>	コピー元の仮想マシン。
<i>idx</i>	コピー元スタックインデックス。

Definition at line 3340 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03341 {
03342     dest->Push(stack_get(src,idx));
03343 }
```

4.1.2.72 sq_newarray()

```
void sq_newarray (
    HSQUIRRELVm v,
    SQInteger size )
```

指定されたサイズの配列を生成し、スタックにプッシュする。

空の要素で初期化された配列を生成し、対象の仮想マシンスタックに追加する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>size</i>	初期化される配列のサイズ（要素数）。

Definition at line 682 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00683 {
00684     v->Push(SQArray::Create(_ss(v), size));
00685 }
```

4.1.2.73 sq_newclass()

```
SQRESULT sq_newclass (
    HSQUIRRELVm v,
    SQBool hasbase )
```

新しいクラスを生成し、スタックにプッシュする。

指定があれば、既存のクラスを継承した新しいクラスを生成する。

- hasbase が true の場合、スタック上のトップにあるクラスを継承元として使用する。
- hasbase が false の場合は継承なしの空クラスが作成される。

クラス生成後、スタックにプッシュされる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>hasbase</i>	ベースクラスを使用するかどうかのフラグ。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	ベース型が無効だった場合。

Definition at line 704 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00705 {
00706     SQClass *baseclass = NULL;
00707     if(hasbase) {
00708         SQObjectPtr &base = stack_get(v,-1);
00709         if(sq_type(base) != OT_CLASS)
00710             return sq_throwerror(v,_SC("invalid base type"));
00711         baseclass = _class(base);
00712     }
00713     SQClass *newclass = SQClass::Create(_ss(v), baseclass);
00714     if(baseclass) v->Pop();
00715     v->Push(newclass);
00716     return SQ_OK;
00717 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.74 sq_newclosure()

```

void sq_newclosure (
    HSQUIRRELVM v,
    SQFUNCTION func,
    SQUnsignedInteger nfreevars )
```

新しいネイティブクロージャを作成してスタックにプッシュする。

指定された C 関数ポインタ *func* と、自由変数の数 *nfreevars* を元に、 ネイティブクロージャ (SQNativeClosure) を生成し、スタックにプッシュする。

スタックのトップから *nfreevars* 個の値が自由変数として取り出され、クロージャにバインドされる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>func</i>	ネイティブ関数のポインタ。
<i>nfreevars</i>	クロージャにバインドする自由変数の個数。

Definition at line 908 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00909 {
00910     SQNativeClosure *nc = SQNativeClosure::Create(_ss(v), func, nfreevars);
00911     nc->nparamscheck = 0;
00912     for(SQUnsignedInteger i = 0; i < nfreevars; i++) {
00913         nc->outervalues[i] = v->Top();
00914         v->Pop();
00915     }
00916     v->Push(SQObjectPtr(nc));
00917 }
```

4.1.2.75 sq_newmember()

```
SQRESULT sq_newmember (
    HSQUIRRELM v,
    SQInteger idx,
    SQBool bstatic )
```

クラスに新しいメンバーを追加する。

スタックの `idx` にあるオブジェクトがクラス型の場合に、スタックトップのキー・値・属性を用いて新しいメンバーを追加する。メンバーを静的にするかどうかは `bstatic` で制御する。クラス型以外の場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クラスのスタックインデックス。
<i>bstatic</i>	静的メンバーとして追加する場合 <code>true</code> 。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	クラス型でない場合、またはキーが <code>null</code> の場合。

Definition at line 2127 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02128 {
02129     SQObjectPtr &self = stack_get(v, idx);
02130     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
02131     SQObjectPtr &key = v->GetUp(-3);
02132     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
02133     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, false)) {
02134         v->Pop(3);
02135         return SQ_ERROR;
02136     }
02137     v->Pop(3);
02138     return SQ_OK;
02139 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.76 sq_newslot()

```
SQRESULT sq_newslot (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool bstatic )
```

テーブルまたはクラスに新しいスロット（キーと値のペア）を追加する。

スタックの指定インデックス `idx` にあるテーブルまたはクラスに対して、スタックトップ2つの要素（キーと値）を使用して新しいスロットを追加する。スロットを静的メンバーとして追加することも可能。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象テーブルまたはクラスのスタックインデックス。
<code>bstatic</code>	スロットを静的にする場合 <code>true</code> 。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	キーが <code>null</code> の場合などのエラー。

Definition at line 1997 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01998 {
01999     sq_aux_paramscheck(v, 3);
02000     SQObjectPtr &self = stack_get(v, idx);
02001     if(sq_type(self) == OT_TABLE || sq_type(self) == OT_CLASS) {
02002         SQObjectPtr &key = v->GetUp(-2);
02003         if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
02004         v->NewSlot(self, key, v->GetUp(-1), bstatic?true:false);
02005         v->Pop(2);
02006     }
02007     return SQ_OK;
02008 }
```

References [sq_aux_paramscheck](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.77 sq_newtable()

```
void sq_newtable (
    HSQUIRRELVM v )
```

新しい空のテーブルを生成してスタックにプッシュする。

初期容量 0 のハッシュテーブルを作成し、対象仮想マシンのスタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 653 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00654 {
00655     v->Push(SQTable::Create(_ss(v), 0));
00656 }
```

4.1.2.78 sq_newtableex()

```
void sq_newtableex (
    HSQUIRRELM v,
    SQInteger initialcapacity )
```

指定した初期容量で新しいテーブルを生成し、スタックにプッシュする。

パフォーマンス最適化のため、事前に想定される要素数を指定して ハッシュテーブルの初期サイズを設定する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>initialcapacity</i>	テーブルの初期要素数（ハッシュサイズの目安）。

Definition at line 668 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00669 {
00670     v->Push(SQTable::Create(_ss(v), initialcapacity));
00671 }
```

4.1.2.79 sq_newthread()

```
HSQUIRRELM sq_newthread (
    HSQUIRRELM friendvm,
    SQInteger initialstacksize )
```

新しいスレッドVM（子VM）を生成する。

指定された親VM (*friendvm*) をベースに共有状態を継承した新しい SQVM を生成する。成功すると生成されたスレッドVM を親VM のスタックにプッシュし、戻り値として返す。初期化に失敗した場合はメモリを解放して NULL を返す。

Parameters

<i>friendvm</i>	親となる HSQUIRRELM インスタンス。
<i>initialstacksize</i>	生成するスレッドVM のスタック初期サイズ。

Returns

HSQUIRRELM 成功時は新しい子VM、失敗時は NULL。

Definition at line 140 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00141 {
00142     SQSharedState *ss;
00143     SQVM *v;
00144     ss=_ss(friendvm);
00145
00146     v= (SQVM *)SQ_MALLOC(sizeof(SQVM));
00147     new (v) SQVM(ss);
00148
00149     if(v->Init(friendvm, initialstacksize)) {
00150         friendvm->Push(v);
00151         return v;
00152     } else {
00153         sq_delete(v, SQVM);
00154         return NULL;
00155     }
00156 }

```

4.1.2.80 sq_newuserdata()

```

SQUserPointer sq_newuserdata (
    HSQUIRRELVm v,
    SQUnsignedInteger size )

```

新しいユーザーデータ領域を確保してスタックにプッシュする。

指定サイズのユーザーデータ (SQUserData) を確保し、アライメントを考慮して 有効なデータ領域の先頭ポインタを返す。データはスタックにもプッシュされる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>size</i>	確保するユーザーデータ領域のサイズ (バイト単位)。

Returns

SQUserPointer 有効なユーザーデータへのポインタ。

Definition at line 638 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00639 {
00640     SQUserData *ud = SQUserData::Create(_ss(v), size + SQ_ALIGNMENT);
00641     v->Push(ud);
00642     return (SQUserPointer)sq_aligning(ud + 1);
00643 }

```

4.1.2.81 sq_next()

```

SQRESULT sq_next (
    HSQUIRRELVm v,
    SQInteger idx )

```

イテレータを進め、次の要素を取得する。

スタックの *idx* にあるオブジェクトを対象に、スタックトップの参照位置から 次の要素を取得する。取得に成功すると、キーと値をスタックにプッシュする。ジェネレータには使用できない。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	失敗。

Definition at line 3257 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03258 {
03259     SQObjectPtr o=stack_get(v,idx),&refpos = stack_get(v,-1),realkey,val;
03260     if(sq_type(o) == OT_GENERATOR) {
03261         return sq_throwerror(v,_SC("cannot iterate a generator"));
03262     }
03263     int faketojump;
03264     if(!v->FOREACH_OP(o,realkey,val,refpos,0,666,faketojump))
03265         return SQ_ERROR;
03266     if(faketojump != 666) {
03267         v->Push(realkey);
03268         v->Push(val);
03269         return SQ_OK;
03270     }
03271     return SQ_ERROR;
03272 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.82 sq_notifyallexceptions()

```

void sq_notifyallexceptions (
    HSQUIRRELVm v,
    SQBool enable )
```

すべての例外に対して通知を行うかどうかを設定する。

通常、通知は未捕捉の例外に対してのみ行われるが、このフラグを有効にすることで 捕捉された例外も含めて通知対象にできる。

この設定は共有状態 `_notifyallexceptions` に保存され、VM の例外処理挙動に影響を与える。

Parameters

<code>v</code>	対象となる仮想マシン。
<code>enable</code>	有効にする場合は <code>true</code> 、無効にする場合は <code>false</code> 。

Definition at line 334 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00335 {
00336     _ss(v)->_notifyallexceptions = enable?true:false;
00337 }
```

4.1.2.83 sq_objtobool()

```
SQBool sq_objtobool (
    const HSQOBJECT * o )
```

オブジェクトをブール値 (SQBool) として取得する。

オブジェクトがブール型である場合、その値を返す。ブール型でない場合は SQFalse を返す。

Parameters

<i>o</i>	対象の HSQOBJECT ポインタ。
----------	---------------------

Returns

SQBool ブール値 (SQTrue または SQFalse)。

Definition at line 497 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00498 {
00499     if(sq_isbool(*o)) {
00500         return _integer(*o);
00501     }
00502     return SQFalse;
00503 }
```

4.1.2.84 sq_objtfloat()

```
SQFloat sq_objtfloat (
    const HSQOBJECT * o )
```

オブジェクトを浮動小数点数 (SQFloat) として取得する。

指定されたオブジェクトが数値型 (整数または浮動小数点) であれば、浮動小数に変換して返す。非数値型の場合は 0.0 を返す。

Parameters

<i>o</i>	対象の HSQOBJECT ポインタ。
----------	---------------------

Returns

SQFloat 浮動小数型での値。非数値型の場合は 0.0。

Definition at line 478 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00479 {
00480     if(sq_isnumeric(*o)) {
00481         return tofloat(*o);
00482     }
00483     return 0;
00484 }
```

4.1.2.85 sq_objtinteger()

```
SQInteger sq_objtinteger (
    const HSQOBJECT * o )
```


オブジェクトを整数 (SQInteger) として取得する。

指定された HSQOBJECT が数値型 (整数または浮動小数) であれば、整数に変換して返す。それ以外の型の場合は 0 を返す。

Parameters

<i>o</i>	対象の HSQOBJECT ポインタ。
----------	---------------------

Returns

SQInteger 数値型であればその整数値、そうでなければ 0。

Definition at line 459 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
00460 {
00461     if(sq_isnumeric(*o)) {
00462         return tointeger(*o);
00463     }
00464     return 0;
00465 }
```

4.1.2.86 sq_objtostring()

```
const SQChar * sq_objtostring (
    const HSQOBJECT * o )
```

オブジェクトを文字列型として取得する。

指定された HSQOBJECT が文字列 (OT_STRING) 型であれば、その文字列へのポインタを返す。それ以外の型であれば NULL を返す。

Parameters

<i>o</i>	対象の HSQOBJECT ポインタ。
----------	---------------------

Returns

const SQChar* オブジェクトが文字列型であればその文字列、そうでなければ NULL。

Definition at line 440 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```
00441 {
00442     if(sq_type(*o) == OT_STRING) {
00443         return _stringval(*o);
00444     }
00445     return NULL;
00446 }
```

4.1.2.87 sq_objtouserpointer()

```
SQUserPointer sq_objtouserpointer (
    const HSQOBJECT * o )
```

オブジェクトをユーザーポインタとして取得する。

指定されたオブジェクトがユーザーポインタ型 (OT_USERPOINTER) である場合、ポインタ値を返す。それ以外の型であれば 0 を返す。

Parameters

<i>o</i>	対象の HSQOBJECT ポインタ。
----------	---------------------

Returns

SQUserPointer オブジェクトがユーザーポインタ型であればその値、そうでなければ 0。

Definition at line 516 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```
00517 {
00518     if(sq_isuserpointer(*o)) {
00519         return _userpointer(*o);
00520     }
00521     return 0;
00522 }
```

4.1.2.88 sq_open()

```
HSQUIRRELVm sq_open (
    SQInteger initialstacksize )
```

新しい Squirrel 仮想マシン (VM) インスタンスを作成し初期化する。

指定された初期スタックサイズで仮想マシンを生成する。内部的に SQSharedState を作成・初期化し、それに紐づいた SQVM を構築する。初期化が失敗した場合はメモリを解放して NULL を返す。

Parameters

<i>initialstacksize</i>	VM スタックの初期サイズ。
-------------------------	----------------

Returns

HSQUIRRELVm 成功時は初期化済みの仮想マシンインスタンス、失敗時は NULL。

Definition at line 109 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```
00110 {
00111     SQSharedState *ss;
00112     SQVM *v;
00113     sq_new(ss, SQSharedState);
00114     ss->Init();
00115     v = (SQVM *)SQ_MALLOC(sizeof(SQVM));
00116     new (v) SQVM(ss);
00117     ss->_root_vm = v;
00118     if(v->Init(NULL, initialstacksize)) {
00119         return v;
00120     } else {
00121         sq_delete(v, SQVM);
00122         return NULL;
00123     }
00124     return v;
00125 }
```

4.1.2.89 sq_pop()

```
void sq_pop (
    HSQUIRRELVm v,
    SQInteger nelemstopop )
```

スタックのトップから指定数の要素をポップする。

スタックの現在のトップから `nelemstopop` 個の要素を削除する。スタックの要素数が指定数未満の場合は内部でアサートが発生する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>nelemstopop</i>	ポップする要素の個数。

Definition at line 1931 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01932 {
01933     assert (v->_top >= nelemstopop);
01934     v->Pop(nelemstopop);
01935 }
```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.90 sq_poptop()

```
void sq_poptop (
    HSQUIRRELVm v )
```

スタックのトップ要素を1つポップする。

スタックのトップにある要素を1つだけ削除する。スタックが空の場合は内部でアサートが発生する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 1946 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01947 {
01948     assert (v->_top >= 1);
01949     v->Pop();
01950 }
```

4.1.2.91 sq_push()

```
void sq_push (
    HSQUIRRELVm v,
    SQInteger idx )
```

スタックに値をプッシュする。

指定インデックス *idx* にある値をスタックトップに複製してプッシュする。インデックスは負値でスタック末尾からの相対指定も可能。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックから複製する値のインデックス。

Definition at line 1407 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01408 {
01409     v->Push(stack_get(v, idx));
01410 }
```

4.1.2.92 sq_pushbool()

```
void sq_pushbool (
    HSQUIRRELVM v,
    SQBool b )
```

スタックにブール値をプッシュする。

指定されたブール値 (true/false) を、仮想マシンのスタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>b</i>	プッシュするブール値 (SQTrue または SQFalse)。

Definition at line 578 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
00579 {
00580     v->Push(b?true:false);
00581 }
```

4.1.2.93 sq_pushconsttable()

```
void sq_pushconsttable (
    HSQUIRRELVM v )
```

定数 (const) テーブルをスタックにプッシュする。

定数テーブルには予約定数やシステム定義の値などが格納されている。これは読み取り専用であり、ユーザーによる直接編集は推奨されない。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 1218 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01219 {
01220     v->Push(_ss(v)->_consts);
01221 }
```

4.1.2.94 sq_pushfloat()

```
void sq_pushfloat (
```

```
HSQUIRRELVM v,
SQFloat n )
```

スタックに浮動小数点数をプッシュする。

指定された SQFloat 型の値を仮想マシンのスタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>n</i>	プッシュする浮動小数点値。

Definition at line 592 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00593 {
00594     v->Push(n);
00595 }
```

4.1.2.95 sq_pushinteger()

```
void sq_pushinteger (
    HSQUIRRELVM v,
    SQInteger n )
```

スタックに整数値をプッシュする。

指定された整数値を仮想マシンのスタック上にプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>n</i>	プッシュする整数値。

Definition at line 564 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00565 {
00566     v->Push(n);
00567 }
```

4.1.2.96 sq_pushnull()

```
void sq_pushnull (
    HSQUIRRELVM v )
```

スタックに null 値をプッシュする。

対象の仮想マシンのスタックトップに null を追加する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 532 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00533 {
```

```
00534     v->PushNull();
00535 }
```

Referenced by [sq_settop\(\)](#).

Here is the caller graph for this function:



4.1.2.97 sq_pushobject()

```
void sq_pushobject (
    HSQUIRRELVm v,
    HSQOBJECT obj )
```

オブジェクトをスタックにプッシュする。

指定された HSQOBJECT 型のオブジェクトを現在の VM のスタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>obj</i>	プッシュするオブジェクト。

Definition at line 2408 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
02409 {
02410     v->Push(SQObjectPtr(obj));
02411 }
```

4.1.2.98 sq_pushregistrytable()

```
void sq_pushregistrytable (
    HSQUIRRELVm v )
```

レジストリテーブルをスタックにプッシュする。

レジストリテーブルは VM の内部管理用途に使用される特殊なテーブルであり、外部からのデータ保存・共有などに使用できる。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 1204 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01205 {
01206     v->Push(_ss(v)->_registry);
01207 }

```

4.1.2.99 sq_pushroottable()

```

void sq_pushroottable (
    HSQUIRRELVM v )

```

現在の仮想マシンに設定されているルートテーブルをスタックにプッシュする。

ルートテーブルは、グローバルスコープに相当するテーブルであり、スクリプトの変数や関数の格納先となる。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 1190 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01191 {
01192     v->Push(v->_roottable);
01193 }

```

4.1.2.100 sq_pushstring()

```

void sq_pushstring (
    HSQUIRRELVM v,
    const SQChar * s,
    SQInteger len )

```

スタックに文字列をプッシュする。

指定された文字列（*s*）を仮想マシンのスタックにプッシュする。文字列が `NULL` の場合は `null` をプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>s</i>	プッシュする文字列へのポインタ。 <code>NULL</code> の場合は <code>null</code> をプッシュ。
<i>len</i>	文字列の長さ（バイト数）。

Definition at line 548 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00549 {
00550     if (s)
00551         v->Push(SQObjectPtr(SQString::Create(_ss(v), s, len)));
00552     else v->PushNull();
00553 }

```

4.1.2.101 sq_pushthread()

```

void sq_pushthread (
    HSQUIRRELVM v,
    HSQUIRRELVM thread )

```


スレッド (VM インスタンス) をスタックにプッシュする。

指定されたスレッド型の仮想マシンインスタンスを、thread 型として 対象 VM のスタックにプッシュする。

Parameters

<i>v</i>	スレッドをプッシュする側の仮想マシン。
<i>thread</i>	プッシュされるスレッド型仮想マシン。

Definition at line 621 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00622 {
00623     v->Push(thread);
00624 }
```

4.1.2.102 sq_pushuserpointer()

```
void sq_pushuserpointer (
    HSQUIRRELVM v,
    SQUserPointer p )
```

スタックにユーザーポインタをプッシュする。

任意のポインタ値を userpointer 型として仮想マシンのスタックにプッシュする。

Parameters

<i>v</i>	対象の仮想マシン。
<i>p</i>	プッシュするユーザーポインタ。

Definition at line 606 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00607 {
00608     v->Push(p);
00609 }
```

4.1.2.103 sq_rawdeleteslot()

```
SQRESULT sq_rawdeleteslot (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool pushval )
```

テーブルのスロットを生で削除する。

スタックの idx にあるテーブルから、スタックトップのキーに対応する スロットを削除する。削除した値を残すかどうかは pushval で制御する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象テーブルのスタックインデックス。
<i>pushval</i>	削除した値をプッシュする場合 true。

Return values

<code>SQ_OK</code>	成功。
--------------------	-----

Definition at line 2226 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02227 {
02228     sq_aux_paramscheck(v, 2);
02229     SQObjectPtr *self;
02230     _GETSAFE_OBJ(v, idx, OT_TABLE, self);
02231     SQObjectPtr &key = v->GetUp(-1);
02232     SQObjectPtr t;
02233     if(_table(*self)->Get(key,t)) {
02234         _table(*self)->Remove(key);
02235     }
02236     if(pushval != 0)
02237         v->GetUp(-1) = t;
02238     else
02239         v->Pop();
02240     return SQ_OK;
02241 }
```

References [_GETSAFE_OBJ](#), and [sq_aux_paramscheck](#).

4.1.2.104 sq_rawget()

```

SQLRESULT sq_rawget (
    HSQUIRRELM v,
    SQInteger idx )
```

テーブル・クラス・インスタンスなどから値を生で取得する。

スタックの `idx` にあるオブジェクトから、スタックトップのキーを使って値を取得する。この関数はフォールバックを行わない低レベルの取得を行う。無効な型やインデックスの場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	失敗。

Definition at line 2307 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02308 {
02309     SQObjectPtr &self=stack_get(v,idx);
02310     SQObjectPtr &obj = v->GetUp(-1);
02311     switch(sq_type(self)) {
02312     case OT_TABLE:
02313         if(_table(self)->Get(obj,obj))
02314             return SQ_OK;
02315         break;
02316     case OT_CLASS:
02317         if(_class(self)->Get(obj,obj))
02318             return SQ_OK;
02319         break;
02320     case OT_INSTANCE:
02321         if(_instance(self)->Get(obj,obj))
02322             return SQ_OK;
02323         break;
02324     case OT_ARRAY:{
02325         if(sq_isnumeric(obj)){
```

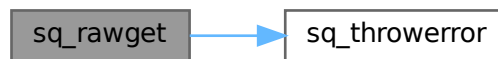
```

02326         if(_array(self)->Get(tointeger(obj),obj)) {
02327             return SQ_OK;
02328         }
02329     }
02330     else {
02331         v->Pop();
02332         return sq_throwerror(v,_SC("invalid index type for an array"));
02333     }
02334     }
02335     break;
02336     default:
02337         v->Pop();
02338         return sq_throwerror(v,_SC("rawget works only on array/table/instance and class"));
02339     }
02340     v->Pop();
02341     return sq_throwerror(v,_SC("the index doesn't exist"));
02342 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.105 sq_rawnewmember()

```

SQRESULT sq_rawnewmember (
    HSQUIRRELM v,
    SQInteger idx,
    SQBool bstatic )

```

クラスに新しいメンバーを生で追加する。

スタックの `idx` にあるオブジェクトがクラス型の場合に、スタックトップのキー・値・属性を用いて新しいメンバーを生で追加する（基底クラスのスロットに直接追加）。メンバーを静的にするかどうかは `bstatic` で制御する。クラス型以外の場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象クラスのスタックインデックス。
<code>bstatic</code>	静的メンバーとして追加する場合 <code>true</code> 。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クラス型でない場合、またはキーが <code>null</code> の場合。

Definition at line 2156 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02157 {
02158     SQObjectPtr &self = stack_get(v, idx);
02159     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
02160     SQObjectPtr &key = v->GetUp(-3);
02161     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
02162     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, true)) {
02163         v->Pop(3);
02164         return SQ_ERROR;
02165     }
02166     v->Pop(3);
02167     return SQ_OK;
02168 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.106 sq_rawset()

```

SQRESULT sq_rawset (
    HSQUIRRELVm v,
    SQInteger idx )

```

テーブル・クラス・インスタンスなどのスロットを生で設定する。

スタックの `idx` にあるオブジェクトに対して、スタックトップのキーと値を生で設定する。型に応じて適切なスロット追加を行う。対応しない型の場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	失敗または無効なキー。

Definition at line 2075 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02076 {
02077     SQObjectPtr &self = stack_get(v, idx);
02078     SQObjectPtr &key = v->GetUp(-2);
02079     if(sq_type(key) == OT_NULL) {
02080         v->Pop(2);
02081         return sq_throwerror(v, _SC("null key"));
02082     }
02083     switch(sq_type(self)) {
02084     case OT_TABLE:
02085         _table(self)->NewSlot(key, v->GetUp(-1));

```

```

02086         v->Pop(2);
02087         return SQ_OK;
02088     break;
02089     case OT_CLASS:
02090         _class(self)->NewSlot(_ss(v), key, v->GetUp(-1), false);
02091         v->Pop(2);
02092         return SQ_OK;
02093     break;
02094     case OT_INSTANCE:
02095         if(_instance(self)->Set(key, v->GetUp(-1))) {
02096             v->Pop(2);
02097             return SQ_OK;
02098         }
02099     break;
02100     case OT_ARRAY:
02101         if(v->Set(self, key, v->GetUp(-1), false)) {
02102             v->Pop(2);
02103             return SQ_OK;
02104         }
02105     break;
02106     default:
02107         v->Pop(2);
02108         return sq_throwerror(v, _SC("rawset works only on array/table/class and instance"));
02109     }
02110     v->Raise_IdxError(v->GetUp(-2)); return SQ_ERROR;
02111 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.107 sq_readclosure()

```

SQLRESULT sq_readclosure (
    HSQUIRRELM v,
    SQREADFUNC r,
    SQUserPointer up )

```

バイトコードストリームからクロージャを読み込む。

バイトコードストリームからクロージャを復元して、スタックにプッシュする。ストリームのタグが不正または読み込みエラーが発生した場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>r</i>	読み込み関数。
<i>up</i>	ユーザーポインタ。

Return values

<i>SQ_OK</i>	成功。
--------------	-----

Return values

<code>SQ_ERROR</code>	IO エラーまたはストリームタグ不正。
-----------------------	---------------------

Definition at line 2749 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02750 {
02751     SQObjectPtr closure;
02752
02753     unsigned short tag;
02754     if(r(up,&tag,2) != 2)
02755         return sq_throwerror(v,_SC("io error"));
02756     if(tag != SQ_BYTECODE_STREAM_TAG)
02757         return sq_throwerror(v,_SC("invalid stream"));
02758     if(!SQClosure::Load(v,up,r,closure))
02759         return SQ_ERROR;
02760     v->Push(closure);
02761     return SQ_OK;
02762 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.108 sq_realloc()

```

void * sq_realloc (
    void * p,
    SQUnsignedInteger oldsize,
    SQUnsignedInteger newsize )
```

メモリを再確保する。

既存のメモリ領域 *p* を新しいサイズ *newsize* に再確保する。元のサイズ *oldsize* を指定することで適切に再割り当てされる。

Parameters

<i>p</i>	再確保するメモリブロック。
<i>oldsize</i>	元のサイズ。
<i>newsize</i>	新しいサイズ。

Returns

`void*` 再確保されたメモリ領域へのポインタ。

Definition at line 3417 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03418 {
03419     return SQ_REALLOC(p,oldsize,newsize);
03420 }

```

4.1.2.109 sq_release()

```

SQBool sq_release (
    HSQUIRRELVm v,
    HSQOBJECT * po )

```

指定されたオブジェクトの参照カウントを減少させ、必要に応じて解放する。

ガーベジコレクションが有効でない構成では、参照カウントが 1 以下の場合に解放される。

ガーベジコレクションが有効な場合は、_refs_table に管理を委ねて Release() を呼び出す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>po</i>	対象の HSQOBJECT ポインタ。

Return values

<i>SQTrue</i>	解放されたか、そもそも参照カウントを持たないオブジェクトだった場合。
<i>SQFalse</i>	ガーベジコレクタなし構成で解放されなかった場合（※副作用なし）。

Definition at line 399 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00400 {
00401     if (!ISREFCOUNTED(sq_type(*po))) return SQTrue;
00402     #ifdef NO_GARBAGE_COLLECTOR
00403         bool ret = (po->_unVal.pRefCounted->_uiRef <= 1) ? SQTrue : SQFalse;
00404         __Release(po->_type,po->_unVal);
00405         return ret; //the ret val doesn't work(and cannot be fixed)
00406     #else
00407         return _ss(v)->_refs_table.Release(*po);
00408     #endif
00409 }

```

4.1.2.110 sq_remove()

```

void sq_remove (
    HSQUIRRELVm v,
    SQInteger idx )

```

スタックの指定インデックスの要素を削除する。

スタック内の指定されたインデックス *idx* にある要素を削除し、それ以降の要素を詰めて順序を維持する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	削除する要素のスタックインデックス。

Definition at line 1962 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01963 {
01964     v->Remove(idx);
01965 }

```

4.1.2.111 sq_reservestack()

```

SQRESULT sq_reservestack (
    HSQUIRRELVM v,
    SQInteger nsize )

```

スタックサイズを予約する。

スタックの必要サイズ `nsize` を予約する。現在のスタックサイズが不足している場合、メタメソッドの実行中でない限りスタックを拡張する。メタメソッド実行中の場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>nsize</code>	予約したいスタックサイズ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	メタメソッド実行中の場合。

Definition at line 2502 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02503 {
02504     if (((SQUnsignedInteger)v->_top + nsize) > v->_stack.size()) {
02505         if (v->_nmetamethodscall) {
02506             return sq_throwerror(v, _SC("cannot resize stack while in a metamethod"));
02507         }
02508         v->_stack.resize(v->_stack.size() + ((v->_top + nsize) - v->_stack.size()));
02509     }
02510     return SQ_OK;
02511 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.112 sq_reseterror()

```

void sq_reseterror (
    HSQUIRRELVM v )

```


ラストエラーをリセットする。

仮想マシンに設定されているラストエラーを NULL に初期化する。エラー処理後にエラー状態をクリアしたい場合に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Definition at line 2471 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02472 {
02473     v->_lasterror.Null();
02474 }
```

4.1.2.113 sq_resetobject()

```
void sq_resetobject (
    HSQOBJECT * po )
```

オブジェクトを初期化状態にリセットする。

指定された HSQOBJECT を NULL 型に初期化し、ユーザーポインタをクリアする。オブジェクトの再利用時に使用する。

Parameters

<i>po</i>	リセットするオブジェクトポインタ。
-----------	-------------------

Definition at line 2422 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02423 {
02424     po->_unVal.pUserPointer=NULL;
02425     po->_type=OT_NULL;
02426 }
```

4.1.2.114 sq_resume()

```
SQRESULT sq_resume (
    HSQUIRRELVm v,
    SQBool retval,
    SQBool raiseerror )
```

サスペンド状態のジェネレータを再開する。

スタックトップにあるジェネレータを再開して実行を継続する。対象がジェネレータ以外の場合はエラー。

Parameters

<i>v</i>	対象の仮想マシン。
<i>retval</i>	戻り値を残す場合は true。
<i>raiseerror</i>	エラー発生時に例外を送出する場合は true。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	ジェネレータ以外の場合や失敗時。

Definition at line 2525 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```

02526 {
02527     if (sq_type(v->GetUp(-1)) == OT_GENERATOR)
02528     {
02529         v->PushNull(); //retval
02530         if (!v->Execute(v->GetUp(-2), 0, v->_top, v->GetUp(-1), raiseerror,
SQVM::ET_RESUME_GENERATOR))
02531         {v->Raise_Error(v->_lasterror); return SQ_ERROR;}
02532         if(!retval)
02533             v->Pop();
02534         return SQ_OK;
02535     }
02536     return sq_throwerror(v,_SC("only generators can be resumed"));
02537 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.115 sq_resurrectunreachable()

```

SQLRESULT sq_resurrectunreachable (
    HSQUIRRELVm v )
```

到達不能オブジェクトを復活させる。

ガーベジコレクタによって到達不能と判定されたオブジェクトを復活させる。ビルドにガーベジコレクタが含まれていない場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	ガーベジコレクタ非対応ビルドの場合。

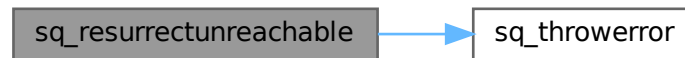
Definition at line 2791 of file squirrel3_squirrel_sqapiByOpenAI.cpp.

```

02792 {
02793     #ifndef NO_GARBAGE_COLLECTOR
02794         _ss(v)->ResurrectUnreachable(v);
02795         return SQ_OK;
02796     #else
02797         return sq_throwerror(v,_SC("sq_resurrectunreachable requires a garbage collector build"));
02798     #endif
02799 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.116 sq_set()

```

SQRESULT sq_set (
    HSQUIRRELVm v,
    SQInteger idx )
  
```

テーブルまたはクラスのスロットを設定する。

スタックの `idx` にあるオブジェクト（テーブルやクラスなど）に対して、スタックトップのキーと値を使用してスロットを設定する。キーが `null` の場合などはエラーになる。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	失敗。

Definition at line 2053 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02054 {
02055     SQObjectPtr &self = stack_get(v, idx);
02056     if(v->Set(self, v->GetUp(-2), v->GetUp(-1), DONT_FALL_BACK)) {
02057         v->Pop(2);
02058         return SQ_OK;
02059     }
02060     return SQ_ERROR;
02061 }
  
```

4.1.2.117 sq_setattributes()

```

SQRESULT sq_setattributes (
    HSQUIRRELVm v,
    SQInteger idx )
  
```

クラスのメンバー属性を設定する。

スタックの `idx` にあるクラスオブジェクトのメンバー属性を設定する。メンバー名が `null` の場合はクラス全体の属性を設定し、それ以外の場合は該当メンバーの属性を更新する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クラスのスタックインデックス。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	無効なインデックスの場合。

Definition at line 2933 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

02934 {
02935     SQObjectPtr *o = NULL;
02936     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02937     SQObjectPtr &key = stack_get(v,-2);
02938     SQObjectPtr &val = stack_get(v,-1);
02939     SQObjectPtr attrs;
02940     if(sq_type(key) == OT_NULL) {
02941         attrs = _class(*o)->_attributes;
02942         _class(*o)->_attributes = val;
02943         v->Pop(2);
02944         v->Push(attrs);
02945         return SQ_OK;
02946     }else if(_class(*o)->GetAttributes(key,attrs)) {
02947         _class(*o)->SetAttributes(key,val);
02948         v->Pop(2);
02949         v->Push(attrs);
02950         return SQ_OK;
02951     }
02952     return sq_throwerror(v,_SC("wrong index"));
02953 }
```

References [_GETSAFE_OBJ](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.118 sq_setbyhandle()

```

SQLRESULT sq_setbyhandle (
    HSQUIRRELVm v,
    SQInteger idx,
    const HSQMEMBERHANDLE * handle )
```

メンバーのハンドル情報を使って値を設定する。

スタックの *idx* にあるクラスまたはインスタンスオブジェクトの、指定されたハンドルのメンバーに スタックトップの値を設定する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックインデックス。
<i>handle</i>	メンバーのハンドル情報。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	失敗。

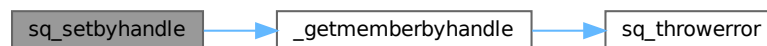
Definition at line 3098 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03099 {
03100     SQObjectPtr &self = stack_get(v,idx);
03101     SQObjectPtr &newval = stack_get(v,-1);
03102     SQObjectPtr *val = NULL;
03103     if(SQ_FAILED(_getmemberbyhandle(v,self,handle,val))) {
03104         return SQ_ERROR;
03105     }
03106     *val = newval;
03107     v->Pop();
03108     return SQ_OK;
03109 }
```

References [_getmemberbyhandle\(\)](#).

Here is the call graph for this function:



4.1.2.119 sq_setclassudsize()

```

SQRESULT sq_setclassudsize (
    HSQUIRRELVm v,
    SQInteger idx,
    SQInteger udsiz )
```

クラスのユーザーデータサイズを設定する。

スタックの *idx* にあるオブジェクトがクラス型である場合に、そのクラスのユーザーデータ領域のサイズを *udsiz* に設定する。ロックされたクラスには設定できない。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クラスのスタックインデックス。
<i>udsiz</i>	設定するユーザーデータサイズ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クラス型でない場合、またはクラスがロックされている場合。

Definition at line 1844 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01845 {
01846     SQObjectPtr &o = stack_get(v, idx);
01847     if(sq_type(o) != OT_CLASS) return sq_throwerror(v, _SC("the object is not a class"));
01848     if(_class(o)->_locked) return sq_throwerror(v, _SC("the class is locked"));
01849     _class(o)->_udsize = udsiz;
01850     return SQ_OK;
01851 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.120 sq_setclosureroot()

```

SQRESULT sq_setclosureroot (
    HSQUIRRELVM v,
    SQInteger idx )
```

クロージャにルートテーブルを設定する。

指定されたクロージャ (OT_CLOSURE) に対して、新しいルートテーブルを設定する。

スタックの -1 にあるオブジェクトが `table` 型である必要がある。設定後はそのオブジェクトはポップされる。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象のクロージャのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	対象がクロージャでない、または与えられたルートがテーブルでない場合。

Definition at line 1119 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01120 {
01121     SQObjectPtr &c = stack_get(v, idx);
01122     SQObject o = stack_get(v, -1);
01123     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
01124     if(sq_istable(o)) {
01125         _closure(c)->SetRoot(_table(o)->GetWeakRef(OT_TABLE));
01126         v->Pop();
01127         return SQ_OK;
01128     }
01129     return sq_throwerror(v, _SC("invalid type"));
01130 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.121 sq_setcompilererrorhandler()

```

void sq_setcompilererrorhandler (
    HSQUIRRELVm v,
    SQCOMPILERERROR f )

```

コンパイラエラーハンドラを設定する。

指定されたコンパイラエラーハンドラ関数を仮想マシンに設定する。スクリプトのコンパイル時に発生するエラーをカスタム処理したい場合に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>f</i>	設定するコンパイラエラーハンドラ関数。

Definition at line 2704 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02705 {
02706     _ss(v)->_compilererrorhandler = f;
02707 }

```

4.1.2.122 sq_setconsttable()

```

SQRESULT sq_setconsttable (
    HSQUIRRELVm v )

```

仮想マシンに新しい定数テーブルを設定する。

スタックのトップにあるテーブルを、共有状態 `_consts` に設定する。定数テーブルには予約語や定義済みの値などが格納される。

設定後、スタックトップはポップされる。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	スタックトップがテーブル型でない場合。

Definition at line 1262 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

01263 {
01264     SQObject o = stack_get(v, -1);
01265     if(sq_istable(o)) {
01266         _ss(v)->_consts = o;
01267         v->Pop();
01268         return SQ_OK;
01269     }
01270     return sq_throwerror(v, _SC("invalid type, expected table"));
01271 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.123 sq_setdebughook()

```

void sq_setdebughook (
    HSQUIRRELVM v )
```

スクリプトレベルのデバッグフッククロージャを設定する。

スタックのトップにあるオブジェクトをデバッグフッククロージャとして設定する。オブジェクトはクロージャ、ネイティブクロージャ、または null である必要がある。設定により、ネイティブのデバッグフックは無効化される (`_debughook_native = NULL`)。

設定後はスタックのトップをポップする。

Parameters

<i>v</i>	対象となる仮想マシンインスタンス。
----------	-------------------

Definition at line 233 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

00234 {
00235     SQObject o = stack_get(v, -1);
```

```

00236     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00237         v->_debughook_closure = o;
00238         v->_debughook_native = NULL;
00239         v->_debughook = !sq_isnull(o);
00240         v->Pop();
00241     }
00242 }

```

4.1.2.124 sq_setdelegate()

```

SQLRESULT sq_setdelegate (
    HSQUIRRELVm v,
    SQInteger idx )

```

テーブルまたはユーザーデータにデリゲートを設定する。

スタックの `idx` にあるオブジェクトがテーブルまたはユーザーデータ型の場合、スタックトップのオブジェクトをデリゲートとして設定する。無効な型の場合や、デリゲートがサイクルする場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	無効な型やサイクルが検出された場合。

Definition at line 2183 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

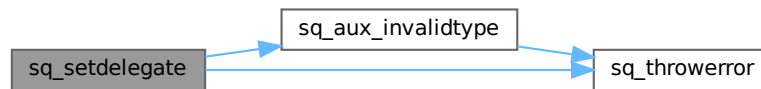
```

2184 {
2185     SQObjectPtr &self = stack_get(v, idx);
2186     SQObjectPtr &mt = v->GetUp(-1);
2187     SQObjectType type = sq_type(self);
2188     switch(type) {
2189     case OT_TABLE:
2190         if(sq_type(mt) == OT_TABLE) {
2191             if(!_table(self)->SetDelegate(_table(mt))) {
2192                 return sq_throwerror(v, _SC("delegate cycle"));
2193             }
2194             v->Pop();
2195         }
2196         else if(sq_type(mt) == OT_NULL) {
2197             _table(self)->SetDelegate(NULL); v->Pop(); }
2198         else return sq_aux_invalidtype(v, type);
2199         break;
2200     case OT_USERDATA:
2201         if(sq_type(mt) == OT_TABLE) {
2202             _userdata(self)->SetDelegate(_table(mt)); v->Pop(); }
2203         else if(sq_type(mt) == OT_NULL) {
2204             _userdata(self)->SetDelegate(NULL); v->Pop(); }
2205         else return sq_aux_invalidtype(v, type);
2206         break;
2207     default:
2208         return sq_aux_invalidtype(v, type);
2209         break;
2210     }
2211     return SQ_OK;
2212 }

```

References `sq_aux_invalidtype()`, and `sq_throwerror()`.

Here is the call graph for this function:



4.1.2.125 sq_seterrorhandler()

```
void sq_seterrorhandler (
    HSQUIRRELVM v )
```

仮想マシンにエラーハンドラを設定する。

スタックのトップにあるオブジェクトをエラーハンドラとして設定する。対象はクロージャ (closure)、ネイティブクロージャ (nativeclosure)、または null のいずれかである必要がある。

設定後はスタックのトップをポップする。

Parameters

<i>v</i>	対象となる仮想マシンインスタンス。
----------	-------------------

Definition at line 195 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00196 {
00197     SQObject o = stack_get(v, -1);
00198     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00199         v->_errorhandler = o;
00200         v->Pop();
00201     }
00202 }
```

4.1.2.126 sq_setforeignptr()

```
void sq_setforeignptr (
    HSQUIRRELVM v,
    SQUserPointer p )
```

仮想マシンに外部ポインタを関連付ける。

指定された任意のユーザーポインタを仮想マシンに設定する。このポインタは、C/C++ 側で VM に関連する追加情報を保持したい場合などに使用される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>p</i>	任意のユーザーポインタ。

Definition at line 1283 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01284 {
01285     v->_foreignptr = p;
01286 }
```

4.1.2.127 sq_setfreevariable()

```
SQRESULT sq_setfreevariable (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUnsignedInteger nval )
```

クロージャの自由変数を設定する。

スタックの `idx` にあるクロージャまたはネイティブクロージャの指定されたインデックスの 自由変数に、スタックトップの値を設定する。インデックスが無効な場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックインデックス。
<i>nval</i>	設定する自由変数のインデックス。

Return values

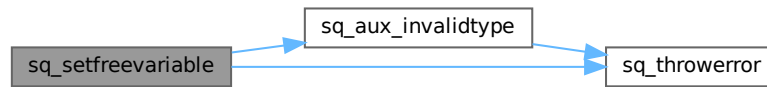
<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	無効なインデックスや型の場合。

Definition at line 2895 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
02896 {
02897     SQObjectPtr &self=stack_get(v,idx);
02898     switch(sq_type(self))
02899     {
02900     case OT_CLOSURE:{
02901         SQFunctionProto *fp = _closure(self)->_function;
02902         if(((SQUnsignedInteger)fp->_noutervalues) > nval){
02903             *(_outer(_closure(self)->_outervalues[nval])->_valptr) = stack_get(v,-1);
02904         }
02905         else return sq_throwerror(v,_SC("invalid free var index"));
02906     }
02907     break;
02908     case OT_NATIVECLOSURE:
02909         if(_nativeclosure(self)->_noutervalues > nval){
02910             _nativeclosure(self)->_outervalues[nval] = stack_get(v,-1);
02911         }
02912         else return sq_throwerror(v,_SC("invalid free var index"));
02913         break;
02914     default:
02915         return sq_aux_invalidtype(v, sq_type(self));
02916     }
02917     v->Pop();
02918     return SQ_OK;
02919 }
```

References [sq_aux_invalidtype\(\)](#), and [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.128 sq_setinstanceup()

```

SQRESULT sq_setinstanceup (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUserPointer p )
  
```

インスタンスにユーザーポインタを設定する。

スタックの `idx` にあるオブジェクトがクラスインスタンス型である場合、そのインスタンスに対して `p` で指定されたユーザーポインタを紐付ける。クラスインスタンス以外の場合はエラーを返す。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象インスタンスのスタックインデックス。
<code>p</code>	設定するユーザーポインタ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クラスインスタンスでない場合。

Definition at line 1822 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

01823 {
01824     SQObjectPtr &o = stack_get(v, idx);
01825     if(sq_type(o) != OT_INSTANCE) return sq_throwerror(v, _SC("the object is not a class instance"));
01826     _instance(o)->_userpointer = p;
01827     return SQ_OK;
01828 }
  
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.129 sq_setnativeclosurename()

```
SQRESULT sq_setnativeclosurename (
    HSQUIRRELVM v,
    SQInteger idx,
    const SQChar * name )
```

ネイティブクロージャに名前を設定する。

スタックの指定インデックスにあるネイティブクロージャ (OT_NATIVECLOSURE) に与えられた名前文字列を設定する。

この名前はデバッグ時の表示や識別に使用される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	対象クロージャのスタックインデックス。
<i>name</i>	設定する名前文字列。

Return values

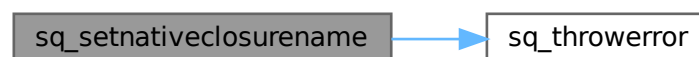
<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がネイティブクロージャでない場合。

Definition at line 972 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
00973 {
00974     SQObject o = stack_get(v, idx);
00975     if(sq_isclosure(o)) {
00976         SQNativeClosure *nc = _nativeclosure(o);
00977         nc->_name = SQString::Create(_ss(v), name);
00978         return SQ_OK;
00979     }
00980     return sq_throwerror(v, _SC("the object is not a nativeclosure"));
00981 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.130 sq_setnativedebughook()

```
void sq_setnativedebughook (
    HSQUIRRELVM v,
    SQDEBUGHOOK hook )
```

ネイティブのデバッグフック関数を設定する。

指定された関数ポインタ `hook` を VM に設定し、ネイティブのデバッグ出力を有効にする。同時に、スクリプトレベルのデバッグクロージャ (`_debughook_closure`) はリセット (`null`) される。

Parameters

<code>v</code>	対象となる仮想マシンインスタンス。
<code>hook</code>	設定するデバッグフック関数 (関数ポインタ)。NULL の場合は無効化。

Definition at line 214 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
00215 {
00216     v->_debughook_native = hook;
00217     v->_debughook_closure.Null();
00218     v->_debughook = hook?true:false;
00219 }
```

4.1.2.131 sq_setparamscheck()

```
SQRESULT sq_setparamscheck (
    HSQUIRRELVm v,
    SQInteger nparamscheck,
    const SQChar * typemask )
```

ネイティブクロージャの引数チェックルールを設定する。

スタックのトップにあるネイティブクロージャに対して、期待される引数の個数 (`nparamscheck`) と、型の制限を示すマスク文字列 (`typemask`) を設定する。

`typemask` 例：

- `"iis"` → int, int, string

`SQ_MATCHTYPMASKSTRING` を引数数に設定すると、マスクの長さから引数数が自動的に決定される。

Parameters

<code>v</code>	対象の仮想マシン。
<code>nparamscheck</code>	引数の期待個数。 <code>SQ_MATCHTYPMASKSTRING</code> を指定することでマスクと一致させる。
<code>typemask</code>	型制約を示すマスク文字列 (省略可能)。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	クロージャでない、または <code>typemask</code> が無効な場合。

Definition at line 1002 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01003 {
01004     SQObject o = stack_get(v, -1);
01005     if(!sq_isnativeclosure(o))
01006         return sq_throwerror(v, _SC("native closure expected"));
01007     SQNativeClosure *nc = _nativeclosure(o);
01008     nc->nparamscheck = nparamscheck;
```

```

01009     if(typemask) {
01010         SQIntVec res;
01011         if(!CompileTypemask(res, typemask))
01012             return sq_throwerror(v, _SC("invalid typemask"));
01013         nc->_typecheck.copy(res);
01014     }
01015     else {
01016         nc->_typecheck.resize(0);
01017     }
01018     if(nparamscheck == SQ_MATCHTYPEMASKSTRING) {
01019         nc->_nparamscheck = nc->_typecheck.size();
01020     }
01021     return SQ_OK;
01022 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.132 sq_setprintfunc()

```

void sq_setprintfunc (
    HSQUIRRELVN v,
    SQPRINTFUNCTION printfunc,
    SQPRINTFUNCTION errfunc )

```

出力関数とエラー出力関数を設定する。

VM が printf などを使用する出力関数とエラー出力関数を設定する。スクリプトのログ出力やエラーログをカスタマイズする際に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>printfunc</i>	標準出力用のコールバック関数。
<i>errfunc</i>	エラー出力用のコールバック関数。

Definition at line 3356 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

03357 {
03358     _ss(v)->_printfunc = printfunc;
03359     _ss(v)->_errorfunc = errfunc;
03360 }

```

4.1.2.133 sq_setreleasehook()

```

void sq_setreleasehook (
    HSQUIRRELVN v,

```



```
SQInteger idx,
SQRELEASEHOOK hook )
```

オブジェクトにリリースフックを設定する。

スタックの `idx` にあるオブジェクトがユーザーデータ・インスタンス・クラス型の場合、指定されたリリースフック関数を登録する。他の型では何も行わない。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。
<code>hook</code>	設定するリリースフック関数。

Definition at line 2661 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
02662 {
02663     SQObjectPtr &ud=stack_get(v,idx);
02664     switch(sq_type(ud) ) {
02665         case OT_USERDATA:  _userdata(ud)->_hook = hook;      break;
02666         case OT_INSTANCE:  _instance(ud)->_hook = hook;      break;
02667         case OT_CLASS:     _class(ud)->_hook = hook;         break;
02668         default: return;
02669     }
02670 }
```

4.1.2.134 sq_setroottable()

```
SQRESULT sq_setroottable (
    HSQUIRRELVM v )
```

仮想マシンに新しいルートテーブルを設定する。

スタックのトップにあるテーブルまたは `null` を、仮想マシンのグローバルルートテーブルとして設定する。

設定後、スタックのトップ要素はポップされる。

Parameters

<code>v</code>	対象の仮想マシン。
----------------	-----------

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	対象がテーブルでも <code>null</code> でもない場合。

Definition at line 1237 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```
01238 {
01239     SQObject o = stack_get(v, -1);
01240     if(sq_istable(o) || sq_isnull(o)) {
01241         v->_roottable = o;
01242         v->Pop();
01243         return SQ_OK;
01244     }
01245     return sq_throwerror(v, _SC("invalid type"));
01246 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.135 sq_setsharedforeignptr()

```
void sq_setsharedforeignptr (
    HSQUIRRELVm v,
    SQUserPointer p )
```

共有状態に外部ポインタを設定する。

このポインタはすべての VM インスタンスに共有される。VM 単体の `sq_setforeignptr()` とは異なり、グローバルに有効な外部情報を保持したい場合に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
<i>p</i>	任意のユーザーポインタ。

Definition at line 1314 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01315 {
01316     _ss(v)->_foreignptr = p;
01317 }
```

4.1.2.136 sq_setsharedreleasehook()

```
void sq_setsharedreleasehook (
    HSQUIRRELVm v,
    SQRELEASEHOOK hook )
```

共有状態のリリースフック関数を設定する。

VM の共有領域 (SharedState) が解放される際に実行されるフック関数を指定する。これは複数の VM 間で共有されるリソースの管理や後処理に使用される。

Parameters

<i>v</i>	対象の仮想マシン。
<i>hook</i>	共有状態が破棄されるときに呼ばれる関数ポインタ。

Definition at line 1376 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01377 {
```

```

01378     _ss(v)->_releasehook = hook;
01379 }

```

4.1.2.137 sq_settop()

```

void sq_settop (
    HSQUIRRELVM v,
    SQInteger newtop )

```

スタックのトップを新しい位置に設定する。

スタックの現在のトップを `newtop` に設定する。現在のトップが新しい値より大きい場合、超過分の要素をポップする。逆に小さい場合は `null` をプッシュして補う。

Parameters

<code>v</code>	対象の仮想マシン。
<code>newtop</code>	新しく設定するスタックトップのインデックス。

Definition at line 1912 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

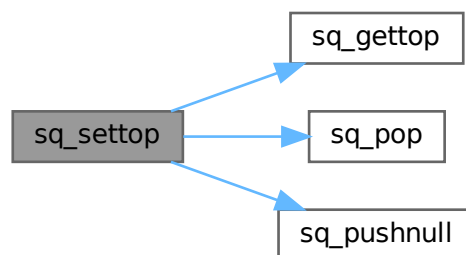
```

01913 {
01914     SQInteger top = sq_gettop(v);
01915     if(top > newtop)
01916         sq_pop(v, top - newtop);
01917     else
01918         while(top++ < newtop) sq_pushnull(v);
01919 }

```

References [sq_gettop\(\)](#), [sq_pop\(\)](#), and [sq_pushnull\(\)](#).

Here is the call graph for this function:



4.1.2.138 sq_settypetag()

```

SQRESULT sq_settypetag (
    HSQUIRRELVM v,
    SQInteger idx,
    SQUserPointer typetag )

```

指定インデックスのオブジェクトに型タグを設定する。

スタックの `idx` にあるオブジェクトがユーザーデータまたはクラス型の場合、与えられた型タグを設定する。それ以外の型の場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>idx</i>	スタックインデックス。
<i>typetag</i>	設定する型タグ。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	対象がユーザーデータでもクラスでもない場合。

Definition at line 1733 of file squirrel3_squirrel_sqapiByOpenAl.cpp.

```

01734 {
01735     SQObjectPtr &o = stack_get(v, idx);
01736     switch(sq_type(o)) {
01737         case OT_USERDATA:  _userdata(o)->_typetag = typetag;    break;
01738         case OT_CLASS:     _class(o)->_typetag = typetag;      break;
01739         default:           return sq_throwerror(v, _SC("invalid object type"));
01740     }
01741     return SQ_OK;
01742 }
```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.139 sq_setvmreleasehook()

```

void sq_setvmreleasehook (
    HSQUIRRELVm v,
    SQRELEASEHOOK hook )
```

仮想マシンの解放時に実行されるリリースフックを設定する。

VM が破棄される際にコールされるユーザー定義の解放処理（フック）関数を登録する。VM のスコープ内で外部リソース管理を行う用途などに便利。

Parameters

<i>v</i>	対象の仮想マシン。
<i>hook</i>	解放時に呼び出す関数ポインタ。

Definition at line 1345 of file squirrel3_squirrel_sqapiByOpenAl.cpp.

```

01346 {
01347     v->_releasehook = hook;
01348 }

```

4.1.2.140 sq_suspendvm()

```

SQRESULT sq_suspendvm (
    HSQUIRRELVM v )

```

仮想マシンをサスペンド状態にする。

実行中の仮想マシンをサスペンド状態にして、後で再開できるようにする。 コルーチンやジェネレータの制御フローに使用される。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Return values

<i>SQ_OK</i>	サスペンド成功時。
--------------	-----------

Definition at line 2609 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02610 {
02611     return v->Suspend();
02612 }

```

4.1.2.141 sq_tailcall()

```

SQRESULT sq_tailcall (
    HSQUIRRELVM v,
    SQInteger nparams )

```

関数またはクロージャを末尾呼び出しする。

スタックトップにある関数（クロージャ）を末尾呼び出しとして実行する。 ジェネレータの末尾呼び出しはできない。 クロージャ以外のオブジェクトの場合はエラーを返す。

Parameters

<i>v</i>	対象の仮想マシン。
<i>nparams</i>	引数の個数。

Return values

<i>SQ_TAILCALL_FLAG</i>	成功した場合の末尾呼び出しフラグ。
<i>SQ_ERROR</i>	失敗時。

Definition at line 2580 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02581 {
02582     SQObjectPtr &res = v->GetUp(-(nparams + 1));

```

```

02583     if (sq_type(res) != OT_CLOSURE) {
02584         return sq_throwerror(v, _SC("only closure can be tail called"));
02585     }
02586     SQClosure *clo = _closure(res);
02587     if (clo->_function->_bgenerator)
02588     {
02589         return sq_throwerror(v, _SC("generators cannot be tail called"));
02590     }
02591
02592     SQInteger stackbase = (v->_top - nparams) - v->_stackbase;
02593     if (!v->TailCall(clo, stackbase, nparams)) {
02594         return SQ_ERROR;
02595     }
02596     return SQ_TAILCALL_FLAG;
02597 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.142 sq_throwerror()

```

SQRESULT sq_throwerror (
    HSQUIRRELVM v,
    const SQChar * err )

```

エラーメッセージを設定してエラーを発生させる。

指定された文字列 `err` を VM のラストエラーとして設定し、`SQ_ERROR` を返す。この関数はスクリプト実行中のエラー通知に使用する。

Parameters

<code>v</code>	対象の仮想マシン。
<code>err</code>	設定するエラーメッセージ。

Return values

<code>SQ_ERROR</code>	常にエラーを返す。
-----------------------	-----------

Definition at line 2439 of file [squirrel3_squirrel_sqapiByOpenAl.cpp](#).

```

02440 {
02441     v->_lasterror=SQString::Create(_ss(v),err);
02442     return SQ_ERROR;
02443 }

```

Referenced by [_getmemberbyhandle\(\)](#), [sq_arrayinsert\(\)](#), [sq_arraypop\(\)](#), [sq_arrayremove\(\)](#), [sq_arrayresize\(\)](#), [sq_aux_invalidtype\(\)](#), [sq_bindenv\(\)](#), [sq_clear\(\)](#), [sq_compile\(\)](#), [sq_deleteslot\(\)](#), [sq_getattributes\(\)](#), [sq_getcallee\(\)](#),

[sq_getclosureinfo\(\)](#), [sq_getclosurename\(\)](#), [sq_getcluseroot\(\)](#), [sq_getdefaultdelegate\(\)](#), [sq_getdelegate\(\)](#), [sq_getinstanceup\(\)](#), [sq_getmemberhandle\(\)](#), [sq_getweakrefval\(\)](#), [sq_instanceof\(\)](#), [sq_newclass\(\)](#), [sq_newmember\(\)](#), [sq_newslot\(\)](#), [sq_next\(\)](#), [sq_rawget\(\)](#), [sq_rawnewmember\(\)](#), [sq_rawset\(\)](#), [sq_readclosure\(\)](#), [sq_reservestack\(\)](#), [sq_resume\(\)](#), [sq_resurrectunreachable\(\)](#), [sq_setattributes\(\)](#), [sq_setclassudsize\(\)](#), [sq_setcluseroot\(\)](#), [sq_setconsttable\(\)](#), [sq_setdelegate\(\)](#), [sq_setfreevariable\(\)](#), [sq_setinstanceup\(\)](#), [sq_setnativeclosurename\(\)](#), [sq_setparamscheck\(\)](#), [sq_setroottable\(\)](#), [sq_settypetag\(\)](#), [sq_tailcall\(\)](#), [sq_wakeupvm\(\)](#), and [sq_writeclosure\(\)](#).

Here is the caller graph for this function:



4.1.2.143 sq_throwobject()

```
SQRESULT sq_throwobject (
    HSQUIRRELVM v )
```

スタックトップのオブジェクトをエラーとして投げる。

スタックトップにあるオブジェクトをラストエラーとして設定し、SQ_ERROR を返す。カスタムオブジェクトをエラーとして返したい場合に使用する。

Parameters

<i>v</i>	対象の仮想マシン。
----------	-----------

Return values

<i>SQ_ERROR</i>	常にエラーを返す。
-----------------	-----------

Definition at line 2455 of file squirrel3_squirrel_sqapiByOpenAl.cpp.

```
02456 {
02457     v->_lasterror = v->GetUp(-1);
02458     v->Pop();
02459     return SQ_ERROR;
02460 }
```

4.1.2.144 sq_tobool()

```
void sq_tobool (
    HSQUIRRELVM v,
    SQInteger idx,
    SQBool * b )
```

指定インデックスのオブジェクトを真偽値として取得する。

スタックのインデックス *idx* にあるオブジェクトの真偽値を評価し、結果を *b* に格納する。

Parameters

	<i>v</i>	対象の仮想マシン。
	<i>idx</i>	評価対象のスタックインデックス。
out	<i>b</i>	評価された真偽値の出力先。

Definition at line 1488 of file squirrel3_squirrel_sqapiByOpenAl.cpp.

```
01489 {
01490     SQObjectPtr &o = stack_get(v, idx);
01491     *b = SQVM::IsFalse(o)?SQFalse:SQTrue;
01492 }
```

4.1.2.145 sq_tostring()

```
SQRESULT sq_tostring (
    HSQUIRRELVM v,
    SQInteger idx )
```

スタックの指定インデックスにあるオブジェクトを文字列に変換してプッシュする。

指定されたインデックス `idx` のオブジェクトを文字列形式に変換し、成功すればその文字列オブジェクトをスタックにプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	スタック上の対象オブジェクトのインデックス。

Return values

<code>SQ_OK</code>	成功した場合。
<code>SQ_ERROR</code>	変換に失敗した場合。

Definition at line 1466 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01467 {
01468     SQObjectPtr &o = stack_get(v, idx);
01469     SQObjectPtr res;
01470     if(!v->ToString(o,res)) {
01471         return SQ_ERROR;
01472     }
01473     v->Push(res);
01474     return SQ_OK;
01475 }
```

4.1.2.146 sq_typeof()

```
SQRESULT sq_typeof (
    HSQUIRRELVm v,
    SQInteger idx )
```

指定されたスタックインデックスのオブジェクトの型名を取得する。

スタック上の指定インデックス `idx` に存在するオブジェクトの型を文字列として取得し、その型名をスタックのトップにプッシュする。

内部的には仮想マシンの `TypeOf` 関数を利用して型を判定し、文字列化している。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	型を取得したいオブジェクトのスタックインデックス。

Return values

<code>SQ_OK</code>	成功時。
<code>SQ_ERROR</code>	型取得に失敗した場合。

Definition at line 1443 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
01444 {
01445     SQObjectPtr &o = stack_get(v, idx);
01446     SQObjectPtr res;
01447     if(!v->TypeOf(o,res)) {
01448         return SQ_ERROR;
01449     }
```

```

01450     v->Push(res);
01451     return SQ_OK;
01452 }

```

4.1.2.147 sq_wakeupvm()

```

SQRESULT sq_wakeupvm (
    HSQUIRRELVm v,
    SQBool wakeupret,
    SQBool retval,
    SQBool raiseerror,
    SQBool throwerror )

```

サスペンド状態の仮想マシンを再開する。

サスペンド状態にある仮想マシンを再開する。戻り値を設定したり、例外を送出するかどうかなどをオプションで制御できる。再開時に対象がサスペンド状態でない場合はエラーとなる。

Parameters

<i>v</i>	対象の仮想マシン。
<i>wakeupret</i>	戻り値を設定する場合 true。
<i>retval</i>	戻り値をスタックにプッシュする場合 true。
<i>raiseerror</i>	エラー時に例外を送出する場合 true。
<i>throwerror</i>	VM 内で例外を投げる場合 true。

Return values

<i>SQ_OK</i>	成功。
<i>SQ_ERROR</i>	失敗。

Definition at line 2629 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```

02630 {
02631     SQObjectPtr ret;
02632     if(!v->_suspended)
02633         return sq_throwerror(v,_SC("cannot resume a vm that is not running any code"));
02634     SQInteger target = v->_suspended_target;
02635     if(wakeupret) {
02636         if(target != -1) {
02637             v->GetAt(v->_stackbase+v->_suspended_target)=v->GetUp(-1); //retval
02638         }
02639         v->Pop();
02640     } else if(target != -1) { v->GetAt(v->_stackbase+v->_suspended_target).Null(); }
02641     SQObjectPtr dummy;
02642     if(!v->Execute(dummy,-1,-1,ret,raiseerror,throwerror?SQVM::ET_RESUME_THROW_VM :
SQVM::ET_RESUME_VM) ) {
02643         return SQ_ERROR;
02644     }
02645     if(retval)
02646         v->Push(ret);
02647     return SQ_OK;
02648 }

```

References [sq_throwerror\(\)](#).

Here is the call graph for this function:



4.1.2.148 sq_weakref()

```
void sq_weakref (
    HSQUIRRELVM v,
    SQInteger idx )
```

弱参照を生成する。

スタックの `idx` にあるオブジェクトが参照カウント型の場合、その弱参照を生成してスタックにプッシュする。参照カウント型でない場合はそのままプッシュする。

Parameters

<code>v</code>	対象の仮想マシン。
<code>idx</code>	対象オブジェクトのスタックインデックス。

Definition at line 3181 of file [squirrel3_squirrel_sqapiByOpenAI.cpp](#).

```
03182 {
03183     SQObject &o=stack_get(v,idx);
03184     if (ISREFCOUNTED(sq_type(o))) {
03185         v->Push(_refcounted(o)->GetWeakRef(sq_type(o)));
03186         return;
03187     }
03188     v->Push(o);
03189 }
```

4.1.2.149 sq_writeclosure()

```
SQRESULT sq_writeclosure (
    HSQUIRRELVM v,
    SQWRITEFUNC w,
    SQUserPointer up )
```

クロージャをストリームに書き出す。

スタックトップにあるクロージャをバイトコードとして書き出す。自由変数がバインドされている場合はエラーを返す。書き出しには指定された書き込み関数 `w` とユーザーポインタ `up` が使用される。

Parameters

<code>v</code>	対象の仮想マシン。
<code>w</code>	書き込み関数。
<code>up</code>	ユーザーポインタ。

Return values

<code>SQ_OK</code>	成功。
<code>SQ_ERROR</code>	IO エラーまたは自由変数がバインドされている場合。

Definition at line 2722 of file `squirrel3_squirrel_sqapiByOpenAI.cpp`.

```

02723 {
02724     SQObjectPtr *o = NULL;
02725     _GETSAFE_OBJ(v, -1, OT_CLOSURE,o);
02726     unsigned short tag = SQ_BYTECODE_STREAM_TAG;
02727     if(_closure(*o)->_function->_noutervalues)
02728         return sq_throwerror(v,_SC("a closure with free variables bound cannot be serialized"));
02729     if(w(up,&tag,2) != 2)
02730         return sq_throwerror(v,_SC("io error"));
02731     if(!_closure(*o)->Save(v,up,w))
02732         return SQ_ERROR;
02733     return SQ_OK;
02734 }
```

References `_GETSAFE_OBJ`, and `sq_throwerror()`.

Here is the call graph for this function:



4.2 squirrel3_squirrel_sqapiByOpenAI.cpp

[Go to the documentation of this file.](#)

```

00001 //cpp
00002
00003 /*
00004     Written By OpenAI
00005 */
00006 #include "sqpheader.h"
00007 #include "sgvm.h"
00008 #include "sqstring.h"
00009 #include "sqtable.h"
00010 #include "sqarray.h"
00011 #include "sqfuncproto.h"
00012 #include "sqclosure.h"
00013 #include "squserdata.h"
00014 #include "sqcompiler.h"
00015 #include "sqfuncstate.h"
00016 #include "sqclass.h"
00017
00033 static bool sq_aux_gettypedarg(HSQUIRELVM v,SQInteger idx,SQObjectType type,SQObjectPtr **o)
00034 {
00035     *o = &stack_get(v,idx);
00036     if(sq_type(**o) != type){
00037         SQObjectPtr oval = v->PrintObjVal(**o);
00038         v->Raise_Error(_SC("wrong argument type, expected '%s' got
00039         '%.50s'",IdType2Name(type),_stringval(oval)));
00039         return false;
00040     }
00041     return true;
00042 }
00043
00059 #define _GETSAFE_OBJ(v,idx,type,o) { if(!sq_aux_gettypedarg(v,idx,type,&o)) return SQ_ERROR; }
00060
00073 #define sq_aux_paramscheck(v,count) \
00074 { \
```

```

00075     if(sq_gettop(v) < count){ v->Raise_Error(_SC("not enough params in the stack")); return SQ_ERROR;
00076 } \
00077 }
00090 SQInteger sq_aux_invalidtype(HSQUIRRELM v, SQObjectType type)
00091 {
00092     SQUnsignedInteger buf_size = 100 * sizeof(SQChar);
00093     scsprintf(_ss(v)->GetScratchPad(buf_size), buf_size, _SC("unexpected type %s"),
00094         IdType2Name(type));
00094     return sq_throwerror(v, _ss(v)->GetScratchPad(-1));
00095 }
00096
00109 HSQUIRRELM sq_open(SQInteger initialstacksize)
00110 {
00111     SQSharedState *ss;
00112     SQVM *v;
00113     sq_new(ss, SQSharedState);
00114     ss->Init();
00115     v = (SQVM *)SQ_MALLOC(sizeof(SQVM));
00116     new (v) SQVM(ss);
00117     ss->_root_vm = v;
00118     if(v->Init(NULL, initialstacksize)) {
00119         return v;
00120     } else {
00121         sq_delete(v, SQVM);
00122         return NULL;
00123     }
00124     return v;
00125 }
00126
00140 HSQUIRRELM sq_newthread(HSQUIRRELM friendvm, SQInteger initialstacksize)
00141 {
00142     SQSharedState *ss;
00143     SQVM *v;
00144     ss=_ss(friendvm);
00145
00146     v= (SQVM *)SQ_MALLOC(sizeof(SQVM));
00147     new (v) SQVM(ss);
00148
00149     if(v->Init(friendvm, initialstacksize)) {
00150         friendvm->Push(v);
00151         return v;
00152     } else {
00153         sq_delete(v, SQVM);
00154         return NULL;
00155     }
00156 }
00157
00174 SQInteger sq_getvmstate(HSQUIRRELM v)
00175 {
00176     if(v->_suspended)
00177         return SQ_VMSTATE_SUSPENDED;
00178     else {
00179         if(v->_callsstacksize != 0) return SQ_VMSTATE_RUNNING;
00180         else return SQ_VMSTATE_IDLE;
00181     }
00182 }
00183
00195 void sq_seterrorhandler(HSQUIRRELM v)
00196 {
00197     SQObject o = stack_get(v, -1);
00198     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00199         v->_errorhandler = o;
00200         v->Pop();
00201     }
00202 }
00203
00214 void sq_setnatividebughook(HSQUIRRELM v, SQDEBUGHOOK hook)
00215 {
00216     v->_debughook_native = hook;
00217     v->_debughook_closure.Null();
00218     v->_debughook = hook?true:false;
00219 }
00220
00233 void sq_setdebughook(HSQUIRRELM v)
00234 {
00235     SQObject o = stack_get(v, -1);
00236     if(sq_isclosure(o) || sq_isnativeclosure(o) || sq_isnull(o)) {
00237         v->_debughook_closure = o;
00238         v->_debughook_native = NULL;
00239         v->_debughook = !sq_isnull(o);
00240         v->Pop();
00241     }
00242 }
00243
00253 void sq_close(HSQUIRRELM v)
00254 {

```

```

00255     SQSharedState *ss = _ss(v);
00256     _thread(ss->_root_vm)->Finalize();
00257     sq_delete(ss, SQSharedState);
00258 }
00259
00269 SQInteger sq_getversion()
00270 {
00271     return SQUIRREL_VERSION_NUMBER;
00272 }
00273
00292 SQRESULT sq_compile(HSQIRRELVm v, SQLEXREADFUNC read, SQUserPointer p, const SQChar *sourcename, SQBool
raiseerror)
00293 {
00294     SQObjectPtr o;
00295 #ifndef NO_COMPILER
00296     if(Compile(v, read, p, sourcename, o, raiseerror?true:false, _ss(v)->_debuginfo)) {
00297         v->Push(SQClosure::Create(_ss(v), _funcproto(o),
00298         _table(v->_roottable)->GetWeakRef(OT_TABLE)));
00299     }
00300     return SQ_OK;
00301 }
00302 #else
00303     return SQ_ERROR;
00304 #endif
00305 }
00317 void sq_enableddebuginfo(HSQIRRELVm v, SQBool enable)
00318 {
00319     _ss(v)->_debuginfo = enable?true:false;
00320 }
00321
00334 void sq_notifyallexceptions(HSQIRRELVm v, SQBool enable)
00335 {
00336     _ss(v)->_notifyallexceptions = enable?true:false;
00337 }
00338
00351 void sq_addref(HSQIRRELVm v, HSQOBJECT *po)
00352 {
00353     if(!ISREFCOUNTED(sq_type(*po))) return;
00354 #ifndef NO_GARBAGE_COLLECTOR
00355     __AddRef(po->_type, po->_unVal);
00356 #else
00357     _ss(v)->_refs_table.AddRef(*po);
00358 #endif
00359 }
00360
00375 SQUnsignedInteger sq_getrefcount(HSQIRRELVm v, HSQOBJECT *po)
00376 {
00377     if(!ISREFCOUNTED(sq_type(*po))) return 0;
00378 #ifndef NO_GARBAGE_COLLECTOR
00379     return po->_unVal.pRefCounted->uiRef;
00380 #else
00381     return _ss(v)->_refs_table.GetRefCount(*po);
00382 #endif
00383 }
00384
00399 SQBool sq_release(HSQIRRELVm v, HSQOBJECT *po)
00400 {
00401     if(!ISREFCOUNTED(sq_type(*po))) return SQTrue;
00402 #ifndef NO_GARBAGE_COLLECTOR
00403     bool ret = (po->_unVal.pRefCounted->uiRef <= 1) ? SQTrue : SQFalse;
00404     __Release(po->_type, po->_unVal);
00405     return ret; //the ret val doesn't work (and cannot be fixed)
00406 #else
00407     return _ss(v)->_refs_table.Release(*po);
00408 #endif
00409 }
00410
00423 SQUnsignedInteger sq_getvmrefcount(HSQIRRELVm v, const HSQOBJECT *po)
00424 {
00425     if (!ISREFCOUNTED(sq_type(*po))) return 0;
00426     return po->_unVal.pRefCounted->uiRef;
00427 }
00428
00440 const SQChar *sq_objtosting(const HSQOBJECT *o)
00441 {
00442     if(sq_type(*o) == OT_STRING) {
00443         return _stringval(*o);
00444     }
00445     return NULL;
00446 }
00447
00459 SQInteger sq_objtointeger(const HSQOBJECT *o)
00460 {
00461     if(sq_isnumeric(*o)) {
00462         return tointeger(*o);
00463     }

```

```

00464     return 0;
00465 }
00466
00478 SQFloat sq_objtfloat(const HSQOBJECT *o)
00479 {
00480     if(sq_isnumeric(*o)) {
00481         return tofloat(*o);
00482     }
00483     return 0;
00484 }
00485
00497 SQBool sq_objtobool(const HSQOBJECT *o)
00498 {
00499     if(sq_isbool(*o)) {
00500         return _integer(*o);
00501     }
00502     return SQFalse;
00503 }
00504
00516 SQUserPointer sq_objtouserpointer(const HSQOBJECT *o)
00517 {
00518     if(sq_isuserpointer(*o)) {
00519         return _userpointer(*o);
00520     }
00521     return 0;
00522 }
00523
00532 void sq_pushnull(HSQIRRELM v)
00533 {
00534     v->PushNull();
00535 }
00536
00548 void sq_pushstring(HSQIRRELM v, const SQChar *s, SQInteger len)
00549 {
00550     if(s)
00551         v->Push(SQObjectPtr(SQString::Create(_ss(v), s, len)));
00552     else v->PushNull();
00553 }
00554
00564 void sq_pushinteger(HSQIRRELM v, SQInteger n)
00565 {
00566     v->Push(n);
00567 }
00568
00578 void sq_pushbool(HSQIRRELM v, SQBool b)
00579 {
00580     v->Push(b?true:false);
00581 }
00582
00592 void sq_pushfloat(HSQIRRELM v, SQFloat n)
00593 {
00594     v->Push(n);
00595 }
00596
00606 void sq_pushuserpointer(HSQIRRELM v, SQUserPointer p)
00607 {
00608     v->Push(p);
00609 }
00610
00621 void sq_pushthread(HSQIRRELM v, HSQIRRELM thread)
00622 {
00623     v->Push(thread);
00624 }
00625
00638 SQUserPointer sq_newuserdata(HSQIRRELM v, SQUnsignedInteger size)
00639 {
00640     SQUserData *ud = SQUserData::Create(_ss(v), size + SQ_ALIGNMENT);
00641     v->Push(ud);
00642     return (SQUserPointer)sq_aligning(ud + 1);
00643 }
00644
00653 void sq_newtable(HSQIRRELM v)
00654 {
00655     v->Push(SQTable::Create(_ss(v), 0));
00656 }
00657
00668 void sq_newtableex(HSQIRRELM v, SQInteger initialcapacity)
00669 {
00670     v->Push(SQTable::Create(_ss(v), initialcapacity));
00671 }
00672
00682 void sq_newarray(HSQIRRELM v, SQInteger size)
00683 {
00684     v->Push(SQArray::Create(_ss(v), size));
00685 }
00686
00704 SQRESULT sq_newclass(HSQIRRELM v, SQBool hasbase)

```



```

00705 {
00706     SQClass *baseclass = NULL;
00707     if(hasbase) {
00708         SQObjectPtr &base = stack_get(v,-1);
00709         if(sq_type(base) != OT_CLASS)
00710             return sq_throwerror(v,_SC("invalid base type"));
00711         baseclass = _class(base);
00712     }
00713     SQClass *newclass = SQClass::Create(_ss(v), baseclass);
00714     if(baseclass) v->Pop();
00715     v->Push(newclass);
00716     return SQ_OK;
00717 }
00718
00731 SQBool sq_instanceof(HSQIRRELV v)
00732 {
00733     SQObjectPtr &inst = stack_get(v,-1);
00734     SQObjectPtr &cl = stack_get(v,-2);
00735     if(sq_type(inst) != OT_INSTANCE || sq_type(cl) != OT_CLASS)
00736         return sq_throwerror(v,_SC("invalid param type"));
00737     return _instance(inst)->InstanceOf(_class(cl)) ? SQTrue : SQFalse;
00738 }
00739
00753 SQRESULT sq_arrayappend(HSQIRRELV v,SQInteger idx)
00754 {
00755     sq_aux_paramscheck(v,2);
00756     SQObjectPtr *arr;
00757     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00758     _array(*arr)->Append(v->GetUp(-1));
00759     v->Pop();
00760     return SQ_OK;
00761 }
00762
00777 SQRESULT sq_arraypop(HSQIRRELV v,SQInteger idx,SQBool pushval)
00778 {
00779     sq_aux_paramscheck(v, 1);
00780     SQObjectPtr *arr;
00781     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00782     if(_array(*arr)->Size() > 0) {
00783         if(pushval != 0){ v->Push(_array(*arr)->Top()); }
00784         _array(*arr)->Pop();
00785         return SQ_OK;
00786     }
00787     return sq_throwerror(v, _SC("empty array"));
00788 }
00789
00805 SQRESULT sq_arrayresize(HSQIRRELV v,SQInteger idx,SQInteger newsize)
00806 {
00807     sq_aux_paramscheck(v,1);
00808     SQObjectPtr *arr;
00809     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00810     if(newsize >= 0) {
00811         _array(*arr)->Resize(newsize);
00812         return SQ_OK;
00813     }
00814     return sq_throwerror(v,_SC("negative size"));
00815 }
00816
00830 SQRESULT sq_arrayreverse(HSQIRRELV v,SQInteger idx)
00831 {
00832     sq_aux_paramscheck(v, 1);
00833     SQObjectPtr *o;
00834     _GETSAFE_OBJ(v, idx, OT_ARRAY,o);
00835     SQArray *arr = _array(*o);
00836     if(arr->Size() > 0) {
00837         SQObjectPtr t;
00838         SQInteger size = arr->Size();
00839         SQInteger n = size » 1; size -= 1;
00840         for(SQInteger i = 0; i < n; i++) {
00841             t = arr->_values[i];
00842             arr->_values[i] = arr->_values[size-i];
00843             arr->_values[size-i] = t;
00844         }
00845         return SQ_OK;
00846     }
00847     return SQ_OK;
00848 }
00849
00863 SQRESULT sq_arrayremove(HSQIRRELV v,SQInteger idx,SQInteger itemidx)
00864 {
00865     sq_aux_paramscheck(v, 1);
00866     SQObjectPtr *arr;
00867     _GETSAFE_OBJ(v, idx, OT_ARRAY,arr);
00868     return _array(*arr)->Remove(itemidx) ? SQ_OK : sq_throwerror(v,_SC("index out of range"));
00869 }
00870
00885 SQRESULT sq_arrayinsert(HSQIRRELV v,SQInteger idx,SQInteger destpos)

```

```

00886 {
00887     sq_aux_paramscheck(v, 1);
00888     SQObjectPtr *arr;
00889     _GETSAFE_OBJ(v, idx, OT_ARRAY, arr);
00890     SQRESULT ret = _array(*arr)->Insert(destpos, v->GetUp(-1)) ? SQ_OK : sq_throwerror(v, _SC("index
out of range"));
00891     v->Pop();
00892     return ret;
00893 }
00894
00908 void sq_newclosure(HSQUIRRELV v, SQFUNCTION func, SQUnsignedInteger nfreevars)
00909 {
00910     SQNativeClosure *nc = SQNativeClosure::Create(_ss(v), func, nfreevars);
00911     nc->_nparamscheck = 0;
00912     for(SQUnsignedInteger i = 0; i < nfreevars; i++) {
00913         nc->_outervalues[i] = v->Top();
00914         v->Pop();
00915     }
00916     v->Push(SQObjectPtr(nc));
00917 }
00918
00936 SQRESULT sq_getclosureinfo(HSQUIRRELV v, SQInteger idx, SQInteger *nparams, SQInteger *nfreevars)
00937 {
00938     SQObject o = stack_get(v, idx);
00939     if(sq_type(o) == OT_CLOSURE) {
00940         SQClosure *c = _closure(o);
00941         SQFunctionProto *proto = c->_function;
00942         *nparams = proto->_nparameters;
00943         *nfreevars = proto->_noutervalues;
00944         return SQ_OK;
00945     }
00946     else if(sq_type(o) == OT_NATIVECLOSURE)
00947     {
00948         SQNativeClosure *c = _nativeclosure(o);
00949         *nparams = c->_nparamscheck;
00950         *nfreevars = (SQInteger)c->_noutervalues;
00951         return SQ_OK;
00952     }
00953     return sq_throwerror(v, _SC("the object is not a closure"));
00954 }
00955
00972 SQRESULT sq_setnativeclosurename(HSQUIRRELV v, SQInteger idx, const SQChar *name)
00973 {
00974     SQObject o = stack_get(v, idx);
00975     if(sq_isnativeclosure(o)) {
00976         SQNativeClosure *nc = _nativeclosure(o);
00977         nc->_name = SQString::Create(_ss(v), name);
00978         return SQ_OK;
00979     }
00980     return sq_throwerror(v, _SC("the object is not a nativeclosure"));
00981 }
00982
01002 SQRESULT sq_setparamscheck(HSQUIRRELV v, SQInteger nparamscheck, const SQChar *typemask)
01003 {
01004     SQObject o = stack_get(v, -1);
01005     if(!sq_isnativeclosure(o))
01006         return sq_throwerror(v, _SC("native closure expected"));
01007     SQNativeClosure *nc = _nativeclosure(o);
01008     nc->_nparamscheck = nparamscheck;
01009     if(typemask) {
01010         SQIntVec res;
01011         if(!CompileTypemask(res, typemask))
01012             return sq_throwerror(v, _SC("invalid typemask"));
01013         nc->_typecheck.copy(res);
01014     }
01015     else {
01016         nc->_typecheck.resize(0);
01017     }
01018     if(nparamscheck == SQ_MATCHTYPEMASKSTRING) {
01019         nc->_nparamscheck = nc->_typecheck.size();
01020     }
01021     return SQ_OK;
01022 }
01023
01038 SQRESULT sq_bindenv(HSQUIRRELV v, SQInteger idx)
01039 {
01040     SQObjectPtr &o = stack_get(v, idx);
01041     if(!sq_isnativeclosure(o) &&
01042         !sq_isclosure(o))
01043         return sq_throwerror(v, _SC("the target is not a closure"));
01044     SQObjectPtr &env = stack_get(v, -1);
01045     if(!sq_istable(env) &&
01046         !sq_isarray(env) &&
01047         !sq_isclass(env) &&
01048         !sq_isinstance(env))
01049         return sq_throwerror(v, _SC("invalid environment"));
01050     SQWeakRef *w = _refcounted(env)->GetWeakRef(sq_type(env));

```

```

01051     SQObjectPtr ret;
01052     if(sq_isclosure(o)) {
01053         SQClosure *c = _closure(o)->Clone();
01054         __ObjRelease(c->_env);
01055         c->_env = w;
01056         __ObjAddRef(c->_env);
01057         if(_closure(o)->_base) {
01058             c->_base = _closure(o)->_base;
01059             __ObjAddRef(c->_base);
01060         }
01061         ret = c;
01062     }
01063     else { //then must be a native closure
01064         SQNativeClosure *c = _nativeclosure(o)->Clone();
01065         __ObjRelease(c->_env);
01066         c->_env = w;
01067         __ObjAddRef(c->_env);
01068         ret = c;
01069     }
01070     v->Pop();
01071     v->Push(ret);
01072     return SQ_OK;
01073 }
01074
01088 SQRRESULT sq_getclosurename(HSQIRRELVM v,SQInteger idx)
01089 {
01090     SQObjectPtr &o = stack_get(v,idx);
01091     if(!sq_isnativeclosure(o) &&
01092        !sq_isclosure(o))
01093         return sq_throwerror(v,_SC("the target is not a closure"));
01094     if(sq_isnativeclosure(o))
01095     {
01096         v->Push(_nativeclosure(o)->_name);
01097     }
01098     else { //closure
01099         v->Push(_closure(o)->_function->_name);
01100     }
01101     return SQ_OK;
01102 }
01103
01119 SQRRESULT sq_setclosureroot(HSQIRRELVM v,SQInteger idx)
01120 {
01121     SQObjectPtr &c = stack_get(v,idx);
01122     SQObject o = stack_get(v, -1);
01123     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
01124     if(sq_istable(o)) {
01125         _closure(c)->SetRoot(_table(o)->GetWeakRef(OT_TABLE));
01126         v->Pop();
01127         return SQ_OK;
01128     }
01129     return sq_throwerror(v, _SC("invalid type"));
01130 }
01131
01145 SQRRESULT sq_getclosureroot(HSQIRRELVM v,SQInteger idx)
01146 {
01147     SQObjectPtr &c = stack_get(v,idx);
01148     if(!sq_isclosure(c)) return sq_throwerror(v, _SC("closure expected"));
01149     v->Push(_closure(c)->_root->_obj);
01150     return SQ_OK;
01151 }
01152
01167 SQRRESULT sq_clear(HSQIRRELVM v,SQInteger idx)
01168 {
01169     SQObject &o=stack_get(v,idx);
01170     switch(sq_type(o)) {
01171         case OT_TABLE: _table(o)->Clear(); break;
01172         case OT_ARRAY: _array(o)->Resize(0); break;
01173         default:
01174             return sq_throwerror(v, _SC("clear only works on table and array"));
01175         break;
01176     }
01177 }
01178     return SQ_OK;
01179 }
01180
01190 void sq_pushroottable(HSQIRRELVM v)
01191 {
01192     v->Push(v->_roottable);
01193 }
01194
01204 void sq_pushregistrytable(HSQIRRELVM v)
01205 {
01206     v->Push(_ss(v)->_registry);
01207 }
01208
01218 void sq_pushconsttable(HSQIRRELVM v)
01219 {

```

```

01220     v->Push(_ss(v)->_consts);
01221 }
01222
01237 SQRESULT sq_setroottable(HSQIRRELV v)
01238 {
01239     SQObject o = stack_get(v, -1);
01240     if(sq_istable(o) || sq_isnull(o)) {
01241         v->_roottable = o;
01242         v->Pop();
01243         return SQ_OK;
01244     }
01245     return sq_throwerror(v, _SC("invalid type"));
01246 }
01247
01262 SQRESULT sq_setconsttable(HSQIRRELV v)
01263 {
01264     SQObject o = stack_get(v, -1);
01265     if(sq_istable(o)) {
01266         _ss(v)->_consts = o;
01267         v->Pop();
01268         return SQ_OK;
01269     }
01270     return sq_throwerror(v, _SC("invalid type, expected table"));
01271 }
01272
01283 void sq_setforeignptr(HSQIRRELV v, SQUserPointer p)
01284 {
01285     v->_foreignptr = p;
01286 }
01287
01299 SQUserPointer sq_getforeignptr(HSQIRRELV v)
01300 {
01301     return v->_foreignptr;
01302 }
01303
01314 void sq_setsharedforeignptr(HSQIRRELV v, SQUserPointer p)
01315 {
01316     _ss(v)->_foreignptr = p;
01317 }
01318
01330 SQUserPointer sq_getsharedforeignptr(HSQIRRELV v)
01331 {
01332     return _ss(v)->_foreignptr;
01333 }
01334
01345 void sq_setvmreleasehook(HSQIRRELV v, SQRELEASEHOOK hook)
01346 {
01347     v->_releasehook = hook;
01348 }
01349
01361 SQRELEASEHOOK sq_getvmreleasehook(HSQIRRELV v)
01362 {
01363     return v->_releasehook;
01364 }
01365
01376 void sq_setsharedreleasehook(HSQIRRELV v, SQRELEASEHOOK hook)
01377 {
01378     _ss(v)->_releasehook = hook;
01379 }
01380
01392 SQRELEASEHOOK sq_getsharedreleasehook(HSQIRRELV v)
01393 {
01394     return _ss(v)->_releasehook;
01395 }
01396
01407 void sq_push(HSQIRRELV v, SQInteger idx)
01408 {
01409     v->Push(stack_get(v, idx));
01410 }
01411
01424 SQObjectType sq_gettype(HSQIRRELV v, SQInteger idx)
01425 {
01426     return sq_type(stack_get(v, idx));
01427 }
01428
01443 SQRESULT sq_typeof(HSQIRRELV v, SQInteger idx)
01444 {
01445     SQObjectPtr &o = stack_get(v, idx);
01446     SQObjectPtr res;
01447     if(!v->TypeOf(o, res)) {
01448         return SQ_ERROR;
01449     }
01450     v->Push(res);
01451     return SQ_OK;
01452 }
01453
01466 SQRESULT sq_tostring(HSQIRRELV v, SQInteger idx)

```

```

01467 {
01468     SQObjectPtr &o = stack_get(v, idx);
01469     SQObjectPtr res;
01470     if(!v->ToString(o,res)) {
01471         return SQ_ERROR;
01472     }
01473     v->Push(res);
01474     return SQ_OK;
01475 }
01476
01488 void sq_tobool(HSQIRRELVM v, SQInteger idx, SQBool *b)
01489 {
01490     SQObjectPtr &o = stack_get(v, idx);
01491     *b = SQVM::IsFalse(o)?SQFalse:SQTrue;
01492 }
01493
01507 SQRRESULT sq_getinteger(HSQIRRELVM v,SQInteger idx,SQInteger *i)
01508 {
01509     SQObjectPtr &o = stack_get(v, idx);
01510     if(sq_isnumeric(o)) {
01511         *i = tointeger(o);
01512         return SQ_OK;
01513     }
01514     if(sq_isbool(o)) {
01515         *i = SQVM::IsFalse(o)?SQFalse:SQTrue;
01516         return SQ_OK;
01517     }
01518     return SQ_ERROR;
01519 }
01520
01534 SQRRESULT sq_getfloat(HSQIRRELVM v,SQInteger idx,SQFloat *f)
01535 {
01536     SQObjectPtr &o = stack_get(v, idx);
01537     if(sq_isnumeric(o)) {
01538         *f = tofloat(o);
01539         return SQ_OK;
01540     }
01541     return SQ_ERROR;
01542 }
01543
01557 SQRRESULT sq_getbool(HSQIRRELVM v,SQInteger idx,SQBool *b)
01558 {
01559     SQObjectPtr &o = stack_get(v, idx);
01560     if(sq_isbool(o)) {
01561         *b = _integer(o);
01562         return SQ_OK;
01563     }
01564     return SQ_ERROR;
01565 }
01566
01580 SQRRESULT sq_getstringandsize(HSQIRRELVM v,SQInteger idx,const SQChar **c,SQInteger *size)
01581 {
01582     SQObjectPtr *o = NULL;
01583     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01584     *c = _stringval(*o);
01585     *size = _string(*o)->_len;
01586     return SQ_OK;
01587 }
01588
01602 SQRRESULT sq_getstring(HSQIRRELVM v,SQInteger idx,const SQChar **c)
01603 {
01604     SQObjectPtr *o = NULL;
01605     _GETSAFE_OBJ(v, idx, OT_STRING,o);
01606     *c = _stringval(*o);
01607     return SQ_OK;
01608 }
01609
01623 SQRRESULT sq_getthread(HSQIRRELVM v,SQInteger idx,HSQIRRELVM *thread)
01624 {
01625     SQObjectPtr *o = NULL;
01626     _GETSAFE_OBJ(v, idx, OT_THREAD,o);
01627     *thread = _thread(*o);
01628     return SQ_OK;
01629 }
01630
01643 SQRRESULT sq_clone(HSQIRRELVM v,SQInteger idx)
01644 {
01645     SQObjectPtr &o = stack_get(v,idx);
01646     v->PushNull();
01647     if(!v->Clone(o, stack_get(v, -1))){
01648         v->Pop();
01649         return SQ_ERROR;
01650     }
01651     return SQ_OK;
01652 }
01653
01665 SQInteger sq_getsize(HSQIRRELVM v, SQInteger idx)

```

```

01666 {
01667     SQObjectPtr &o = stack_get(v, idx);
01668     SQObjectType type = sq_type(o);
01669     switch(type) {
01670     case OT_STRING:      return _string(o)->_len;
01671     case OT_TABLE:      return _table(o)->CountUsed();
01672     case OT_ARRAY:      return _array(o)->Size();
01673     case OT_USERDATA:   return _userdata(o)->_size;
01674     case OT_INSTANCE:   return _instance(o)->_class->_udsize;
01675     case OT_CLASS:      return _class(o)->_udsize;
01676     default:
01677         return sq_aux_invalidtype(v, type);
01678     }
01679 }
01680
01692 SQHash sq_gethash(HSQIRRELV v, SQInteger idx)
01693 {
01694     SQObjectPtr &o = stack_get(v, idx);
01695     return HashObj(o);
01696 }
01697
01711 SQRESULT sq_getuserdata(HSQIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer *typetag)
01712 {
01713     SQObjectPtr *o = NULL;
01714     _GETSAFE_OBJ(v, idx, OT_USERDATA, o);
01715     (*p) = _userdataval(*o);
01716     if(typetag) *typetag = _userdata(*o)->_typetag;
01717     return SQ_OK;
01718 }
01719
01733 SQRESULT sq_settypetag(HSQIRRELV v, SQInteger idx, SQUserPointer typetag)
01734 {
01735     SQObjectPtr &o = stack_get(v, idx);
01736     switch(sq_type(o)) {
01737     case OT_USERDATA:    _userdata(o)->_typetag = typetag; break;
01738     case OT_CLASS:      _class(o)->_typetag = typetag; break;
01739     default:            return sq_throwerror(v, _SC("invalid object type"));
01740     }
01741     return SQ_OK;
01742 }
01743
01756 SQRESULT sq_getobjtypetag(const HSQOBJECT *o, SQUserPointer * typetag)
01757 {
01758     switch(sq_type(*o)) {
01759     case OT_INSTANCE:    *typetag = _instance(*o)->_class->_typetag; break;
01760     case OT_USERDATA:    *typetag = _userdata(*o)->_typetag; break;
01761     case OT_CLASS:      *typetag = _class(*o)->_typetag; break;
01762     default:            return SQ_ERROR;
01763     }
01764     return SQ_OK;
01765 }
01766
01780 SQRESULT sq_gettypetag(HSQIRRELV v, SQInteger idx, SQUserPointer *typetag)
01781 {
01782     SQObjectPtr &o = stack_get(v, idx);
01783     if (SQ_FAILED(sq_getobjtypetag(&o, typetag)))
01784         return SQ_ERROR; // this is not an error it should be a bool but would break backward
compatibility
01785     return SQ_OK;
01786 }
01787
01800 SQRESULT sq_getuserpointer(HSQIRRELV v, SQInteger idx, SQUserPointer *p)
01801 {
01802     SQObjectPtr *o = NULL;
01803     _GETSAFE_OBJ(v, idx, OT_USERPOINTER, o);
01804     (*p) = _userpointer(*o);
01805     return SQ_OK;
01806 }
01807
01822 SQRESULT sq_setinstanceup(HSQIRRELV v, SQInteger idx, SQUserPointer p)
01823 {
01824     SQObjectPtr &o = stack_get(v, idx);
01825     if(sq_type(o) != OT_INSTANCE) return sq_throwerror(v, _SC("the object is not a class instance"));
01826     _instance(o)->_userpointer = p;
01827     return SQ_OK;
01828 }
01829
01844 SQRESULT sq_setclassudsize(HSQIRRELV v, SQInteger idx, SQInteger udsize)
01845 {
01846     SQObjectPtr &o = stack_get(v, idx);
01847     if(sq_type(o) != OT_CLASS) return sq_throwerror(v, _SC("the object is not a class"));
01848     if(_class(o)->_locked) return sq_throwerror(v, _SC("the class is locked"));
01849     _class(o)->_udsize = udsize;
01850     return SQ_OK;
01851 }
01852
01870 SQRESULT sq_getinstanceup(HSQIRRELV v, SQInteger idx, SQUserPointer *p, SQUserPointer typetag,

```

```

        SQBool throwerror)
01871 {
01872     SQObjectPtr &o = stack_get(v, idx);
01873     if (sq_type(o) != OT_INSTANCE) return throwerror ? sq_throwerror(v, _SC("the object is not a class
instance")) : SQ_ERROR;
01874     (*p) = _instance(o)->_userpointer;
01875     if (typetag != 0) {
01876         SQClass *cl = _instance(o)->_class;
01877         do {
01878             if (cl->_typetag == typetag)
01879                 return SQ_OK;
01880             cl = cl->_base;
01881         } while (cl != NULL);
01882         return throwerror ? sq_throwerror(v, _SC("invalid type tag")) : SQ_ERROR;
01883     }
01884     return SQ_OK;
01885 }
01886
01897 SQInteger sq_gettop(HSQIRRELV v)
01898 {
01899     return (v->_top) - v->_stackbase;
01900 }
01901
01912 void sq_settop(HSQIRRELV v, SQInteger newtop)
01913 {
01914     SQInteger top = sq_gettop(v);
01915     if (top > newtop)
01916         sq_pop(v, top - newtop);
01917     else
01918         while (top++ < newtop) sq_pushnull(v);
01919 }
01920
01931 void sq_pop(HSQIRRELV v, SQInteger nelemstopop)
01932 {
01933     assert(v->_top >= nelemstopop);
01934     v->Pop(nelemstopop);
01935 }
01936
01946 void sq_poptop(HSQIRRELV v)
01947 {
01948     assert(v->_top >= 1);
01949     v->Pop();
01950 }
01951
01962 void sq_remove(HSQIRRELV v, SQInteger idx)
01963 {
01964     v->Remove(idx);
01965 }
01966
01976 SQInteger sq_cmp(HSQIRRELV v)
01977 {
01978     SQInteger res;
01979     v->ObjCmp(stack_get(v, -1), stack_get(v, -2), res);
01980     return res;
01981 }
01982
01997 SQRESULT sq_newslot(HSQIRRELV v, SQInteger idx, SQBool bstatic)
01998 {
01999     sq_aux_paramscheck(v, 3);
02000     SQObjectPtr &self = stack_get(v, idx);
02001     if (sq_type(self) == OT_TABLE || sq_type(self) == OT_CLASS) {
02002         SQObjectPtr &key = v->GetUp(-2);
02003         if (sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
02004         v->NewSlot(self, key, v->GetUp(-1), bstatic?true:false);
02005         v->Pop(2);
02006     }
02007     return SQ_OK;
02008 }
02009
02023 SQRESULT sq_deleteslot(HSQIRRELV v, SQInteger idx, SQBool pushval)
02024 {
02025     sq_aux_paramscheck(v, 2);
02026     SQObjectPtr *self;
02027     _GETSAFE_OBJ(v, idx, OT_TABLE, self);
02028     SQObjectPtr &key = v->GetUp(-1);
02029     if (sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null is not a valid key"));
02030     SQObjectPtr res;
02031     if (!v->DeleteSlot(*self, key, res)) {
02032         v->Pop();
02033         return SQ_ERROR;
02034     }
02035     if (pushval) v->GetUp(-1) = res;
02036     else v->Pop();
02037     return SQ_OK;
02038 }
02039
02053 SQRESULT sq_set(HSQIRRELV v, SQInteger idx)

```

```

02054 {
02055     SQObjectPtr &self = stack_get(v, idx);
02056     if(v->Set(self, v->GetUp(-2), v->GetUp(-1), DONT_FALL_BACK)) {
02057         v->Pop(2);
02058         return SQ_OK;
02059     }
02060     return SQ_ERROR;
02061 }
02062
02075 SQRRESULT sq_rawset(HSQIRRELVM v, SQInteger idx)
02076 {
02077     SQObjectPtr &self = stack_get(v, idx);
02078     SQObjectPtr &key = v->GetUp(-2);
02079     if(sq_type(key) == OT_NULL) {
02080         v->Pop(2);
02081         return sq_throwerror(v, _SC("null key"));
02082     }
02083     switch(sq_type(self)) {
02084     case OT_TABLE:
02085         _table(self)->NewSlot(key, v->GetUp(-1));
02086         v->Pop(2);
02087         return SQ_OK;
02088     break;
02089     case OT_CLASS:
02090         _class(self)->NewSlot(_ss(v), key, v->GetUp(-1), false);
02091         v->Pop(2);
02092         return SQ_OK;
02093     break;
02094     case OT_INSTANCE:
02095         if(_instance(self)->Set(key, v->GetUp(-1))) {
02096             v->Pop(2);
02097             return SQ_OK;
02098         }
02099     break;
02100     case OT_ARRAY:
02101         if(v->Set(self, key, v->GetUp(-1), false)) {
02102             v->Pop(2);
02103             return SQ_OK;
02104         }
02105     break;
02106     default:
02107         v->Pop(2);
02108         return sq_throwerror(v, _SC("rawset works only on array/table/class and instance"));
02109     }
02110     v->Raise_IdxError(v->GetUp(-2)); return SQ_ERROR;
02111 }
02112
02127 SQRRESULT sq_newmember(HSQIRRELVM v, SQInteger idx, SQBool bstatic)
02128 {
02129     SQObjectPtr &self = stack_get(v, idx);
02130     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
02131     SQObjectPtr &key = v->GetUp(-3);
02132     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
02133     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, false)) {
02134         v->Pop(3);
02135         return SQ_ERROR;
02136     }
02137     v->Pop(3);
02138     return SQ_OK;
02139 }
02140
02156 SQRRESULT sq_rawnewmember(HSQIRRELVM v, SQInteger idx, SQBool bstatic)
02157 {
02158     SQObjectPtr &self = stack_get(v, idx);
02159     if(sq_type(self) != OT_CLASS) return sq_throwerror(v, _SC("new member only works with classes"));
02160     SQObjectPtr &key = v->GetUp(-3);
02161     if(sq_type(key) == OT_NULL) return sq_throwerror(v, _SC("null key"));
02162     if(!v->NewSlotA(self, key, v->GetUp(-2), v->GetUp(-1), bstatic?true:false, true)) {
02163         v->Pop(3);
02164         return SQ_ERROR;
02165     }
02166     v->Pop(3);
02167     return SQ_OK;
02168 }
02169
02183 SQRRESULT sq_setdelegate(HSQIRRELVM v, SQInteger idx)
02184 {
02185     SQObjectPtr &self = stack_get(v, idx);
02186     SQObjectPtr &mt = v->GetUp(-1);
02187     SQObjectType type = sq_type(self);
02188     switch(type) {
02189     case OT_TABLE:
02190         if(sq_type(mt) == OT_TABLE) {
02191             if(!_table(self)->SetDelegate(_table(mt))) {
02192                 return sq_throwerror(v, _SC("delegate cycle"));
02193             }
02194         }
02195     }

```



```

02195     }
02196     else if(sq_type(mt)==OT_NULL) {
02197         _table(self)->SetDelegate(NULL); v->Pop(); }
02198     else return sq_aux_invalidtype(v,type);
02199     break;
02200 case OT_USERDATA:
02201     if(sq_type(mt)==OT_TABLE) {
02202         _userdata(self)->SetDelegate(_table(mt)); v->Pop(); }
02203     else if(sq_type(mt)==OT_NULL) {
02204         _userdata(self)->SetDelegate(NULL); v->Pop(); }
02205     else return sq_aux_invalidtype(v, type);
02206     break;
02207 default:
02208     return sq_aux_invalidtype(v, type);
02209     break;
02210 }
02211 return SQ_OK;
02212 }
02213
02226 SQRRESULT sq_rawdeleteslot(HSQIRRELVM v,SQInteger idx,SQBool pushval)
02227 {
02228     sq_aux_paramscheck(v, 2);
02229     SQObjectPtr *self;
02230     _GETSAFE_OBJ(v, idx, OT_TABLE,self);
02231     SQObjectPtr &key = v->GetUp(-1);
02232     SQObjectPtr t;
02233     if(_table(*self)->Get(key,t)) {
02234         _table(*self)->Remove(key);
02235     }
02236     if(pushval != 0)
02237         v->GetUp(-1) = t;
02238     else
02239         v->Pop();
02240     return SQ_OK;
02241 }
02242
02255 SQRRESULT sq_getdelegate(HSQIRRELVM v,SQInteger idx)
02256 {
02257     SQObjectPtr &self=stack_get(v,idx);
02258     switch(sq_type(self)){
02259     case OT_TABLE:
02260     case OT_USERDATA:
02261         if(!_delegable(self)->_delegate){
02262             v->PushNull();
02263             break;
02264         }
02265         v->Push(SQObjectPtr(_delegable(self)->_delegate));
02266         break;
02267     default: return sq_throwerror(v,_SC("wrong type")); break;
02268     }
02269     return SQ_OK;
02270 }
02271 }
02272
02285 SQRRESULT sq_get(HSQIRRELVM v,SQInteger idx)
02286 {
02287     SQObjectPtr &self=stack_get(v,idx);
02288     SQObjectPtr &obj = v->GetUp(-1);
02289     if(v->Get(self,obj,obj,false,DONT_FALL_BACK))
02290         return SQ_OK;
02291     v->Pop();
02292     return SQ_ERROR;
02293 }
02294
02307 SQRRESULT sq_rawget(HSQIRRELVM v,SQInteger idx)
02308 {
02309     SQObjectPtr &self=stack_get(v,idx);
02310     SQObjectPtr &obj = v->GetUp(-1);
02311     switch(sq_type(self)) {
02312     case OT_TABLE:
02313         if(_table(self)->Get(obj,obj))
02314             return SQ_OK;
02315         break;
02316     case OT_CLASS:
02317         if(_class(self)->Get(obj,obj))
02318             return SQ_OK;
02319         break;
02320     case OT_INSTANCE:
02321         if(_instance(self)->Get(obj,obj))
02322             return SQ_OK;
02323         break;
02324     case OT_ARRAY:{
02325         if(sq_isnumeric(obj)){
02326             if(_array(self)->Get(tointeger(obj),obj)) {
02327                 return SQ_OK;
02328             }
02329         }
02330     }

```

```

02330         else {
02331             v->Pop();
02332             return sq_throwerror(v,_SC("invalid index type for an array"));
02333         }
02334     }
02335     break;
02336 default:
02337     v->Pop();
02338     return sq_throwerror(v,_SC("rawget works only on array/table/instance and class"));
02339 }
02340 v->Pop();
02341 return sq_throwerror(v,_SC("the index doesn't exist"));
02342 }
02343
02356 SQRESULT sq_getstackobj(HSQIRRELV v,SQInteger idx,HSQOBJECT *po)
02357 {
02358     *po=stack_get(v,idx);
02359     return SQ_OK;
02360 }
02361
02374 const SQChar *sq_getlocal(HSQIRRELV v,SQUnsignedInteger level,SQUnsignedInteger idx)
02375 {
02376     SQUnsignedInteger cstksize=v->_callsstacksize;
02377     SQUnsignedInteger lvl=(cstksize-level)-1;
02378     SQInteger stackbase=v->_stackbase;
02379     if(lvl<cstksize){
02380         for(SQUnsignedInteger i=0;i<level;i++){
02381             SQVM::CallInfo &ci=v->_callsstack[(cstksize-i)-1];
02382             stackbase=ci._prevstkbase;
02383         }
02384         SQVM::CallInfo &ci=v->_callsstack[lvl];
02385         if(sq_type(ci._closure)!=OT_CLOSURE)
02386             return NULL;
02387         SQClosure *c=_closure(ci._closure);
02388         SQFunctionProto *func=c->_function;
02389         if(func->_noutervalues > (SQInteger)idx) {
02390             v->Push(*_outer(c->_outervalues[idx])->_valptr);
02391             return _stringval(func->_outervalues[idx]._name);
02392         }
02393         idx -= func->_noutervalues;
02394         return func->GetLocal(v,stackbase,idx,(SQInteger)(ci._ip-func->_instructions)-1);
02395     }
02396     return NULL;
02397 }
02398
02408 void sq_pushobject(HSQIRRELV v,HSQOBJECT obj)
02409 {
02410     v->Push(SQObjectPtr(obj));
02411 }
02412
02422 void sq_resetobject(HSQOBJECT *po)
02423 {
02424     po->_unVal.pUserPointer=NULL;
02425     po->_type=OT_NULL;
02426 }
02427
02439 SQRESULT sq_throwerror(HSQIRRELV v,const SQChar *err)
02440 {
02441     v->_lasterror=SQString::Create(_ss(v),err);
02442     return SQ_ERROR;
02443 }
02444
02455 SQRESULT sq_throwobject(HSQIRRELV v)
02456 {
02457     v->_lasterror = v->GetUp(-1);
02458     v->Pop();
02459     return SQ_ERROR;
02460 }
02461
02471 void sq_resetererror(HSQIRRELV v)
02472 {
02473     v->_lasterror.Null();
02474 }
02475
02485 void sq_getlasterror(HSQIRRELV v)
02486 {
02487     v->Push(v->_lasterror);
02488 }
02489
02502 SQRESULT sq_reservestack(HSQIRRELV v,SQInteger nsize)
02503 {
02504     if (((SQUnsignedInteger)v->_top + nsize) > v->_stack.size()) {
02505         if(v->_nmetamethodscall) {
02506             return sq_throwerror(v,_SC("cannot resize stack while in a metamethod"));
02507         }
02508         v->_stack.resize(v->_stack.size() + ((v->_top + nsize) - v->_stack.size()));
02509     }

```

```

02510     return SQ_OK;
02511 }
02512
02525 SQRRESULT sq_resume(HSQIRRELVM v, SQBool retval, SQBool raiseerror)
02526 {
02527     if (sq_type(v->GetUp(-1)) == OT_GENERATOR)
02528     {
02529         v->PushNull(); //retval
02530         if (!v->Execute(v->GetUp(-2), 0, v->_top, v->GetUp(-1), raiseerror,
02531             SQVM::ET_RESUME_GENERATOR))
02532         {v->Raise_Error(v->_lasterror); return SQ_ERROR;}
02533         if (!retval)
02534             v->Pop();
02535         return SQ_OK;
02536     }
02537     return sq_throwerror(v, _SC("only generators can be resumed"));
02538 }
02539
02554 SQRRESULT sq_call(HSQIRRELVM v, SQInteger params, SQBool retval, SQBool raiseerror)
02555 {
02556     SQObjectPtr res;
02557     if (!v->Call(v->GetUp(-(params+1)), params, v->_top-params, res, raiseerror?true:false)) {
02558         v->Pop(params); //pop args
02559         return SQ_ERROR;
02560     }
02561     if (!v->_suspended)
02562         v->Pop(params); //pop args
02563     if (retval)
02564         v->Push(res); // push result
02565     return SQ_OK;
02566 }
02567
02580 SQRRESULT sq_tailcall(HSQIRRELVM v, SQInteger nparams)
02581 {
02582     SQObjectPtr &res = v->GetUp(-(nparams + 1));
02583     if (sq_type(res) != OT_CLOSURE) {
02584         return sq_throwerror(v, _SC("only closure can be tail called"));
02585     }
02586     SQClosure *clo = _closure(res);
02587     if (clo->_function->_bgenerator)
02588     {
02589         return sq_throwerror(v, _SC("generators cannot be tail called"));
02590     }
02591     SQInteger stackbase = (v->_top - nparams) - v->_stackbase;
02592     if (!v->TailCall(clo, stackbase, nparams)) {
02593         return SQ_ERROR;
02594     }
02595     return SQ_TAILCALL_FLAG;
02596 }
02597
02598
02609 SQRRESULT sq_suspendvm(HSQIRRELVM v)
02610 {
02611     return v->Suspend();
02612 }
02613
02629 SQRRESULT sq_wakeupvm(HSQIRRELVM v, SQBool wakeupret, SQBool retval, SQBool raiseerror, SQBool throwerror)
02630 {
02631     SQObjectPtr ret;
02632     if (!v->_suspended)
02633         return sq_throwerror(v, _SC("cannot resume a vm that is not running any code"));
02634     SQInteger target = v->_suspended_target;
02635     if (wakeupret) {
02636         if (target != -1) {
02637             v->GetAt(v->_stackbase+v->_suspended_target)=v->GetUp(-1); //retval
02638         }
02639         v->Pop();
02640     } else if (target != -1) { v->GetAt(v->_stackbase+v->_suspended_target).Null(); }
02641     SQObjectPtr dummy;
02642     if (!v->Execute(dummy, -1, -1, ret, raiseerror, throwerror?SQVM::ET_RESUME_THROW_VM :
02643         SQVM::ET_RESUME_VM)) {
02644         return SQ_ERROR;
02645     }
02646     if (retval)
02647         v->Push(ret);
02648     return SQ_OK;
02649 }
02650
02661 void sq_setreleasehook(HSQIRRELVM v, SQInteger idx, SQRELEASEHOOK hook)
02662 {
02663     SQObjectPtr &ud=stack_get(v, idx);
02664     switch(sq_type(ud)) {
02665         case OT_USERDATA: _userdata(ud)->_hook = hook; break;
02666         case OT_INSTANCE: _instance(ud)->_hook = hook; break;
02667         case OT_CLASS: _class(ud)->_hook = hook; break;
02668         default: return;
02669     }

```

```

02670 }
02671
02683 SQRELEASEHOOK sq_getreleasehook(HSQIRRELVM v,SQInteger idx)
02684 {
02685     SQObjectPtr &ud=stack_get(v,idx);
02686     switch(sq_type(ud) ) {
02687         case OT_USERDATA: return _userdata(ud)->_hook; break;
02688         case OT_INSTANCE: return _instance(ud)->_hook; break;
02689         case OT_CLASS: return _class(ud)->_hook; break;
02690         default: return NULL;
02691     }
02692 }
02693
02704 void sq_setcompilererrorhandler(HSQIRRELVM v,SQCOMPILERERROR f)
02705 {
02706     _ss(v)->_compilererrorhandler = f;
02707 }
02708
02722 SQRESULT sq_writeclosure(HSQIRRELVM v,SQWRITEFUNC w,SQUserPointer up)
02723 {
02724     SQObjectPtr *o = NULL;
02725     _GETSAFE_OBJ(v, -1, OT_CLOSURE,o);
02726     unsigned short tag = SQ_BYTECODE_STREAM_TAG;
02727     if(_closure(*o)->_function->_noutervalues)
02728         return sq_throwerror(v,_SC("a closure with free variables bound cannot be serialized"));
02729     if(w(up,&tag,2) != 2)
02730         return sq_throwerror(v,_SC("io error"));
02731     if(!_closure(*o)->Save(v,up,w))
02732         return SQ_ERROR;
02733     return SQ_OK;
02734 }
02735
02749 SQRESULT sq_readclosure(HSQIRRELVM v,SQREADFUNC r,SQUserPointer up)
02750 {
02751     SQObjectPtr closure;
02752
02753     unsigned short tag;
02754     if(r(up,&tag,2) != 2)
02755         return sq_throwerror(v,_SC("io error"));
02756     if(tag != SQ_BYTECODE_STREAM_TAG)
02757         return sq_throwerror(v,_SC("invalid stream"));
02758     if(!SQClosure::Load(v,up,r,closure))
02759         return SQ_ERROR;
02760     v->Push(closure);
02761     return SQ_OK;
02762 }
02763
02775 SQChar *sq_getscratchpad(HSQIRRELVM v,SQInteger minsize)
02776 {
02777     return _ss(v)->GetScratchPad(minsize);
02778 }
02779
02791 SQRESULT sq_resurrectunreachable(HSQIRRELVM v)
02792 {
02793     #ifndef NO_GARBAGE_COLLECTOR
02794         _ss(v)->ResurrectUnreachable(v);
02795         return SQ_OK;
02796     #else
02797         return sq_throwerror(v,_SC("sq_resurrectunreachable requires a garbage collector build"));
02798     #endif
02799 }
02800
02811 SQInteger sq_collectgarbage(HSQIRRELVM v)
02812 {
02813     #ifndef NO_GARBAGE_COLLECTOR
02814         return _ss(v)->CollectGarbage(v);
02815     #else
02816         return -1;
02817     #endif
02818 }
02819
02831 SQRESULT sq_getcallee(HSQIRRELVM v)
02832 {
02833     if(v->_callsstacksize > 1)
02834     {
02835         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
02836         return SQ_OK;
02837     }
02838     return sq_throwerror(v,_SC("no closure in the calls stack"));
02839 }
02840
02853 const SQChar *sq_getfreevariable(HSQIRRELVM v,SQInteger idx,SQUnsignedInteger nval)
02854 {
02855     SQObjectPtr &self=stack_get(v,idx);
02856     const SQChar *name = NULL;
02857     switch(sq_type(self))
02858     {

```

```

02859     case OT_CLOSURE:{
02860         SQClosure *clo = _closure(self);
02861         SQFunctionProto *fp = clo->_function;
02862         if(((SQUnsignedInteger)fp->noutervalues) > nval) {
02863             v->Push(*(_outer(clo->_outervalues[nval])->_valptr));
02864             SQOuterVar &ov = fp->_outervalues[nval];
02865             name = _stringval(ov._name);
02866         }
02867     }
02868     break;
02869     case OT_NATIVECLOSURE:{
02870         SQNativeClosure *clo = _nativeclosure(self);
02871         if(clo->noutervalues > nval) {
02872             v->Push(clo->_outervalues[nval]);
02873             name = _SC("@NATIVE");
02874         }
02875     }
02876     break;
02877     default: break; //shutup compiler
02878 }
02879 return name;
02880 }
02881
02895 SQRRESULT sq_setfreevariable(HSQIRRELVM v,SQInteger idx,SQUnsignedInteger nval)
02896 {
02897     SQObjectPtr &self=stack_get(v,idx);
02898     switch(sq_type(self))
02899     {
02900     case OT_CLOSURE:{
02901         SQFunctionProto *fp = _closure(self)->_function;
02902         if(((SQUnsignedInteger)fp->noutervalues) > nval){
02903             *(_outer(_closure(self)->_outervalues[nval])->_valptr) = stack_get(v,-1);
02904         }
02905         else return sq_throwerror(v,_SC("invalid free var index"));
02906     }
02907     break;
02908     case OT_NATIVECLOSURE:
02909         if(_nativeclosure(self)->noutervalues > nval){
02910             _nativeclosure(self)->_outervalues[nval] = stack_get(v,-1);
02911         }
02912         else return sq_throwerror(v,_SC("invalid free var index"));
02913         break;
02914     default:
02915         return sq_aux_invalidtype(v, sq_type(self));
02916     }
02917     v->Pop();
02918     return SQ_OK;
02919 }
02920
02933 SQRRESULT sq_setattributes(HSQIRRELVM v,SQInteger idx)
02934 {
02935     SQObjectPtr *o = NULL;
02936     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02937     SQObjectPtr &key = stack_get(v,-2);
02938     SQObjectPtr &val = stack_get(v,-1);
02939     SQObjectPtr attrs;
02940     if(sq_type(key) == OT_NULL) {
02941         attrs = _class(*o)->_attributes;
02942         _class(*o)->_attributes = val;
02943         v->Pop(2);
02944         v->Push(attrs);
02945         return SQ_OK;
02946     }else if(_class(*o)->GetAttributes(key,attrs)) {
02947         _class(*o)->SetAttributes(key,val);
02948         v->Pop(2);
02949         v->Push(attrs);
02950         return SQ_OK;
02951     }
02952     return sq_throwerror(v,_SC("wrong index"));
02953 }
02954
02967 SQRRESULT sq_getattributes(HSQIRRELVM v,SQInteger idx)
02968 {
02969     SQObjectPtr *o = NULL;
02970     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
02971     SQObjectPtr &key = stack_get(v,-1);
02972     SQObjectPtr attrs;
02973     if(sq_type(key) == OT_NULL) {
02974         attrs = _class(*o)->_attributes;
02975         v->Pop();
02976         v->Push(attrs);
02977         return SQ_OK;
02978     }
02979     else if(_class(*o)->GetAttributes(key,attrs)) {
02980         v->Pop();
02981         v->Push(attrs);
02982         return SQ_OK;

```

```

02983     }
02984     return sq_throwerror(v,_SC("wrong index"));
02985 }
02986
03000 SQRRESULT sq_getmemberhandle(HSQIRRELVM v,SQInteger idx,HSQMEMBERHANDLE *handle)
03001 {
03002     SQObjectPtr *o = NULL;
03003     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03004     SQObjectPtr &key = stack_get(v,-1);
03005     SQTable *m = _class(*o)->_members;
03006     SQObjectPtr val;
03007     if(m->Get(key,val)) {
03008         handle->_static = _isfield(val) ? SQFalse : SQTrue;
03009         handle->_index = _member_idx(val);
03010         v->Pop();
03011         return SQ_OK;
03012     }
03013     return sq_throwerror(v,_SC("wrong index"));
03014 }
03015
03030 SQRRESULT _getmemberbyhandle(HSQIRRELVM v,SQObjectPtr &self,const HSQMEMBERHANDLE *handle,SQObjectPtr
*&val)
03031 {
03032     switch(sq_type(self)) {
03033         case OT_INSTANCE: {
03034             SQInstance *i = _instance(self);
03035             if(handle->_static) {
03036                 SQClass *c = i->_class;
03037                 val = &c->_methods[handle->_index].val;
03038             }
03039             else {
03040                 val = &i->_values[handle->_index];
03041             }
03042         }
03043         break;
03044         case OT_CLASS: {
03045             SQClass *c = _class(self);
03046             if(handle->_static) {
03047                 val = &c->_methods[handle->_index].val;
03048             }
03049             else {
03050                 val = &c->_defaultvalues[handle->_index].val;
03051             }
03052         }
03053         break;
03054         default:
03055             return sq_throwerror(v,_SC("wrong type(expected class or instance)"));
03056     }
03057     return SQ_OK;
03058 }
03059
03074 SQRRESULT sq_getbyhandle(HSQIRRELVM v,SQInteger idx,const HSQMEMBERHANDLE *handle)
03075 {
03076     SQObjectPtr &self = stack_get(v,idx);
03077     SQObjectPtr *val = NULL;
03078     if(SQ_FAILED(_getmemberbyhandle(v,self,handle,val))) {
03079         return SQ_ERROR;
03080     }
03081     v->Push(_realval(*val));
03082     return SQ_OK;
03083 }
03084
03098 SQRRESULT sq_setbyhandle(HSQIRRELVM v,SQInteger idx,const HSQMEMBERHANDLE *handle)
03099 {
03100     SQObjectPtr &self = stack_get(v,idx);
03101     SQObjectPtr &newval = stack_get(v,-1);
03102     SQObjectPtr *val = NULL;
03103     if(SQ_FAILED(_getmemberbyhandle(v,self,handle,val))) {
03104         return SQ_ERROR;
03105     }
03106     *val = newval;
03107     v->Pop();
03108     return SQ_OK;
03109 }
03110
03122 SQRRESULT sq_getbase(HSQIRRELVM v,SQInteger idx)
03123 {
03124     SQObjectPtr *o = NULL;
03125     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03126     if(_class(*o)->_base)
03127         v->Push(SQObjectPtr(_class(*o)->_base));
03128     else
03129         v->PushNull();
03130     return SQ_OK;
03131 }
03132

```

```

03144 SQRESULT sq_getclass(HSQIRRELVM v,SQInteger idx)
03145 {
03146     SQObjectPtr *o = NULL;
03147     _GETSAFE_OBJ(v, idx, OT_INSTANCE,o);
03148     v->Push(SQObjectPtr(_instance(*o)->_class));
03149     return SQ_OK;
03150 }
03151
03163 SQRESULT sq_createinstance(HSQIRRELVM v,SQInteger idx)
03164 {
03165     SQObjectPtr *o = NULL;
03166     _GETSAFE_OBJ(v, idx, OT_CLASS,o);
03167     v->Push(_class(*o)->CreateInstance());
03168     return SQ_OK;
03169 }
03170
03181 void sq_weakref(HSQIRRELVM v,SQInteger idx)
03182 {
03183     SQObject &o=stack_get(v,idx);
03184     if (ISREFCOUNTED(sq_type(o))) {
03185         v->Push(_refcounted(o)->GetWeakRef(sq_type(o)));
03186         return;
03187     }
03188     v->Push(o);
03189 }
03190
03203 SQRESULT sq_getweakrefval(HSQIRRELVM v,SQInteger idx)
03204 {
03205     SQObjectPtr &o = stack_get(v,idx);
03206     if(sq_type(o) != OT_WEAKREF) {
03207         return sq_throwerror(v,_SC("the object must be a weakref"));
03208     }
03209     v->Push(_weakref(o)->_obj);
03210     return SQ_OK;
03211 }
03212
03225 SQRESULT sq_getdefaultdelegate(HSQIRRELVM v,SQObjectType t)
03226 {
03227     SQSharedState *ss = _ss(v);
03228     switch(t) {
03229     case OT_TABLE: v->Push(ss->_table_default_delegate); break;
03230     case OT_ARRAY: v->Push(ss->_array_default_delegate); break;
03231     case OT_STRING: v->Push(ss->_string_default_delegate); break;
03232     case OT_INTEGER: case OT_FLOAT: v->Push(ss->_number_default_delegate); break;
03233     case OT_GENERATOR: v->Push(ss->_generator_default_delegate); break;
03234     case OT_CLOSURE: case OT_NATIVECLOSURE: v->Push(ss->_closure_default_delegate); break;
03235     case OT_THREAD: v->Push(ss->_thread_default_delegate); break;
03236     case OT_CLASS: v->Push(ss->_class_default_delegate); break;
03237     case OT_INSTANCE: v->Push(ss->_instance_default_delegate); break;
03238     case OT_WEAKREF: v->Push(ss->_weakref_default_delegate); break;
03239     default: return sq_throwerror(v,_SC("the type doesn't have a default delegate"));
03240     }
03241     return SQ_OK;
03242 }
03243
03257 SQRESULT sq_next(HSQIRRELVM v,SQInteger idx)
03258 {
03259     SQObjectPtr o=stack_get(v,idx),&refpos = stack_get(v,-1),realkey,val;
03260     if(sq_type(o) == OT_GENERATOR) {
03261         return sq_throwerror(v,_SC("cannot iterate a generator"));
03262     }
03263     int faketojump;
03264     if(!v->FOREACH_OP(o,realkey,val,refpos,0,666,faketojump))
03265         return SQ_ERROR;
03266     if(faketojump != 666) {
03267         v->Push(realkey);
03268         v->Push(val);
03269         return SQ_OK;
03270     }
03271     return SQ_ERROR;
03272 }
03273
03282 struct BufState{
03283     const SQChar *buf;
03284     SQInteger ptr;
03285     SQInteger size;
03286 };
03287
03298 SQInteger buf_lexfeed(SQUserPointer file)
03299 {
03300     BufState *buf=(BufState*) file;
03301     if(buf->size<(buf->ptr+1))
03302         return 0;
03303     return buf->buf[buf->ptr++];
03304 }
03305
03321 SQRESULT sq_compilebuffer(HSQIRRELVM v,const SQChar *s,SQInteger size,const SQChar *sourcename,SQBool

```

```
        raiseerror) {
03322     BufState buf;
03323     buf.buf = s;
03324     buf.size = size;
03325     buf.ptr = 0;
03326     return sq_compile(v, buf_lexfeed, &buf, sourcename, raiseerror);
03327 }
03328
03340 void sq_move(HSQIRRELVM dest,HSQIRRELVM src,SQInteger idx)
03341 {
03342     dest->Push(stack_get(src,idx));
03343 }
03344
03356 void sq_setprintfunc(HSQIRRELVM v, SQPRINTFUNCTION printfunc,SQPRINTFUNCTION errfunc)
03357 {
03358     _ss(v)->_printfunc = printfunc;
03359     _ss(v)->_errorfunc = errfunc;
03360 }
03361
03371 SQPRINTFUNCTION sq_getprintfunc(HSQIRRELVM v)
03372 {
03373     return _ss(v)->_printfunc;
03374 }
03375
03385 SQPRINTFUNCTION sq_geterrorfunc(HSQIRRELVM v)
03386 {
03387     return _ss(v)->_errorfunc;
03388 }
03389
03400 void *sq_malloc(SQUnsignedInteger size)
03401 {
03402     return SQ_MALLOC(size);
03403 }
03404
03417 void *sq_realloc(void* p,SQUnsignedInteger oldsize,SQUnsignedInteger newsize)
03418 {
03419     return SQ_REALLOC(p,oldsize,newsize);
03420 }
03421
03432 void sq_free(void *p,SQUnsignedInteger size)
03433 {
03434     SQ_FREE(p,size);
03435 }
```